ANALYSE DE LA DYNAMIQUE D'APPRENTISSAGE DES RÉSEAUX DE NEURONES PROFONDS À TRAVERS LE PRISME DU NTK

Deeper or Wider ? A 1st answer under the NTK perspective

20 août 2025

Janis AIAD

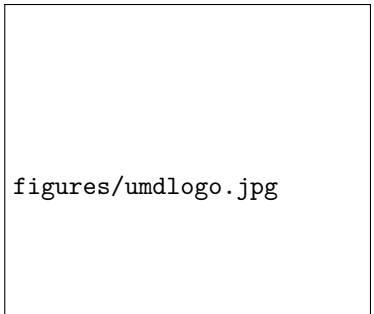


TABLE DES MATIÈRES

ABSTRACT

This report presents the results of a research internship focused on studying the learning dynamics of deep neural networks. We explore the underlying mechanisms through the Neural Tangent Kernel (NTK) formalism, with particular attention to finite-width corrections. Our work revolves around the decomposition of the NTK via Sobolev training, the analysis of its reproducing kernel Hilbert space (RKHS), and the study of extreme eigenvalues. We present theoretical bounds for the smallest eigenvalue of the NTK using random matrix theory (RMT) techniques. These results are then applied to analyze different architectures, including feed-forward neural networks (FCNN), matrix memory neural networks (MMNN), and transformers. We also discuss the implications of our findings for scientific machine learning (SciML) applications and propose directions for future research.

INTRODUCTION

The study of learning dynamics in deep neural networks constitutes a fundamental research area. Our approach focuses on the desingularization of Sobolev training by decomposing the problem into two distinct contributions : an independent neural tangent kernel (NTK) and a Sobolev contribution. This decomposition relies on the commutation between integral operators and the fractional Laplace operator.

Our analysis continues with the study of NTK preconditioning through a freeze weights method, illustrated by neural networks factorizing low-rank weight matrices. This approach is supported by a rigorous approximation theorem. For classical feed-forward networks (FCNN), we develop a finite-width correction that proves particularly fruitful. Finally, we explore Transformers as an alternative solution, offering a new learning paradigm.

Across these different architectures, we study feature learning regimes and their scaling with network width. Our results rely on statistical physics tools to characterize model convergence and asymptotic behavior. This analysis enables a better understanding of finite-width corrections and their implications for the practical optimization of deep neural networks.

NOTATIONS

$\_\!\!/$: The ReLU (Rectified Linear Unit) activation function, defined as $\_\!\!/(x) = \max(0, x)$.
$\mathbb{E}[\cdot]$: Mathematical expectation.
$\mathrm{Var}(\cdot)$: Variance of a random variable.
$\mathbb{1}\{\cdot\}$: Indicator function, equal to 1 if the condition is true, 0 otherwise.
dx : Differential element used in integrals.
$(\cdot, \cdot)$: Covariance between two random variables.
$(\cdot, \cdot)$: Correlation between two random variables.
$\mathbb{R}$: Set of real numbers.
$\mathbb{N}$: Set of natural numbers.
$\mathbb{S}^{d-1}$: Unit sphere of dimension $d-1$.
∇ : Gradient operator.
Δ : Laplacian operator.
$\|\cdot\|$: Euclidean norm.
$\langle \cdot, \cdot \rangle$: Inner product.
$\mathcal{O}(\cdot)$: Landau notation (big O).
$\tilde{\Omega}(\cdot)$: Landau notation (big Omega tilde).

1

THEORETICAL FRAMEWORK OF THE NEURAL TANGENT KERNEL

1.1 Introduction : Eigenvalue Scaling Laws and Learning Dynamics

1.1.1 • The Neural Tangent Kernel

The Neural Tangent Kernel (NTK) has emerged as a fundamental tool for understanding the training dynamics of deep neural networks. For a neural network $f(;\theta)$ with parameters θ, the NTK is defined as :

$$K^{\infty}(x_i, x_j) = \left\langle \frac{\partial f(_i;\theta)}{\partial \theta}, \frac{\partial f(_j;\theta)}{\partial \theta} \right\rangle = \sum_k \frac{\partial f(_i;\theta)}{\partial \theta_k} \frac{\partial f(_j;\theta)}{\partial \theta_k}$$

For L2 gradient descent with learning rate η, the dynamics are given by :

$$\frac{d}{dt} f_t(x) = -\eta \sum_{i=1}^{n} K^{\infty}(x, x_i)(f_t(x_i) - y_i)$$

The following theorem from Jacot et al. [**jacot2020neuraltangentkernelconvergence**] establishes the convergence of neural networks to their linearized version in the infinite width limit, along with the recursive kernel formulation :

[Neural Tangent Kernel, Jacot et al. 2018] Consider a neural network $f(;\theta)$ trained by gradient descent with infinitesimal learning rate. As the width of the hidden layers tends to infinity :

1) The network converges to a Gaussian process with kernel K^{∞}, with dynamics :

$$\frac{\partial f_t()}{\partial t} = -\eta \sum_{i=1}^{n} K^{\infty}(,_i)(f_t(_i) - y_i)$$

2) The kernel follows the recursive formula for depth l :

$$K^{(l)}(,') = \sigma_w^2 \cdot K^{(l-1)}(,') \cdot \mathbb{E}_{g\sim\mathcal{N}(0,\Sigma^{(l-1)})}[\phi'(g())\phi'(g('))]$$

where $\Sigma^{(l)}$ is the covariance of pre-activations at layer l :

$$\Sigma^{(l)}(,') = \sigma_w^2 \cdot \mathbb{E}_{g\sim\mathcal{N}(0,\Sigma^{(l-1)})}[\phi(g())\phi(g('))]$$

with $\Sigma^{(0)}(,') = {}^{\top}{}'$ and ϕ the activation function.

3) For ReLU networks, the correlation function $\dot{\Lambda}^{(l)}$ between gradients at layer l is :

$$\dot{\Lambda}^{(l)}(\rho) = \frac{1}{2\pi}\sqrt{1-\rho^2} + \frac{\rho}{2\pi}(\pi - \arccos(\rho))$$

where $\rho = \frac{\Sigma^{(l)}(,')}{\sqrt{\Sigma^{(l)}(,)\Sigma^{(l)}('.')}}$

1.1.2 • The Sobolev Training

Sobolev training is an approach that incorporates derivative information into the learning process. For a function f and target function f^*, the Sobolev training loss is defined as :

$$\mathcal{L}_{\text{Sobolev}}(f) = \|f - f^*\|_{H^s}^2 = \sum_{k=0}^{s} \int_\Omega |D^k f(x) - D^k f^*(x)|^2 dx$$

where D^k denotes the k-th derivative operator and H^s is the Sobolev space of order s. This formulation leads to several key properties :

1) The loss incorporates both function values and derivatives up to order s 2) Training in H^s norm induces smoother function approximation 3) The NTK eigenvalues exhibit polynomial decay related to the Sobolev order s

For neural networks, this translates to augmenting the training data with derivative information :

$$\mathcal{L}_{\text{discrete}}(f) = \sum_{i=1}^{n} \sum_{k=0}^{s} |D^k f(x_i) - D^k f^*(x_i)|^2$$

This framework provides a natural connection between neural network training and classical function approximation theory in Sobolev spaces.

1.1.3 • Two Equivalent Sobolev Norms

The Sobolev space $H^s(\Omega)$ can be equipped with two equivalent norms :

1) **Gradient-Based Norm** : Defined using derivatives up to order s :

$$\|f\|_{H^s}^2 = \sum_{k=0}^{s} \int_\Omega |D^k f(x)|^2 dx$$

where D^k represents the k-th derivative operator.

2) **Fourier-Based Norm** : Defined using the Fourier transform $\hat{f}$:

$$\|f\|_{H^s}^2 = \int_{\mathbb{R}^d} (1 + |\xi|^2)^s |\hat{f}(\xi)|^2 d\xi$$

where ξ represents the frequency variable.

These norms are equivalent in the sense that there exist constants $c, C > 0$ such that :

$$c\|f\|_{H^s,\text{grad}} \leq \|f\|_{H^s,\text{Four}} \leq C\|f\|_{H^s,\text{grad}}$$

The gradient-based norm is more intuitive for implementation in neural network training, while the Fourier-based norm provides a natural connection to spectral properties and eigenvalue decay rates of the NTK operator.

1.2 The NN regression framework

In our analysis, we consider two distinct learning frameworks :

1) **Standard Neural Network Regression** : Here we only observe output values y_i for inputs x_i (as in Fourier Neural Operators for instance), leading to the classical mean squared error loss :

$$\mathcal{L}_{\text{MSE}}(f) = \sum_{i=1}^{n} |f(x_i) - y_i|^2$$

The gradient descent dynamics follow :

$$\frac{d}{dt} f_t(x) = -\eta \sum_{i=1}^{n} K^{\infty}(x, x_i)(f_t(x_i) - y_i)$$

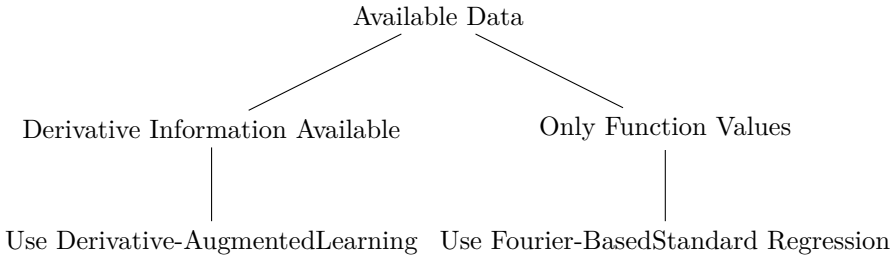
2) **Derivative-Augmented Learning** : We observe both outputs and derivatives, as in Sobolev training and Physics-Informed Neural Networks (PINNs) or DeepONet architectures, with loss :

$$\mathcal{L}_{\text{deriv}}(f) = \sum_{i=1}^{n} \left(|f(x_i) - y_i|^2 + \sum_{k=1}^{s} |D^k f(x_i) - D^k y_i|^2 \right)$$

Leading to modified gradient dynamics incorporating derivative information.

These frameworks result in different gradient descent trajectories and convergence behaviors, despite sharing the same underlying NTK structure. The choice between them depends on whether derivative information is available and beneficial for the specific learning task.

The framework selection can be summarized by the following decision tree :



1.3 Key Objectives

This document addresses three fundamental objectives : deriving decay rates $\mu_\ell \sim \ell^{-\alpha}$ for NTK operator eigenvalues, understanding how spectral properties determine learning dynamics, and analyzing scaling laws for discrete matrix eigenvalues with respect to network depth l and data size n. A crucial distinction exists between the NTK Matrix (discrete sampled version $K \in \mathbb{R}^{n \times n}$ with entries $K_{ij} = K^{\infty}(x_i, x_j)$) and the NTK Operator (continuous integral operator $(\mathcal{L}f)(x) = \int K^{\infty}(x, y) f(y) dy$). It is important to note that the eigenvalues of the sampled NTK matrix are *not* the same as the NTK operator eigenvalues - the matrix spectrum provides discrete approximations that depend on the sampling strategy and data distribution. All analysis assumes Edge of Chaos (EOC) initialization, where weights are initialized as $w_{ij} \sim \mathcal{N}(0, \sigma_w^2/\text{fan-in})$, with $\sigma_w^2 = 2$ for ReLU networks to maintain unit variance through layers, ensuring activations neither explode nor vanish with depth.

2
NTK MATRIX STRUCTURE AND SPECTRUM

2.1 Example NTK Matrix Structure

Consider a dataset of n points $x_1, x_2, \ldots, x_n \in^d$. The NTK matrix has entries :

$$K^\infty = D{\cdot}R{\cdot}D = \begin{pmatrix} \|x_1\| & 0 & \cdots & 0 \\ 0 & \|x_2\| & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \|x_n\| \end{pmatrix} \begin{pmatrix} \tilde{K}(\rho_{1,1}) & \tilde{K}(\rho_{1,2}) & \cdots & \tilde{K}(\rho_{1,n}) \\ \tilde{K}(\rho_{2,1}) & \tilde{K}(\rho_{2,2}) & \cdots & \tilde{K}(\rho_{2,n}) \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{K}(\rho_{n,1}) & \tilde{K}(\rho_{n,2}) & \cdots & \tilde{K}(\rho_{n,n}) \end{pmatrix} \begin{pmatrix} \|x_1\| & 0 & \cdots & 0 \\ 0 & \|x_2\| & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \|x_n\| \end{pmatrix}$$

where $\rho_{i,j} = \frac{x_i^T x_j}{\|x_i\|\|x_j\|}$ represents the cosine distance between points x_i and x_j. For general depth L networks at EOC with bi-homogeneous leaky ReLU activation $\phi(x) = a \cdot \mathrm{id}(x) + b \cdot |x|$, where $a, b \in \mathbb{R}$ are fixed parameters, the NTK kernel function is :

$$\tilde{K}^\infty(\rho) = \left(\sum_{k=1}^{L} \varrho^{\circ(k-1)}(\rho) \prod_{k'=k+1}^{L} \varrho'(\varrho^{\circ(k'-2)}(\rho)) \right)$$

where ϱ and ϱ' are defined recursively through Gaussian expectations :

$$\begin{aligned} \rho_1(x_1, x_2) &= \frac{x_1^T x_2}{\|x_1\|\|x_2\|} \\ \varrho(\rho) &= \mathbb{E}_{[u_1,u_2]\sim\mathcal{N}(0,\Sigma_u)} \not{\_}(u_1)\, \not{\_}(u_2) \text{ where } \Sigma_u = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \\ \varrho'(\rho) &= \mathbb{E}_{[u_1,u_2]\sim\mathcal{N}(0,\Sigma_u)} \not{\_}'(u_1)\, \not{\_}'(u_2) \text{ where } \Sigma_u = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \\ \varrho^{\circ k}(\rho) &= \underbrace{\varrho(\varrho(\cdots \varrho}_{k \text{ times}}(\rho)\cdots)) \end{aligned}$$

for $k \in [2 : L]$, where $\sigma^2 = (a^2 + b^2)$ at the Edge of Chaos initialization and ϕ is the activation function.

For 2-layer ReLU networks, this simplifies to :

$$\tilde{K}^\infty(\rho) = \frac{1}{\pi}\left(\rho(\pi - \arccos(\rho)) + \sqrt{1-\rho^2}\right)$$

2.2 NTK Operator Structure and Spectral Analysis

The Neural Tangent Kernel operator K^∞ acts on $L^2(\mathbb{S}^{d-1})$ via :

$$(K^\infty f)(x) = \int_{\mathbb{S}^{d-1}} K^\infty(x, y) f(y) d\sigma(y)$$

By rotational invariance, the kernel is zonal :

$$K^\infty(x, y) = \tilde{K}^\infty(\langle x, y\rangle)$$

where $\langle x, y\rangle = \frac{x^T y}{\|x\|\|y\|}$ is the cosine distance.

We use fubini theorem using parametrization $y = \cos(\theta)x + \sin(\theta)u$ with $u \perp x$, $\|u\| = 1$, and setting $t = \cos(\theta)$:

$$(K^\infty f)(x) = \omega_{d-2} \int_{-1}^{1} \tilde{K}^\infty(t)\bar{f}(t)(1-t^2)^{(d-3)/2}dt$$

$$= \int_{\mathbb{S}^{d-1}} K^\infty(x,y)f(y)d\sigma(y) = \omega_{d-2} \int_{-1}^{1} \tilde{K}^\infty(t)\bar{f}(t)(1-t^2)^{(d-3)/2}dt$$

where ω_{d-2} is the surface area of $\mathbb{S}^{d-2}$ and $\bar{f}(t)$ represents the average of f over directions making angle $\arccos(t)$ with x.

Spectral Properties and Funk-Hecke Theory : The NTK operator possesses several important spectral properties that characterize its behavior on the sphere. The operator is symmetric positive definite by construction, guaranteeing the existence of a complete spectral decomposition with real, non-negative eigenvalues. The eigenfunctions are the spherical harmonics $Y_{\ell,p}$, which form a complete orthonormal basis of $L^2(\mathbb{S}^{d-1})$ and provide the natural coordinate system for analyzing functions on the sphere. Due to the zonal symmetry, the Funk-Hecke theorem applies, yielding eigenvalues μ_ℓ given by :

$$\mu_\ell = \frac{2\pi^{d/2}}{\Gamma(d/2)} \int_{-1}^{1} \tilde{K}^\infty(t) P_\ell^{((d-2)/2)}(t)(1-t^2)^{(d-3)/2}dt$$

where $P_\ell^{(\alpha)}$ are the Gegenbauer polynomials. The eigenvalues μ_ℓ exhibit polynomial decay characterized by $\mu_\ell \sim \ell^{-d}$, reflecting the smoothness and regularity properties of the kernel function. The multiplicity of each eigenvalue μ_ℓ is given by the dimension of the corresponding spherical harmonic space, which grows as $N(d,\ell) \sim \frac{2\ell^{d-2}}{(d-2)!}$ for large ℓ, indicating the increasing complexity of higher-order harmonics.

3

SOBOLEV TRAINING AND NTK SPECTRAL MODIFICATION

3.1 NTK-Sobolev Operator Framework

The key innovation in Sobolev training is the modification of the standard L^2 loss to incorporate high-order derivatives. The NTK-Sobolev operator P_s acts on the sphere $\mathbb{S}^{d-1}$ with the commutation property :

Spherical Harmonics Transform : Let $\mathcal{F}$ denote the mapping from $L^2(\mathbb{S}^d)$ to the spectral domain $\bigoplus_{\ell=0}^{\infty} \mathbb{C}^{N(d,\ell)}$, where $N(d,\ell)$ is the dimension of the ℓ-th spherical harmonic space.

The Sobolev operator P_s is defined through its action in Fourier space :

$$P_s = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} P_{\ell,p}$$

where $P_{\ell,p} = a_{\ell,p} a_{\ell,p}^T$ with spectral coefficients $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$.

3.2 From Sobolev Loss to Discrete Matrix Operator

Sobolev loss with gradients :

$$\mathcal{L}_s[f] = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2 = \int_{\mathbb{S}^d} f(x)^2 d\sigma(x) + \int_{\mathbb{S}^d} \|\nabla f(x)\|^2 d\sigma(x)$$

Fourier decomposition : For $f(x) = \sum_{\ell,p} \hat{f}_{\ell,p} Y_{\ell,p}(x)$:

$$\mathcal{L}_s[f] = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} |\hat{f}_{\ell,p}|^2$$

Discretization : At points $\{x_i\}_{i=1}^n$:

$$\hat{f}_{\ell,p} \approx \sum_{i=1}^{n} c_i f(x_i) Y_{\ell,p}(x_i)$$

Final matrix form : $\mathcal{L}_s[f] \approx f^T P_s f$ where $f = (f(x_1), \ldots, f(x_n))^T$.

3.3 Spectral Properties and Dimension Growth

The dimension of spherical harmonic spaces grows as :

$$N(d,\ell) \sim \frac{2\ell^{d-2}}{(d-2)!} \quad \text{as } \ell \to \infty$$

This exponential growth in dimension determines the computational complexity of the Sobolev operator discretization.

3.4 Common Eigenfunctions Property

[Spherical Harmonics as Common Eigenfunctions] For rotationally invariant kernels on $\mathbb{S}^{d-1}$:

- NTK operator K^∞ and Sobolev operator P_s share spherical harmonics $Y_{\ell,p}$ as eigenfunctions
- This is due to rotational invariance and Schur's lemma for irreducible representations
- Enables direct spectral modification analysis

[Commutation Property] The NTK operator K^∞ and Sobolev operator P_s commute :

$$[K^\infty, P_s] = 0$$

This holds for any sampling distribution $\rho(x)$ on $\mathbb{S}^{d-1}$ and implies that spectral decompositions are compatible.

4
DEEP NTK ANALYSIS AND THEORETICAL RESULTS

4.1 NTK at Edge of Chaos

For MLPs with (a, b)-ReLU activation $\phi(s) = as + b|s|$, the Edge of Chaos initialization requires :

- Initialization : $\sigma^2 = (a^2 + b^2)^{-1}$
- Key parameter : $\Delta_\phi = \frac{b^2}{a^2+b^2}$ (for standard ReLU : $\Delta_\phi = 0.5$)
- Cosine map : $\varrho(\rho) = \rho + \Delta_\phi \frac{2}{\pi}\left(\sqrt{1-\rho^2} - \rho \arccos(\rho)\right)$

The Edge of Chaos is the unique initialization that remains invariant to network depth, preventing activation explosion or vanishing.

5
RECONCILING NTK MATRIX SPECTRUM AND SOBOLEV TRAINING

5.1 A Unified Spectral Viewpoint : Commutation and Eigenvalue Products

The reconciliation between the geometric view of the NTK (approximated by the inverse cosine distance matrix) and the functional view of Sobolev training lies in their shared algebraic structure.

- **Shared Invariance** : Both the NTK matrix K and the discrete Sobolev operator matrix P_s are rotationally invariant. This means their entries $(K)_{ij}$ and $(P_s)_{ij}$ depend only on the inner product $\langle x_i, x_j \rangle$.
- **Matrix Commutation** : As a consequence, both matrices can be expressed as functions of the Gram matrix G (where $G_{ij} = \langle x_i, x_j \rangle$). For two such matrices, $K = f(G)$ and $P_s = g(G)$, they commute :

$$KP_s = f(G)g(G) = g(G)f(G) = P_sK$$

  This is the discrete analogue of the commutation of their underlying continuous operators.
- **Simultaneous Diagonalization** : Since they commute, K and P_s are simultaneously diagonalizable. They share the same set of eigenvectors, which are the eigenvectors of the Gram matrix G.
- **Product of Eigenvalues** : The spectrum of the Sobolev-modified NTK operator, KP_s, which governs the learning dynamics, is directly given by the product of the individual spectra :

$$\lambda_i(KP_s) = \lambda_i(K) \cdot \lambda_i(P_s)$$

  (after aligning the eigenvector bases).

Key Insight : The challenge of analyzing the complex operator KP_s is reduced to separately analyzing the spectra of K and P_s. We can leverage the geometric approximation for K and combine it with a spectral analysis of P_s.

5.2 A Path Forward : Geometric Approximation and Experimental Validation

Based on the unified spectral viewpoint, a clear research path emerges to develop a predictive model for Sobolev training dynamics.

Goal : Develop a semi-analytic model for the spectrum of the Sobolev-modified NTK matrix KP_s.

Proposed Strategy :

1. **Approximate the NTK Spectrum** : Use the established near-affine relationship between the NTK and the inverse cosine distance matrix W_l :

   $$K \approx AW_l + B$$

   The spectrum of the geometric matrix W_l can be analyzed to provide an approximation for the eigenvalues $\lambda_i(K)$.
2. **Analyze the Sobolev Operator Spectrum** : The matrix P_s is itself a kernel matrix with a well-defined kernel $p_s(t) = \sum_\ell (1+\ell)^{2s} N(d,\ell) P_\ell(t)$, where P_ℓ are Legendre polynomials. Its spectrum $\lambda_i(P_s)$ can be characterized :
   — Analytically, through its kernel polynomial expansion.
   — Or by viewing it as a differential operator on the data manifold (graph), relating its spectrum to that of the graph Laplacian.
3. **Combine Spectra and Validate** : The core prediction is that the eigenvalues of the learning operator are given by the product of the component eigenvalues :

   $$\lambda_i(KP_s) \approx \lambda_i(AW_l + B) \cdot \lambda_i(P_s)$$

   Experimental Validation Plan :
   — For a dataset on $\mathbb{S}^{d-1}$, numerically compute the matrices K, P_s, and the product KP_s.
   — Compare the analytically predicted spectrum with the true, numerically computed spectrum of KP_s.
   — Investigate the quality of this approximation as a function of the Sobolev parameter s, dimension d, data size n, and network depth l.

[The Sobolev Operator as a Kernel Matrix] The statement that the discrete Sobolev operator matrix P_s is itself a kernel matrix is a subtle but fundamental point. Here is a detailed derivation.

1. Definition of the Matrix P_s We start from its construction via projectors on the spherical harmonics :

$$(P_s)_{ij} = \left(\sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} a_{\ell,p} a_{\ell,p}^\top \right)_{ij}$$

With the coefficient $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$, this becomes :

$$\begin{aligned}(P_s)_{ij} &= \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} (a_{\ell,p})_i (a_{\ell,p})_j \\ &= \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} (c_i Y_{\ell,p}(x_i))(c_j Y_{\ell,p}(x_j))\end{aligned}$$

2. Reorganization and Application of the Addition Theorem We can factor out the quadrature weights c_i, c_j and apply the spherical harmonic addition theorem, which is the key step.

$$(P_s)_{ij} = c_i c_j \sum_{\ell=0}^{\ell_{\max}} (1+\ell)^{2s} \left[\sum_{p=1}^{N(d,\ell)} Y_{\ell,p}(x_i) Y_{\ell,p}(x_j) \right]$$

The addition theorem states that the term in brackets is a function only of the inner product $\langle x_i, x_j \rangle$:

$$\sum_{p=1}^{N(d,\ell)} Y_{\ell,p}(x) Y_{\ell,p}(y) = K_\ell(\langle x, y \rangle)$$

where $K_\ell(t) = \frac{N(d,\ell)}{\text{Area}(\mathbb{S}^{d-1})} P_\ell^{((d-2)/2)}(t)$ and $P_\ell^{(\lambda)}$ is a Gegenbauer polynomial.

3. Definition of the Kernel $p_s(t)$ By substituting this back, we can define a kernel function $p_s(t)$ that depends only on the inner product $t = \langle x_i, x_j \rangle$:

$$p_s(t) := \sum_{\ell=0}^{\ell_{\max}} (1+\ell)^{2s} K_\ell(t)$$

Thus, the matrix element is indeed a weighted kernel evaluation :

$$(P_s)_{ij} = c_i c_j \cdot p_s(\langle x_i, x_j \rangle)$$

Conclusion The matrix P_s is a **weighted kernel matrix**. If the sampling is uniform, the weights c_i are constant and P_s becomes a standard kernel matrix (up to a scaling factor). This rotational invariance, shared with the NTK, is what guarantees that they commute.

5.3 Deep NTK Properties

The limiting NTK at EOC for L-layer networks is given by :

$$K^\infty(\mathbf{x}_1, \mathbf{x}_2) = \|\mathbf{x}_1\| \|\mathbf{x}_2\| \left(\sum_{k=1}^{l} \varrho^{\circ(k-1)} \left(\rho_1^\infty(\mathbf{x}_1, \mathbf{x}_2) \right) \right. \tag{1}$$

$$\left. \times \prod_{k'=k}^{l-1} \varrho' \left(\varrho^{\circ(k'-1)} \left(\rho_1^\infty(\mathbf{x}_1, \mathbf{x}_2) \right) \right) \right) \mathbf{I}_{m_l} \tag{2}$$

Eigenvalue Decay : For L-layer ReLU networks with $L \geq 3$:

$$\mu_k \sim C(d, L) k^{-d}$$

where $C(d, L)$ depends on the parity of k and grows quadratically with L.

For the normalized NTK $\kappa_{\text{NTK}}^L / L$, the constant $C(d, L)$ grows linearly with L.

5.4 Inverse Cosine Distance Matrix Analysis

Inverse cosine distance matrix W_k for depth k :

$$W_{ki,i} = 0, \quad W_{ki_1,i_2} = \left(\frac{1-\rho_k(x_{i_1},x_{i_2})}{2}\right)^{-\frac{1}{2}} \text{ for } i_1 \neq i_2$$

Near-affine behavior :
- NTK matrix $K^\infty \approx A \cdot W_l + B$ (affine dependence)
- Spectral bounds transfer from W_k to NTK via this affine relationship
- Error terms : $O(k^{-1})$ - decreases with depth

This relationship enables indirect analysis of NTK spectral properties through simpler geometric matrices.

6
DEEP NARROW NEURAL NETWORKS

6.1 Scaled NTK at Initialization

[Theorem 1 : Scaled NTK at Initialization] For f_θ^L initialized appropriately, as $L \to \infty$:

$$\tilde{\Theta}_0^L(x,x') \xrightarrow{p} \tilde{\Theta}^\infty(x,x')$$

where

$$\tilde{\Theta}^\infty(x,x') = (x^T x' + 1 + \mathbb{E}_g[\sigma(g(x))\sigma(g(x'))])I_{d_{out}}$$

with $g \sim \mathrm{GP}(0, \rho^2 d_{in}^{-1} x^T x' + \beta^2)$ (Gaussian random field).

Alternative formulation : $\kappa_1(\cos(u)\cdot v)$ where :
- $v = \frac{1}{1+\beta^2/\alpha^2}$, where β is the bias variance
- $\alpha = \frac{\|x\|\|x'\|\rho}{d_{in}}$, where $\cos(u)$ is the cosine distance between x, x'

This framework provides a promising direction for analyzing deep narrow networks through limited expansion analysis.

6.2 Research Perspectives for Deep Narrow Networks

Architectural modifications :
- **Unit concatenation** : Combine multiple narrow networks to increase expressivity
- **Skip connections** : Investigate ResNet-style connections :

$$\tilde{\Theta}^\infty_{\text{skip}}(x,x') = \tilde{\Theta}^\infty(x,x') + \text{skip terms}$$

Initialization studies :
- **Alternative initializations** : Explore different schemes beyond current setup

— $\beta \to 0$ **limit** : Analyze behavior when bias variance vanishes :

$$v = \frac{1}{1 + \beta^2/\alpha^2} \to 1 \text{ as } \beta \to 0$$

— **Complex kernel structures** : Find initializations that yield more intricate kernel forms

Theoretical extensions :

— Extension of Hayou & Yang's work on ResNets showing "wide and deep limits commute"
— Mean field analysis for deep narrow frameworks
— Careful analysis of stochasticity in initialization (not dropout)
— Experimental validation on practical tasks

7
MAIN PROOFS

7.1 Proof 1 : Sobolev Loss as Fractional Laplacian

[Fractional Laplacian Representation of Sobolev Loss] For a function $f \in H^s(\mathbb{S}^{d-1})$ with $s > 0$, the Sobolev loss can be written as :

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x)$$

where $(I + (-\Delta)^{1/2})^s$ is the fractional Laplacian operator of order s.

Consider the spherical harmonic expansion $f(x) = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} \hat{f}_{\ell,p} Y_{\ell,p}(x)$. The spherical Laplacian acts on these harmonics as eigenvalue operators : $(-\Delta)^{1/2}_{\mathbb{S}^{d-1}} Y_{\ell,p}(x) = \sqrt{\ell(\ell+d-2)} Y_{\ell,p}(x)$.

The fractional operator $(I + (-\Delta)^{1/2})^s$ is naturally defined through its spectral action : $(I + (-\Delta)^{1/2})^s Y_{\ell,p}(x) = (1 + \sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}(x)$. For the unit sphere where $d = 2$, this simplifies to $(1 + \ell)^s Y_{\ell,p}(x)$.

Computing the bilinear form using orthonormality of spherical harmonics :

$$\int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x) = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} (1 + \sqrt{\ell(\ell+d-2)})^s |\hat{f}_{\ell,p}|^2 \tag{3}$$

The standard Sobolev norm is defined as $\|f\|^2_{H^s} = \sum_{\ell,p} (1+\ell)^{2s} |\hat{f}_{\ell,p}|^2$. The connection becomes clear when we observe that for large ℓ, the expressions $(1+\ell)^{2s}$ and $(1 + \sqrt{\ell(\ell+d-2)})^s$ are asymptotically equivalent up to normalization constants. The precise relationship depends on the specific Sobolev space convention, but the essential spectral structure remains identical.

7.2 Alternative Proof : Sobolev Loss on Unbounded Domains via Integration by Parts

[Sobolev Loss on Unbounded Domains] For functions $f \in H^s(\mathbb{R}^d)$ with $s \geq 1$ and sufficient decay at infinity, the Sobolev loss can be expressed via integration by parts as :

$$\mathcal{L}_s[f] = \int_{\mathbb{R}^d} f(x)^2 dx + \int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx + \text{higher order terms}$$

Starting Point For $s = 1$, the H^1 Sobolev norm is :

$$\|f\|_{H^1}^2 = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2 = \int_{\mathbb{R}^d} f(x)^2 dx + \int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx$$

Integration by Parts (Stokes' Theorem) The key insight is that we can relate the gradient term to the Laplacian using integration by parts. For functions with sufficient decay at infinity (so boundary terms vanish) :

$$\int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx = \int_{\mathbb{R}^d} \nabla f(x) \cdot \nabla f(x) dx \tag{4}$$

$$= -\int_{\mathbb{R}^d} f(x) \Delta f(x) dx \quad \text{(by integration by parts)} \tag{5}$$

$$= \int_{\mathbb{R}^d} f(x)(-\Delta) f(x) dx \tag{6}$$

Therefore :

$$\|f\|_{H^1}^2 = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^2 f(x) dx$$

Extension to Higher Orders For general $s \geq 1$, repeated integration by parts yields :

$$\|f\|_{H^s}^2 = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^{2s} f(x) dx$$

Fractional Case For fractional s, this extends to :

$$\mathcal{L}_s[f] = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^s f(x) dx$$

The integration by parts approach via Stokes' theorem transforms the gradient squared terms into Laplacian terms, providing the connection between differential and spectral formulations.

7.3 Case of Limited Regularity : Fourier-Based Proof

[Non-Differentiable Functions] When functions are not twice differentiable, the integration by parts approach fails due to insufficient regularity. In such cases, the **Fourier-based proof becomes the master approach**.

For $f \in L^2(\mathbb{R}^d)$ with Fourier transform $\hat{f}(\xi)$, the fractional Sobolev norm is defined directly in frequency space :

$$\|f\|_{H^s}^2 = \int_{\mathbb{R}^d} (1 + |\xi|^2)^s |\hat{f}(\xi)|^2 d\xi$$

This definition :

— Requires no differentiability assumptions on f
— Extends naturally to negative Sobolev indices $s < 0$
— Provides the most general framework for Sobolev spaces
— Reduces to classical definitions when sufficient regularity exists

The operator $(I + (-\Delta)^{1/2})^s$ acts in Fourier space as multiplication by $(1 + |\xi|)^s$, making this the fundamental definition from which all other formulations are derived.

7.4 Proof 2 : NTK Operator Multiplication by Sobolev Operator

[NTK-Sobolev Composition] Under Sobolev training, the learning operator is given by the composition :

$$\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$$

where K^∞ is the NTK operator and $(I + (-\Delta)^{1/2})^s$ is the fractional Laplacian.

Standard gradient descent on the L^2 loss $\mathcal{L}(\theta) = \frac{1}{2}\|f(\cdot;\theta) - y\|_{L^2}^2$ gives parameter dynamics $\frac{d\theta}{dt} = -\nabla_\theta \mathcal{L}(\theta)$. In the NTK regime, this translates to function dynamics $\frac{df}{dt} = -K^\infty(f - y)$.

When we replace the L^2 loss with the Sobolev loss $\mathcal{L}_s(\theta) = \frac{1}{2}\|f(\cdot;\theta) - y\|_{H^s}^2$, the gradient computation changes fundamentally. Using our fractional Laplacian representation from Theorem 1, the Sobolev loss becomes :

$$\mathcal{L}_s(\theta) = \frac{1}{2}\int (f(x;\theta) - y(x))(I + (-\Delta)^{1/2})^s (f(x;\theta) - y(x)) d\sigma(x)$$

The gradient with respect to parameters now involves the chain rule through the fractional operator :

$$\nabla_\theta \mathcal{L}_s(\theta) = \int \nabla_\theta f(x;\theta) \cdot (I + (-\Delta)^{1/2})^s (f(x;\theta) - y(x)) d\sigma(x)$$

Since the NTK is defined as $K^\infty(x, x') = \langle \nabla_\theta f(x;\theta), \nabla_\theta f(x';\theta) \rangle$, the function dynamics become :

$$\frac{df}{dt} = -\int K^\infty(x, x')(I + (-\Delta)^{1/2})^s (f(x') - y(x')) d\sigma(x') = -K^\infty \circ (I + (-\Delta)^{1/2})^s (f - y)$$

Therefore, the learning operator is indeed $\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$. The rotational invariance of both operators ensures they share the same spherical harmonic eigenfunctions, making this composition well-defined and commutative.

7.5 Proof 3 : Spectral Properties of the Composite Operator

[Spectrum of NTK-Sobolev Operator] The eigenvalues of the composite operator $\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$ are given by the product of individual eigenvalues : $\mu_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot (1 + \sqrt{\ell(\ell + d - 2)})^s$.

Since both operators K^∞ and $(I + (-\Delta)^{1/2})^s$ share the same eigenfunctions (spherical harmonics $Y_{\ell,p}$), we can analyze their composition through eigenvalues :

1) For the NTK operator K^∞ :

$$K^\infty Y_{\ell,p} = \mu_\ell^{(K)} Y_{\ell,p}$$

2) For the Sobolev operator $(I + (-\Delta)^{1/2})^s$:

$$(I + (-\Delta)^{1/2})^s Y_{\ell,p} = (1 + \sqrt{\ell(\ell + d - 2)})^s Y_{\ell,p}$$

3) Therefore, for their composition :

$$\begin{aligned}\mathcal{T}_s Y_{\ell,p} &= K^\infty \circ (I + (-\Delta)^{1/2})^s Y_{\ell,p} \\ &= K^\infty((1 + \sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}) \\ &= (1 + \sqrt{\ell(\ell+d-2)})^s \cdot \mu_\ell^{(K)} Y_{\ell,p}\end{aligned}$$

Thus, $\mu_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot (1 + \sqrt{\ell(\ell+d-2)})^s$ are indeed the eigenvalues of the composite operator.

7.6 NTK-Sobolev Operator Framework

The key innovation in Sobolev training is the modification of the standard L^2 loss to incorporate high-order derivatives. The NTK-Sobolev operator P_s acts on the sphere $\mathbb{S}^{d-1}$ with the commutation property :

Spherical Harmonics Transform : Let $\mathcal{F}$ denote the mapping from $L^2(\mathbb{S}^d)$ to the spectral domain $\bigoplus_{\ell=0}^\infty \mathbb{C}^{N(d,\ell)}$, where $N(d,\ell)$ is the dimension of the ℓ-th spherical harmonic space.

The Sobolev operator P_s is defined through its action in Fourier space :

$$P_s = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} P_{\ell,p}$$

where $P_{\ell,p} = a_{\ell,p} a_{\ell,p}^T$ with spectral coefficients $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$.

7.7 Proof 4 : Commutation of Discrete Matrix Operators

[Matrix Commutation] The discrete matrices K and P_s commute : $KP_s = P_s K$.

We establish commutation by expanding both matrices in terms of spherical harmonic projectors using the sampling measure (sum of Dirac masses).

NTK Matrix Expansion The NTK matrix can be written using its spectral decomposition with respect to spherical harmonics :

$$K_{ij} = K^\infty(x_i, x_j) = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} \mu_\ell^{(K)} (a_{\ell,p})_i (a_{\ell,p})_j$$

where $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$ with quadrature weights c_i and $\mu_\ell^{(K)}$ are the NTK operator eigenvalues.

This gives us the matrix form :

$$K = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T$$

Sobolev Matrix Expansion Similarly, the Sobolev matrix has the expansion :

$$P_s = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} (1 + \sqrt{\ell(\ell+d-2)})^s a_{\ell,p} a_{\ell,p}^T$$

Matrix Product Computation Computing the product KP_s :

$$KP_s = \left(\sum_{\ell,p} \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T\right)\left(\sum_{\ell',p'} (1+\sqrt{\ell'(\ell'+d-2)})^s a_{\ell',p'} a_{\ell',p'}^T\right) \tag{7}$$

$$= \sum_{\ell,p}\sum_{\ell',p'} \mu_\ell^{(K)} (1+\sqrt{\ell'(\ell'+d-2)})^s a_{\ell,p}(a_{\ell,p}^T a_{\ell',p'}) a_{\ell',p'}^T \tag{8}$$

Orthogonality and Commutation The key observation is that $a_{\ell,p}^T a_{\ell',p'} = \delta_{\ell,\ell'}\delta_{p,p'}\|a_{\ell,p}\|^2$ due to the orthogonality of spherical harmonics under the sampling measure. Therefore :

$$KP_s = \sum_{\ell,p} \mu_\ell^{(K)} (1+\sqrt{\ell(\ell+d-2)})^s a_{\ell,p} a_{\ell,p}^T \tag{9}$$

$$P_s K = \sum_{\ell,p} (1+\sqrt{\ell(\ell+d-2)})^s \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T \tag{10}$$

Since multiplication of scalars commutes, we have $KP_s = P_s K$.

Underlying Reason : This commutation property reflects the fact that both K^∞ and P_s operators commute in the continuous setting because they share spherical harmonics as eigenfunctions due to rotational invariance.

7.8 Proof 5 : Eigenvalue Scaling Laws

[Asymptotic Scaling Laws] For the NTK-Sobolev operator, eigenvalues follow the scaling laws $\lambda_\ell \sim \ell^{-d} \cdot (1+\sqrt{\ell(\ell+d-2)})^s$.

Individual Operator Eigenvalues From previous analysis :

- NTK operator eigenvalues : $\mu_\ell^{(K)} \sim C(d,L)\ell^{-d}$ for large ℓ
- Sobolev operator eigenvalues : $\mu_\ell^{(P_s)} = (1+\sqrt{\ell(\ell+d-2)})^s$

Composite Operator Spectrum Since the operators commute and share eigenfunctions, the eigenvalues of the composite operator are products :

$$\lambda_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot \mu_\ell^{(P_s)} = C(d,L)\ell^{-d} \cdot (1+\sqrt{\ell(\ell+d-2)})^s$$

Asymptotic Analysis For large ℓ, we have $\sqrt{\ell(\ell+d-2)} \sim \ell\sqrt{1+(d-2)/\ell} \sim \ell$ for d fixed. Therefore :

$$\lambda_\ell^{(\mathcal{T}_s)} \sim C(d,L)\ell^{-d} \cdot (1+\ell)^s = C(d,L)\ell^{s-d}$$

Critical Scaling Behavior The spectral behavior depends on the relationship between s and d :

- $s < d$: Eigenvalues decay ($\lambda_\ell \to 0$) - regularizing effect
- $s = d$: Logarithmic corrections appear - critical regime
- $s > d$: Eigenvalues grow ($\lambda_\ell \to \infty$) - amplifying high frequencies

This scaling law $\lambda_\ell \sim \ell^{s-d}$ determines the spectral properties and learning dynamics of Sobolev-trained networks, with the critical exponent $s-d$ controlling frequency bias.

7.9 Synthesis : Unified Framework for NTK-Sobolev Analysis

Summary : Rotational invariance of both NTK and Sobolev operators ensures matrix commutation $KP_s = P_s K$, with composite eigenvalues $\lambda_\ell \sim \ell^{s-d}$ determining learning dynamics.

[Practical Implementation Warning] **Dataset Context** : In practical machine learning settings, we only have access to function values $f(x_i)$ at sampled points, *not* the gradients $\nabla f(x_i)$. This makes the Fourier-based approach the most relevant for theoretical analysis and implementation.

Gradient Availability : If gradients were available (e.g., through physics-informed neural networks or when the true function derivatives are known), the theoretical results presented here remain valid. However, the practical loss implementation in PyTorch should then utilize :

- Automatic differentiation (autograd) to compute gradients
- Direct gradient-based Sobolev norms : $\|f\|_{H^1}^2 = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2$
- Higher-order derivative computations via successive autograd calls

The spectral analysis framework developed here provides the theoretical foundation regardless of the implementation approach.

7.10 From Sobolev Loss to Discrete Matrix Operator

7.10.1 • Two Integral Formulations

Formulation 1 : Uniform Lebesgue Measure on the Sphere

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x)$$

where $d\sigma(x)$ is the uniform surface measure on $\mathbb{S}^{d-1}$ with $\int_{\mathbb{S}^{d-1}} d\sigma(x) = \text{Area}(\mathbb{S}^{d-1})$.

Formulation 2 : Sampling Measure from Dataset

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\mu_n(x)$$

where $\mu_n = \frac{1}{n}\sum_{i=1}^n \delta_{x_i}$ is the empirical measure from our sampled dataset $\{x_i\}_{i=1}^n$.

Discrete Implementation : Under the sampling measure μ_n, the integral becomes :

$$\mathcal{L}_s[f] = \frac{1}{n}\sum_{i=1}^n f(x_i)\left[(I + (-\Delta)^{1/2})^s f\right](x_i)$$

This leads to the matrix form $\mathcal{L}_s[f] \approx \mathbf{f}^T P_s \mathbf{f}$ where $\mathbf{f} = (f(x_1), \ldots, f(x_n))^T$.

Relationship Between Formulations : The uniform measure provides the theoretical foundation, while the sampling measure μ_n represents the practical implementation. As $n \to \infty$ with appropriate sampling, $\mu_n \to d\sigma$ in the weak sense, ensuring convergence of the discrete formulation to the continuous theory.

7.10.2 • Practical Implementation Considerations

1. **Data-Only Setting** : Since we only have function values $\{f(x_i)\}_{i=1}^n$, the matrix P_s must be constructed via :
$$(P_s)_{ij} = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1 + \sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}(x_i) Y_{\ell,p}(x_j)$$
2. **Computational Complexity** : The matrix construction requires :
   - Evaluation of spherical harmonics $Y_{\ell,p}(x_i)$ for all (ℓ, p) and data points
   - Summation over $\mathcal{O}(\ell_{\max}^{d-1})$ harmonics (due to dimension growth $N(d,\ell) \sim \ell^{d-2}$)
   - Total complexity : $\mathcal{O}(n^2 \ell_{\max}^{d-1})$ for matrix construction

3. **Truncation Strategy** : In practice, $\ell_{\max}$ must be chosen to balance :
   — Spectral accuracy (larger $\ell_{\max}$ captures more frequencies)
   — Computational feasibility (smaller $\ell_{\max}$ reduces cost)
   — Statistical stability (avoid overfitting to high-frequency noise)
4. **Alternative with Gradients** : If gradients $\{\nabla f(x_i)\}$ were available, direct computation becomes possible :
$$\mathcal{L}_s[f] \approx \frac{1}{n}\sum_{i=1}^{n}\left[f(x_i)^2 + s\|\nabla f(x_i)\|^2 + \text{higher order terms}\right]$$
This approach scales as $\mathcal{O}(nd)$ instead of $\mathcal{O}(n^2\ell_{\max}^{d-1})$.

Fourier decomposition : For $f(x) = \sum_{\ell,p}\hat{f}_{\ell,p}Y_{\ell,p}(x)$:
$$\mathcal{L}_s[f] = \sum_{\ell=0}^{\infty}\sum_{p=1}^{N(d,\ell)}(1+\ell)^{2s}|\hat{f}_{\ell,p}|^2$$

Discretization : At points $\{x_i\}_{i=1}^n$:
$$\hat{f}_{\ell,p} \approx \sum_{i=1}^{n} c_i f(x_i) Y_{\ell,p}(x_i)$$

Final matrix form : $\mathcal{L}_s[f] \approx f^T P_s f$ where $f = (f(x_1), \ldots, f(x_n))^T$.

7.11 Spectral Properties and Dimension Growth

The dimension of spherical harmonic spaces grows as :
$$N(d,\ell) \sim \frac{2\ell^{d-2}}{(d-2)!} \quad \text{as } \ell \to \infty$$

This exponential growth in dimension determines the computational complexity of the Sobolev operator discretization.

7.12 Common Eigenfunctions Property

[Spherical Harmonics as Common Eigenfunctions] For rotationally invariant kernels on $\mathbb{S}^{d-1}$:
— NTK operator K^∞ and Sobolev operator P_s share spherical harmonics $Y_{\ell,p}$ as eigenfunctions
— This is due to rotational invariance and Schur's lemma for irreducible representations
— Enables direct spectral modification analysis
[Commutation Property] The NTK operator K^∞ and Sobolev operator P_s commute :
$$[K^\infty, P_s] = 0$$
This holds for any sampling distribution $\rho(x)$ on $\mathbb{S}^{d-1}$ and implies that spectral decompositions are compatible.

7.13 Analyse Spectrale du NTK et Bornes sur les Valeurs Propres

(Discussion sur la plus grande valeur propre du NTK. Présentation des théorèmes de Terjek et du plan de preuve pour obtenir une borne sur la plus petite valeur propre en utilisant la RMT.)

7.14 Matrix Spectrum Results

The key scaling laws for the discrete NTK matrix are :

Condition Number :

$$\kappa(K^{\infty}) \sim 1 + \frac{n}{3} + \mathcal{O}(n\xi/l)$$

The condition number grows linearly with data size n but improves with depth l.

Eigenvalue Distribution :

$$\lambda_{\min} \sim \frac{3l}{4}, \quad \lambda_{\max} \sim \frac{3nl}{4} \pm \xi \text{ where } \xi \sim \log(l)$$

The minimum eigenvalue scales linearly with depth, while the maximum scales with both depth and data size.

Key Insight : The training dynamics are dictated by the NTK matrix eigenvalues, not the operator eigenvalues. Deeper networks ($l \uparrow$) improve conditioning but with diminishing returns.

8
MATHEMATICAL FOUNDATION : CONCENTRATION INEQUALITIES

8.1 Scalar Concentration Inequalities

The foundation of modern NTK analysis rests on powerful concentration inequalities that control the deviation of random variables from their expectations.

[Chernoff Bound for Bounded Random Variables] Let $X_1, \ldots, X_n$ be independent random variables with $X_i \in [0, 1]$ and $\mathbb{E}[X_i] = \mu_i$. Let $S = \sum_{i=1}^{n} X_i$ and $\mu = \mathbb{E}[S] = \sum_{i=1}^{n} \mu_i$. Then :

$$\Pr(S \geq (1+\delta)\mu) \leq e^{-\frac{\delta^2 \mu}{2+\delta}}$$

$$\Pr(S \leq (1-\delta)\mu) \leq e^{-\frac{\delta^2 \mu}{2}}$$

for $\delta > 0$.

3[Bernstein's Inequality] Let $X_1, \ldots, X_n$ be independent random variables with $\mathbb{E}[X_i] = 0$, $|X_i| \leq M$, and $\mathbb{E}[X_i^2] \leq \sigma_i^2$. Let $S = \sum_{i=1}^{n} X_i$ and $\sigma^2 = \sum_{i=1}^{n} \sigma_i^2$. Then :

$$\Pr(|S| \geq t) \leq 2 \exp\left(-\frac{t^2/2}{\sigma^2 + Mt/3}\right)$$

8.2 Quadratic Forms and Hanson-Wright Inequalities

For neural networks, we frequently encounter quadratic forms in random variables, necessitating more sophisticated concentration tools.

[Classical Hanson-Wright Inequality] Let $Y_1, \ldots, Y_n$ be independent mean-zero α-subgaussian random variables, and $\mathbf{A} = (a_{i,j})$ be a symmetric matrix. Then :

$$\Pr\left(\left|\sum_{i,j=1}^{n} a_{i,j}(Y_iY_j - \mathbb{E}[Y_iY_j])\right| \geq t\right) \leq 2\exp\left(-\frac{1}{C}\min\left\{\frac{t^2}{\beta^4\|\mathbf{A}\|_{HS}^2}, \frac{t}{\beta^2\|\mathbf{A}\|_{op}}\right\}\right)$$

where $\|\mathbf{A}\|_{HS} = \sqrt{\sum_{i,j}|a_{i,j}|^2}$ is the Hilbert-Schmidt norm and $\|\mathbf{A}\|_{op}$ is the operator norm.

[Generalized Hanson-Wright for Random Tensors (Chang, 2022)] For random tensor vector $\overline{\mathcal{X}} \in^{(n\times I_1\times\cdots\times I_M)\times(I_1\times\cdots\times I_M)}$ and fixed tensor $\overline{\overline{\mathcal{A}}}$, the polynomial function :

$$f_j(\overline{\mathcal{X}}) = \left(\sum_{i,k=1}^{n} \mathcal{A}_{i,k} \star_M (\mathcal{X}_i - \mathbb{E}[\mathcal{X}_i]) \star_M (\mathcal{X}_k - \mathbb{E}[\mathcal{X}_k])\right)^j$$

satisfies concentration bounds involving Ky Fan k-norms of tensor sums, extending classical matrix concentration to tensor settings.

8.3 Matrix Concentration Inequalities

Matrix concentration inequalities provide direct control over eigenvalues of random matrix sums, which is essential for NTK analysis.

[Matrix Chernoff Bound (Tropp, 2012)] Let $X_1, \ldots, X_n$ be independent random Hermitian matrices with $X_i \preceq R \cdot I$ and $\mathbb{E}[X_i] \preceq \mu \cdot I$. Let $S = \sum_{i=1}^{n} X_i$. Then :

$$\Pr(\lambda_{\max}(S) \geq (1+\delta)\mu n) \leq d\left(\frac{e^\delta}{(1+\delta)^{1+\delta}}\right)^{\mu n/R}$$

[Matrix Bernstein Inequality (Vershynin, 2018)] Let $X_1, \ldots, X_n$ be independent random matrices with $\mathbb{E}[X_i] = 0$, $\|X_i\| \leq L$, and $\|\sum_{i=1}^{n}\mathbb{E}[X_iX_i^*]\| \leq \sigma^2$. Then :

$$\Pr\left(\left\|\sum_{i=1}^{n} X_i\right\| \geq t\right) \leq 2d\exp\left(-\frac{t^2/2}{\sigma^2 + Lt/3}\right)$$

9

FUNDAMENTAL MATHEMATICAL TOOLS

9.1 Matrix Analysis Techniques

[Weyl's Inequality] For Hermitian matrices A, B :

$$\lambda_i(A) + \lambda_n(B) \leq \lambda_i(A+B) \leq \lambda_i(A) + \lambda_1(B)$$

[Schur Product Theorem] For PSD matrices P, Q :

$$P \circ Q \geq P \min_i Q_{ii}$$

[Gershgorin Circle Theorem] For matrix $A = (a_{ij})$, all eigenvalues lie in the union of discs :

$$\bigcup_{i=1}^{n} \left\{ z \in : |z - a_{ii}| \leq \sum_{j \neq i} |a_{ij}| \right\}$$

9.2 Specialized Neural Network Tools

[Khatri-Rao Product] For matrices $A \in^{m \times k}$ and $B \in^{n \times k}$, the Khatri-Rao product $A \star B \in^{mn \times k}$ is the column-wise Kronecker product.

[Hadamard Power] For matrix A and positive integer r, the r-th Hadamard power $A^{\odot r}$ is the element-wise r-th power.

10
PROOF SCHEMES AND HIGH-LEVEL STRATEGIES

10.1 General Framework for Eigenvalue Bounds

All modern approaches to bounding () follow a common high-level structure, though they differ in technical details and specific tools employed.

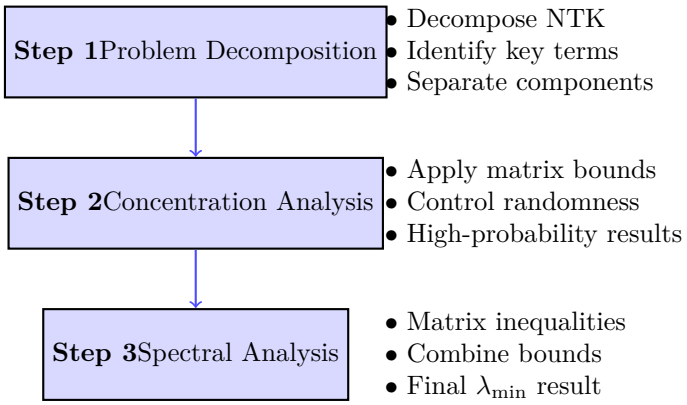


Figure 1 – General Framework for NTK Eigenvalue Bounds

10.2 Proof Scheme 1 : Empirical NTK Decomposition (Nguyen et al.)

High-Level Strategy : Direct decomposition of empirical NTK $\bar{K}^{(L)} = JJ^T$ using feature matrices and chain rule to obtain precise bounds on .

10.2.1 • Step 1 : Fundamental Chain Rule Decomposition

The proof begins by decomposing the empirical NTK $\bar{K}^{(L)} = JJ^T$ where J is the Jacobian matrix (**??**). Applying the chain rule and standard algebraic manipulations yields the fundamental decomposition :

$$\bar{K}^{(L)} = JJ^T = \sum_{k=0}^{L-1} F_k F_k^T \circ B_{k+1} B_{k+1}^T \tag{11}$$

where :
- $F_k = [f_k(x_1), \ldots, f_k(x_N)]^T \in^{N \times n_k}$ are the **feature matrices** at layer k
- $B_k \in^{N \times n_k}$ are the **derivative term matrices** with i-th row given by :

$$(B_k)_{i:} = \begin{cases} \Sigma_k(x_i) \left(\prod_{l=k+1}^{L-1} W_l \Sigma_l(x_i) \right) W_L, & k \in [L-2] \\ \Sigma_{L-1}(x_i) W_L, & k = L-1 \\ \frac{1}{\sqrt{N}} \mathbf{1}_N, & k = L \end{cases} \tag{12}$$

where $\Sigma_k(x) = \mathrm{diag}([\sigma'(g_{k,j}(x))]_{j=1}^{n_k})$ is the diagonal matrix of activation derivatives.

10.2.2 • Step 2 : Application of Schur Product Theorem and Weyl's Inequality

For PSD matrices $P, Q \in^{n \times n}$, the Schur product theorem :

$$P \circ Q \geq P \min_{i \in [n]} Q_{ii} \tag{13}$$

Applying this to each term, then using Weyl's inequality for the sum :

$$JJ^T \geq \sum_{k=0}^{L-1} F_k F_k^T \circ B_{k+1} B_{k+1}^T \tag{14}$$

$$\geq \sum_{k=0}^{L-1} F_k F_k^T \min_{i \in [N]} \|(B_{k+1})_{i:}\|_2^2 \tag{15}$$

This reduction separates the problem into :

1. The **minimum eigenvalues of feature Gram matrices** : $F_k F_k^T$
2. The **minimum row norms of derivative matrices** : $\min_{i \in [N]} \|(B_{k+1})_{i:}\|_2^2$

10.2.3 • Step 3 : Bounding Feature Matrix Eigenvalues

To bound $F_k F_k^T$, the strategy uses Chernoff-type matrix concentration. The fundamental bound is given by : [Matrix Concentration for Features] Let $\lambda = \mathbb{E}_{w\sim\mathcal{N}(0,\beta_k^2\mathbf{I}_{n_{k-1}})}[\sigma(F_{k-1}w)\sigma(F_{k-1}w)^T]$. If

$$n_k \geq \max\left(N, cQ\max(1,\log(4Q))\log\frac{N}{\delta}\right) \tag{16}$$

where $Q = \frac{\beta_k^2\|F_{k-1}\|_F^2}{\lambda}$, then with probability at least $1-\delta$:

$$F_k{}^2 \geq \frac{n_k\lambda}{4} \tag{17}$$

This lemma assumes that :

— The width n_k of layer k is sufficiently large compared to the number of samples N

— The feature matrix F_{k-1} from the previous layer has bounded Frobenius norm

These conditions ensure concentration of the minimum singular value of the feature matrix around its expected value.

10.2.4 • Step 4 : Spectral Analysis via Hermite Expansion

To bound λ, we use the Hermite expansion of the ReLU. and exploiting the homogeneity of σ (unless, generalized hermite) :

$$\lambda = \mathbb{E}[\sigma(F_{k-1}w)\sigma(F_{k-1}w)^T] \tag{18}$$

$$= D\mathbb{E}[\sigma(\hat{F}_{k-1}w)\sigma(\hat{F}_{k-1}w)^T]D \tag{19}$$

where $D = \mathrm{diag}(\|\{F_{k-1}\}_{i:}\|_2)$ and $\hat{F}_{k-1} = D^{-1}F_{k-1}$.

Applying the Hermite expansion $\sigma(x) = \sum_{r=0}^{\infty}\mu_r(\sigma)H_r(x)$:

$$\mathbb{E}[\sigma(\hat{F}_{k-1}w)\sigma(\hat{F}_{k-1}w)^T] = \mu_0(\sigma)^2\mathbf{1}_N\mathbf{1}_N^T + \sum_{s=1}^{\infty}\mu_s(\sigma)^2(\hat{F}_{k-1}^{*s})(\hat{F}_{k-1}^{*s})^T \tag{20}$$

where $\hat{F}_{k-1}^{*s}$ represents the s-th Khatri-Rao power of $\hat{F}_{k-1}$.

10.2.5 • Step 5 : Reduction to Khatri-Rao Powers

For an integer $r \geq 2$, we can lower bound :

$$\lambda \geq \mu_r(\sigma)^2 D(\hat{F}_{k-1}^{*r})(\hat{F}_{k-1}^{*r})^T D = \mu_r(\sigma)^2\frac{(F_{k-1}^{*r})(F_{k-1}^{*r})^T}{\max_{i\in[N]}\|(F_{k-1})_{i:}\|_2^{2(r-1)}} \tag{21}$$

10.2.6 • Step 6 : Feature Centering and Hadamard Power Analysis

To analyze $(F_{k-1}^{*r})(F_{k-1}^{*r})^T = (F_{k-1}F_{k-1}^T)^{\circ r}$, we introduce centered features $\tilde{F}_{k-1} = F_{k-1} - \mathbb{E}_X[F_{k-1}]$.

Let $\mu = \mathbb{E}_x[f_{k-1}(x)] \in^{n_{k-1}}$ and $\Lambda = \mathrm{diag}(F_{k-1}\mu - \|\mu\|_2^2\mathbf{1}_N)$. Then :

$$(F_{k-1}^{*r})(F_{k-1}^{*r})^T = (F_{k-1}F_{k-1}^T)^{\circ r} \succeq \left(\tilde{F}_{k-1}\tilde{F}_{k-1}^T - \frac{\Lambda\mathbf{1}_N\mathbf{1}_N^T\Lambda}{\|\mu\|_2^2}\right)^{\circ r} \tag{22}$$

10.2.7 • Step 7 : Application of Gershgorin Circle Theorem

To bound the minimum eigenvalue of $(\tilde{F}_{k-1}\tilde{F}_{k-1}^T)^{\circ r}$, we apply the Gershgorin circle theorem :

$$(\tilde{F}_{k-1}^{*r})(\tilde{F}_{k-1}^{*r})^T \geq \min_{i\in[N]} \|(\tilde{F}_{k-1})_{i:}\|_2^{2r} - N \max_{i\neq j} |\langle (\tilde{F}_{k-1})_{i:}, (\tilde{F}_{k-1})_{j:}\rangle|^r \tag{23}$$

Using concentration properties of centered features, we show :

$$\|(\tilde{F}_{k-1})_{i:}\|_2^{2r} = \Theta\left(\left(d\prod_{l=1}^{k-1} n_l\beta_l^2\right)^r\right) \tag{24}$$

$$N \max_{i\neq j} |\langle (\tilde{F}_{k-1})_{i:}, (\tilde{F}_{k-1})_{j:}\rangle|^r = o\left(\left(d\prod_{l=1}^{k-1} n_l\beta_l^2\right)^r\right) \tag{25}$$

with exponentially high probability.

10.2.8 • Step 8 : Bounding Derivative Term Norms, very technical

To bound $\min_{i\in[N]} \|(B_{k+1})_{i:}\|_2^2$, we analyze each case :

For $k \in [L-2]$:

$$\|(B_{k+1})_{i:}\|_2^2 = \|\Sigma_{k+1}(x_i)\left(\prod_{l=k+2}^{L-1} W_l\Sigma_l(x_i)\right)W_L\|_2^2 \tag{26}$$

$$= \Theta\left(\beta_L^2 n_{k+1}\prod_{l=k+2}^{L-1} n_l\beta_l^2\right) \tag{27}$$

This bound uses some very technical and important tools :

- **Operator norm for concentration of Gaussian matrices**
- **Inductive analysis of random matrix products for every layer**
- **Properties of derivative matrices $\Sigma_l(x)$ for every layer**

10.2.9 • Step 9 : Final Assembly and Width Optimization

Combining all bounds via union bound yields :

$$\bar{K}^{(L)} \geq \sum_{k=0}^{L-1} \mu_r(\sigma)^2\Theta\left(d\prod_{l=1}^{k} n_l\beta_l^2\right)\Theta\left(\beta_L^2\prod_{l=k+1}^{L-1} n_l\beta_l^2\right) \tag{28}$$

$$= \mu_r(\sigma)^2\Theta\left(d\beta_L^2\sum_{k=0}^{L-1}\prod_{l=1}^{L-1} n_l\beta_l^2\right) \tag{29}$$

The width condition requires at least one layer to satisfy :

$$n_k = \tilde{\Omega}(N\log N), \quad \text{with } \xi_k = 1 \tag{30}$$

where ξ_k indicates layer k is "wide".

10.2.10 • Upper Bound via Diagonal Analysis

For the upper bound, we use :

$$JJ^T \leq (JJ^T)_{11} = \sum_{k=0}^{L-1} \|(F_k)_{1:}\|_2^2 \|(B_{k+1})_{1:}\|_2^2 \tag{31}$$

$$= \sum_{k=0}^{L-1} \|f_k(x_1)\|_2^2 \|(B_{k+1})_{1:}\|_2^2 \tag{32}$$

$$= \mathcal{O}\left(d\beta_L^2 \sum_{k=0}^{L-1} \prod_{l=1}^{L-1} n_l \beta_l^2\right) \tag{33}$$

10.2.11 • Final Result

Assuming $\beta_l = 1$ for simplicity and that some layer k^* satisfies $n_{k^*} = \tilde{\Omega}(N)$, the main theorem gives :

$$\mu_r(\sigma)^2 \Omega(d) \leq \bar{K}^{(L)} \leq \mathcal{O}(dL) \tag{34}$$

with probability at least $1 - \text{poly}(N) \exp(-\Omega(\min_l n_l))$.

This approach significantly reduces width requirements compared to classical methods, requiring only one wide layer rather than all layers.

11
CORRECTIONS À LARGEUR FINIE ET RÉGIMES D'APPRENTISSAGE

11.1 Développement Théorique pour les FCNN

(Présentation des théorèmes et formules pour les corrections à largeur finie dans les FCNN. Analyse des lois de scaling du reste et discussion sur les termes d'erreur, en lien avec les ensembles de matrices aléatoires QUE GOE.)

12
FRAMEWORK AND NOTATIONS

In this section, we introduce the mathematical framework and notations used throughout this document, building upon the definitions established in the work on the Neural Tangent Hierarchy (NTH) [**dyer2019asymptoticswidenetworksf**

12.1 Neural Network Architecture

We consider a fully-connected feedforward neural network with H hidden layers of width m.

- The network input is a vector $x \in \mathbb{R}^d$.
- The output of layer ℓ is denoted by $x^{(\ell)}$. For $\ell \in \{1, \dots, H\}$, it is computed recursively as :

$$x^{(\ell)} = \frac{1}{\sqrt{m}} \sigma(W^{(\ell)} x^{(\ell-1)}) \tag{35}$$

  where $x^{(0)} = x$ is the input, $W^{(\ell)}$ is the weight matrix of layer ℓ, and σ is a non-linear activation function applied component-wise.
- The final output of the network is a linear combination of the last hidden layer's output :

$$f(x, \theta) = a^T x^{(H)} \tag{36}$$

  where $a \in \mathbb{R}^m$ is the weight vector of the output layer.
- The set of all trainable parameters of the network is denoted $\theta = (\mathrm{vec}(W^{(1)}), \dots, \mathrm{vec}(W^{(H)}), a)$.
- The derivative of the activation function for layer ℓ is represented by a diagonal matrix $\sigma'_\ell(x) = \mathrm{diag}(\sigma'(W^{(\ell)} x^{(\ell-1)}))$.

12.2 The Neural Tangent Kernel (NTK)

The network is trained via gradient descent on the quadratic loss : $L(\theta) = \frac{1}{2n} \sum_{\alpha=1}^{n} (f(x_\alpha, \theta) - y_\alpha)^2$. The dynamics of the network function $f(x, \theta_t)$ during continuous-time training (gradient flow) are governed by the Neural Tangent Kernel (NTK), denoted $K_t^{(2)}$.

$$\frac{d}{dt} f(x, \theta_t) = -\frac{1}{n} \sum_{\beta=1}^{n} K_t^{(2)}(x, x_\beta)(f(x_\beta, \theta_t) - y_\beta) \tag{37}$$

The NTK is defined as the inner product of the gradients of the output function with respect to the network parameters :

$$K_t^{(2)}(x_\alpha, x_\beta) := \langle \nabla_\theta f(x_\alpha, \theta_t), \nabla_\theta f(x_\beta, \theta_t) \rangle \tag{38}$$

It can be decomposed as a sum over the network layers :

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{39}$$

where the vector $G_\mu^{(\ell)}$ represents the backpropagated gradient up to layer ℓ :

$$G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t \tag{40}$$

12.3 The Neural Tangent Hierarchy (NTH)

For finite-width networks ($m < \infty$), the NTK $K_t^{(2)}$ is not constant but evolves during training. Its dynamics are dictated by the higher-order kernel $K_t^{(3)}$. This gives rise to an infinite sequence of coupled ordinary differential equations, known as the Neural Tangent Hierarchy (NTH) :

$$\frac{d}{dt} K_t^{(r)}(x_{\alpha_1}, \dots, x_{\alpha_r}) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(r+1)}(x_{\alpha_1}, \dots, x_{\alpha_r}, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{41}$$

valid for all $r \geq 2$. The higher-order kernel $K^{(r+1)}$ is defined recursively as the derivative of $K^{(r)}$ with respect to the parameters, contracted with the gradient of the network output :

$$K_t^{(r+1)}(x_1, \ldots, x_{r+1}) := \langle \nabla_\theta K_t^{(r)}(x_1, \ldots, x_r), \nabla_\theta f_t(x_{r+1}) \rangle \tag{42}$$

This hierarchy captures the complete training dynamics beyond the infinite-width approximation where only $K^{(2)}$ is considered and is constant. The purpose of this document is to derive and analyze the explicit form of $K^{(3)}$, which represents the first correction to the NTK dynamics.

13
OTHER PROOF

In the framework of the Neural Tangent Hierarchy (NTH), the kernel $K^{(3)}$ arises as the operator governing the time evolution of the Neural Tangent Kernel $K^{(2)}$. The dynamics of $K^{(2)}$ under gradient flow are given by :

$$\frac{d}{dt} K_t^{(2)}(x_\alpha, x_\beta) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{43}$$

where $K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)$ is defined as the contraction of the gradient of $K_t^{(2)}(x_\alpha, x_\beta)$ with respect to the network parameters θ with the gradient of the network output $f_t(x_\gamma)$:

$$K_t^{(3)}(x_\alpha, x_\beta, x_\gamma) := \langle \nabla_\theta K_t^{(2)}(x_\alpha, x_\beta), \nabla_\theta f_t(x_\gamma) \rangle. \tag{44}$$

Let's derive the explicit formula for $K_t^{(3)}$. We start from the expression for $K_t^{(2)}(x_\alpha, x_\beta)$:

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{45}$$

where we use the notation :

- $x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma(W^{(p)} x_\mu^{(p-1)})$ for $p \geq 1$, and $x_\mu^{(0)} = x_\mu$.
- $G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t$, with $\sigma'_\ell(x_\mu) = \mathrm{diag}(\sigma'(W^{(\ell)} x_\mu^{(\ell-1)}))$.

Let $\delta_\gamma(\cdot) := \langle \nabla_\theta(\cdot), \nabla_\theta f_t(x_\gamma) \rangle$ denote the directional derivative along $\nabla_\theta f_t(x_\gamma)$. Then $K^{(3)}_{\alpha\beta\gamma} = \delta_\gamma(K^{(2)}_{\alpha\beta})$. Applying the product rule, we get :

$$K^{(3)}_{\alpha\beta\gamma} = \delta_\gamma \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \delta_\gamma \left(\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \right) \tag{46}$$

Each term expands further, following $\delta_\gamma \langle A, B \rangle = \langle \delta_\gamma A, B \rangle + \langle A, \delta_\gamma B \rangle$. This results in a sum over all components of $K^{(2)}_{\alpha\beta}$, where each component is acted upon by δ_γ. The core of the derivation lies in computing δ_γ for the building blocks $x_\mu^{(p)}$ and $G_\mu^{(p)}$.

The recursive formula for the derivative of the activations is :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)}(\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right) \tag{47}$$

with the base case $\delta_\gamma x_\mu^{(0)} = \delta_\gamma x_\mu = 0$.

The derivative of the backpropagated vectors $G_\mu^{(\ell)}$ is given by a sum of terms, where δ_γ acts on one factor at a time :

$$\delta_\gamma G_\mu^{(\ell)} = \sum_{p=\ell}^{H} \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \delta_\gamma(\frac{1}{\sqrt{m}}(W^{(p+1)})^T) \cdots \sigma'_H(x_\mu) a_t \right) \tag{48}$$

$$+ \sum_{p=\ell}^{H} \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \delta_\gamma(\sigma'_p(x_\mu)) \cdots \sigma'_H(x_\mu) a_t \right) \tag{49}$$

$$+ \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \sigma'_H(x_\mu)(\delta_\gamma a_t) \right) \tag{50}$$

where

- $\delta_\gamma a_t = x_\gamma^{(H)}$
- The term $\delta_\gamma((W^{(p+1)})^T)$ corresponds to a rank-1 update which, when contracted in context, yields terms involving $G_\gamma^{(p+1)}$ and inner products of activations.
- $\delta_\gamma(\sigma'_p(x_\mu)) = \sigma''_p(x_\mu)\mathrm{diag}(\delta_\gamma(W^{(p)} x_\mu^{(p-1)}/\sqrt{m}))$. This term vanishes for ReLU activation functions as $\sigma'' = 0$ almost everywhere.

Combining these rules gives the complete formula for $K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)$. It is a sum of many terms, each corresponding to differentiating one part of $K_t^{(2)}(x_\alpha, x_\beta)$. For clarity, we write it as a sum of contributions :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\mathrm{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right) \tag{51}$$

where

$$K^{(3,\mathrm{out})}_{\alpha\beta\gamma} = \langle \delta_\gamma x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \langle x_\alpha^{(H)}, \delta_\gamma x_\beta^{(H)} \rangle \tag{52}$$

$$K^{(3,G)}_{\alpha\beta\gamma,\ell} = \left(\langle \delta_\gamma G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \delta_\gamma G_\beta^{(\ell)} \rangle \right) \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{53}$$

$$K^{(3,x)}_{\alpha\beta\gamma,\ell} = \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \left(\langle \delta_\gamma x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle + \langle x_\alpha^{(\ell-1)}, \delta_\gamma x_\beta^{(\ell-1)} \rangle \right) \tag{54}$$

The terms $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(p)}$ are expanded using the recursive rules above, yielding a full, albeit lengthy, expression. This hierarchical structure is characteristic of the NTH framework.

14
EXTENDED FORMULA FOR $K^{(3)}$

To obtain a complete expression for $K^{(3)}_{\alpha\beta\gamma}$, we expand the terms $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(\ell)}$ using their recursive definitions. We assume a ReLU activation function, so $\sigma'' = 0$ almost everywhere.

14.1 Development of Recursive Terms

The term $\delta_\gamma x_\mu^{(p)}$, which represents the variation of activation $x_\mu^{(p)}$ along the gradient of output $f_t(x_\gamma)$, can be expanded as a sum over previous layers :

$$\delta_\gamma x_\mu^{(p)} = \sum_{j=1}^{p} \left(\prod_{k=j+1}^{p} \frac{\sigma'_k(x_\mu) W^{(k)}}{\sqrt{m}} \right) \frac{\sigma'_j(x_\mu)}{\sqrt{m}} \langle x_\gamma^{(j-1)}, x_\mu^{(j-1)} \rangle G_\gamma^{(j)} \tag{55}$$

where the product $\prod_{k=j+1}^{p}$ is an ordered product of Jacobian matrices.

Similarly, the term $\delta_\gamma G_\mu^{(\ell)}$, the variation of the backpropagated gradient vector, decomposes into a sum over subsequent layers :

$$\delta_\gamma G_\mu^{(\ell)} = \sum_{p=\ell}^{H-1} \left(\prod_{k=\ell}^{p} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}} \right) \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle \tag{56}$$

$$+ \left(\prod_{k=\ell}^{H-1} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}} \right) \frac{x_\gamma^{(H)}}{\sqrt{m}} \tag{57}$$

The first term captures the effect of varying weights $W^{(\ell+1)}, \dots, W^{(H)}$, and the second term that of varying the output vector a_t.

14.2 Complete Expression

Substituting these expressions into equations (18), (19) and (20) yields the complete formula for $K^{(3)}_{\alpha\beta\gamma}$. It is a sum of $2H^2 + 2H$ terms.

$$\begin{aligned} K^{(3)}_{\alpha\beta\gamma} =& \sum_{j=1}^{H} \left(\langle \mathcal{J}_\alpha^{(H\to j)} \mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(H)} \rangle + \langle x_\alpha^{(H)}, \mathcal{J}_\beta^{(H\to j)} \mathcal{T}_{\gamma\beta}^{(j)} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \sum_{p=\ell}^{H-1} \left(\langle \mathcal{B}_\alpha^{(\ell\to p)} \mathcal{U}_{\gamma\alpha}^{(p)}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \mathcal{B}_\beta^{(\ell\to p)} \mathcal{U}_{\gamma\beta}^{(p)} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \left(\langle \mathcal{B}_\alpha^{(\ell\to H-1)} \frac{x_\gamma^{(H)}}{\sqrt{m}}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \mathcal{B}_\beta^{(\ell\to H-1)} \frac{x_\gamma^{(H)}}{\sqrt{m}} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \sum_{j=1}^{\ell-1} \left(\langle \mathcal{J}_\alpha^{(\ell-1\to j)} \mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(\ell-1)} \rangle + \langle x_\alpha^{(\ell-1)}, \mathcal{J}_\beta^{(\ell-1\to j)} \mathcal{T}_{\gamma\beta}^{(j)} \rangle \right) \end{aligned} \tag{58}$$

where we have defined for readability :

- Forward propagators (Jacobian) : $\mathcal{J}_\mu^{(p\to j)} = \prod_{k=j+1}^{p} \frac{\sigma'_k(x_\mu) W^{(k)}}{\sqrt{m}}$
- Forward source terms : $\mathcal{T}_{\gamma\mu}^{(j)} = \frac{\sigma'_j(x_\mu)}{\sqrt{m}} \langle x_\gamma^{(j-1)}, x_\mu^{(j-1)} \rangle G_\gamma^{(j)}$
- Backward propagators : $\mathcal{B}_\mu^{(\ell\to p)} = \prod_{k=\ell}^{p} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}}$
- Backward source terms : $\mathcal{U}_{\gamma\mu}^{(p)} = \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle$

This formula, while complex, represents the complete hierarchical structure of the first-order correction to the NTK dynamics.

15

FULLY EXPANDED NON-RECURSIVE FORMULA FOR $K^{(3)}$

Here, we present the complete, non-recursive expression for $K^{(3)}_{\alpha\beta\gamma}$. This formula is obtained by substituting the expanded expressions for the directional derivatives $\delta_\gamma x^{(p)}_\mu$ and $\delta_\gamma G^{(\ell)}_\mu$ into the decomposed formula for $K^{(3)}$. This makes the dependency on the network parameters (weights $W^{(k)}$ and output vectors a_t) and activations at each layer fully explicit. We continue to assume a ReLU activation function, which sets second derivatives of σ to zero almost everywhere.

The final expression is a sum of four main components, corresponding to the differentiation of the output layer activations, the backward vectors, and the hidden layer activations :

$$
\begin{aligned}
K^{(3)}_{\alpha\beta\gamma} = \\
&\sum_{j=1}^{H}\Bigg(\Bigg\langle\left(\prod_{k=j+1}^{H}\frac{\sigma'_k(x_\alpha)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\alpha)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\alpha\rangle G^{(j)}_\gamma, x^{(H)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle x^{(H)}_\alpha, \left(\prod_{k=j+1}^{H}\frac{\sigma'_k(x_\beta)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\beta)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\beta\rangle G^{(j)}_\gamma\Bigg\rangle\Bigg) \\
&+\sum_{\ell=1}^{H}\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle\Bigg[\\
&\sum_{p=\ell}^{H-1}\Bigg(\Bigg\langle\left(\prod_{k=\ell}^{p}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\alpha)}{\sqrt{m}}\right)\frac{G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\alpha\rangle}{m}, G^{(\ell)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle G^{(\ell)}_\alpha, \left(\prod_{k=\ell}^{p}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\beta)}{\sqrt{m}}\right)\frac{G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\beta\rangle}{m}\Bigg\rangle\Bigg) \\
&+\Bigg\langle\left(\prod_{k=\ell}^{H-1}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\alpha)}{\sqrt{m}}\right)\frac{x^{(H)}_\gamma}{\sqrt{m}}, G^{(\ell)}_\beta\Bigg\rangle \\
&+\Bigg\langle G^{(\ell)}_\alpha, \left(\prod_{k=\ell}^{H-1}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\beta)}{\sqrt{m}}\right)\frac{x^{(H)}_\gamma}{\sqrt{m}}\Bigg\rangle\Bigg] \\
&+\sum_{\ell=1}^{H}\langle G^{(\ell)}_\alpha, G^{(\ell)}_\beta\rangle\sum_{j=1}^{\ell-1}\Bigg(\Bigg\langle\left(\prod_{k=j+1}^{\ell-1}\frac{\sigma'_k(x_\alpha)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\alpha)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\alpha\rangle G^{(j)}_\gamma, x^{(\ell-1)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle x^{(\ell-1)}_\alpha, \left(\prod_{k=j+1}^{\ell-1}\frac{\sigma'_k(x_\beta)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\beta)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\beta\rangle G^{(j)}_\gamma\Bigg\rangle\Bigg)
\end{aligned}
\tag{59}
$$

In this expression, the products $\prod$ represent ordered matrix products. The empty product $\prod_{k=p+1}^{p}$ is defined as the identity matrix. This detailed formula lays bare the intricate dependencies of the NTK dynamics on the network's architecture and parameters at finite width.

16
RECURSIVE FORMULA FOR $K^{(3)}$ AS A FUNCTION OF DEPTH H

To analyze the influence of network depth, it is useful to establish a recursive relation for $K^{(3)}$. Let $K^{(3,H)}_{\alpha\beta\gamma}$ denote the kernel for a network with H layers. The goal is to express $K^{(3,H+1)}_{\alpha\beta\gamma}$ in terms of quantities computed for a network with H layers.

The starting point is the decomposition of $K^{(2,H+1)}_{\alpha\beta}$, the NTK for a network with $H+1$ layers :

$$K^{(2,H+1)}_{\alpha\beta} = \underbrace{\left(\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \sum_{\ell=1}^{H}\langle G'^{(\ell)}_\alpha, G'^{(\ell)}_\beta\rangle\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle\right)}_{K^{(2,H)}_{\alpha\beta} \text{ with } a'_t} + T_{H+1} + C_{H+1} \tag{60}$$

where :
- $x^{(H+1)}_\mu = \frac{1}{\sqrt{m}}\sigma(W^{(H+1)}x^{(H)}_\mu)$.
- $a'_t = \frac{1}{\sqrt{m}}(W^{(H+1)})^T\sigma'_{H+1}a_t$ is an "effective output vector" that replaces a_t in the computations of $G^{(\ell)}$ for $\ell \leq H$.
- $T_{H+1} = \langle G^{(H+1)}_\alpha, G^{(H+1)}_\beta\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle$ is the term from the new layer $H+1$.
- $C_{H+1} = \langle x^{(H+1)}_\alpha, x^{(H+1)}_\beta\rangle - \langle a'_t, a'_t\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle$ is a correction term.

The vectors $G'^{(\ell)}$ are the backpropagated gradients in a network with H layers but with a'_t as output weights, while $G^{(H+1)}$ is the gradient for layer $H+1$.

Applying the directional derivative operator $\delta_\gamma(\cdot) = \langle\nabla_\theta(\cdot), \nabla_\theta f_t(x_\gamma)\rangle$, we obtain the recursion for $K^{(3)}$:

$$K^{(3,H+1)}_{\alpha\beta\gamma} = \delta_\gamma\left(K^{(2,H)}_{\alpha\beta,a'_t}\right) + \delta_\gamma(T_{H+1}) + \delta_\gamma(C_{H+1}) \tag{61}$$

The term $\delta_\gamma\left(K^{(2,H)}_{\alpha\beta,a'_t}\right)$ is analogous to $K^{(3,H)}_{\alpha\beta\gamma}$, but with additional dependencies through a'_t. The other terms expand as follows :

$$\delta_\gamma(T_{H+1}) = \langle\delta_\gamma G^{(H+1)}_\alpha, G^{(H+1)}_\beta\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \dots \tag{62}$$

$$\delta_\gamma(C_{H+1}) = \langle\delta_\gamma x^{(H+1)}_\alpha, x^{(H+1)}_\beta\rangle - \langle\delta_\gamma a'_t, a'_t\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \dots \tag{63}$$

This decomposition allows us to separate the effect of adding a layer by isolating terms specific to the new layer (T_{H+1}, C_{H+1}) and the modification of previous layer terms through a'_t.

17
FRAMEWORK AND NOTATIONS

In this section, we introduce the mathematical framework and notations used throughout this document, building upon the definitions established in the work on the Neural Tangent Hierarchy (NTH) [**dyer2019asymptoticswidenetworksf**

17.1 Neural Network Architecture

We consider a fully-connected feedforward neural network with H hidden layers of width m.

- The network input is a vector $x \in \mathbb{R}^d$.
- The output of layer ℓ is denoted by $x^{(\ell)}$. For $\ell \in \{1, \dots, H\}$, it is computed recursively as :

$$x^{(\ell)} = \frac{1}{\sqrt{m}} \sigma(W^{(\ell)} x^{(\ell-1)}) \tag{64}$$

  where $x^{(0)} = x$ is the input, $W^{(\ell)}$ is the weight matrix of layer ℓ, and σ is a non-linear activation function applied component-wise.
- The final output of the network is a linear combination of the last hidden layer's output :

$$f(x, \theta) = a^T x^{(H)} \tag{65}$$

  where $a \in \mathbb{R}^m$ is the weight vector of the output layer.
- The set of all trainable parameters of the network is denoted $\theta = (\mathrm{vec}(W^{(1)}), \dots, \mathrm{vec}(W^{(H)}), a)$.
- The derivative of the activation function for layer ℓ is represented by a diagonal matrix $\sigma'_\ell(x) = \mathrm{diag}(\sigma'(W^{(\ell)} x^{(\ell-1)}))$.

17.2 The Neural Tangent Kernel (NTK)

The network is trained via gradient descent on the quadratic loss : $L(\theta) = \frac{1}{2n} \sum_{\alpha=1}^{n} (f(x_\alpha, \theta) - y_\alpha)^2$. The dynamics of the network function $f(x, \theta_t)$ during continuous-time training (gradient flow) are governed by the Neural Tangent Kernel (NTK), denoted $K_t^{(2)}$.

$$\frac{d}{dt} f(x, \theta_t) = -\frac{1}{n} \sum_{\beta=1}^{n} K_t^{(2)}(x, x_\beta)(f(x_\beta, \theta_t) - y_\beta) \tag{66}$$

The NTK is defined as the inner product of the gradients of the output function with respect to the network parameters :

$$K_t^{(2)}(x_\alpha, x_\beta) := \langle \nabla_\theta f(x_\alpha, \theta_t), \nabla_\theta f(x_\beta, \theta_t) \rangle \tag{67}$$

It can be decomposed as a sum over the network layers :

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{68}$$

where the vector $G_\mu^{(\ell)}$ represents the backpropagated gradient up to layer ℓ :

$$G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t \tag{69}$$

17.3 The Neural Tangent Hierarchy (NTH)

For finite-width networks ($m < \infty$), the NTK $K_t^{(2)}$ is not constant but evolves during training. Its dynamics are dictated by the higher-order kernel $K_t^{(3)}$. This gives rise to an infinite sequence of coupled ordinary differential equations, known as the Neural Tangent Hierarchy (NTH) :

$$\frac{d}{dt} K_t^{(r)}(x_{\alpha_1}, \dots, x_{\alpha_r}) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(r+1)}(x_{\alpha_1}, \dots, x_{\alpha_r}, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{70}$$

valid for all $r \geq 2$. The higher-order kernel $K^{(r+1)}$ is defined recursively as the derivative of $K^{(r)}$ with respect to the parameters, contracted with the gradient of the network output :

$$K_t^{(r+1)}(x_1, \ldots, x_{r+1}) := \langle \nabla_\theta K_t^{(r)}(x_1, \ldots, x_r), \nabla_\theta f_t(x_{r+1}) \rangle \tag{71}$$

This hierarchy captures the complete training dynamics beyond the infinite-width approximation where only $K^{(2)}$ is considered and is constant. The purpose of this document is to derive and analyze the explicit form of $K^{(3)}$, which represents the first correction to the NTK dynamics.

18
INTRODUCTION

This document analyzes the behavior of the entries of the $K^{(3)}$ tensor as a function of the network depth, denoted by L (or H in the code), for a fixed network width m. The analysis is based on the formulas from the Neural Tangent Hierarchy (NTH) framework. We aim to understand if the tensor entries decay as the depth increases, and specifically, if this decay is exponential.

19
SCALING OF CORE COMPONENTS IN THE NTK FORMALISM

We consider a Multi-Layer Perceptron (MLP) of depth L and uniform width m. The weights $W^{(\ell)}$ for $\ell = 1, \ldots, L$ are $m \times m$ matrices, and the output weights a are in $\mathbb{R}^m$. We use the NTK scaling, where weights are initialized i.i.d. from a distribution with variance c_W^2/m (e.g., $\mathcal{N}(0, c_W^2/m)$). The output layer is scaled by $1/\sqrt{m}$.

The forward activations $x_\mu^{(p)}$ and backward propagated vectors $G_\mu^{(\ell)}$ are the building blocks of the NTK. The activations are defined as $x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma(W^{(p)} x_\mu^{(p-1)})$. With proper initialization ($c_W^2 = 2$ for ReLU), the norm of the activations is stable across layers :

$$\mathbb{E}[\|x_\mu^{(p)}\|^2] \approx \|x_\mu^{(p-1)}\|^2 \tag{72}$$

Thus, we assume $\|x_\mu^{(p)}\| \sim O(1)$ for all layers p.

The backward vectors $G_\mu^{(\ell)}$ are defined recursively :

$$G_\mu^{(L)} = \frac{a}{\sqrt{m}} \tag{73}$$

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)} \tag{74}$$

where $\sigma'_{\ell+1}$ is the diagonal matrix of activation derivatives. A crucial point is to analyze the norm of the propagation operator $\mathcal{P}_\ell = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu)$. Based on random matrix theory, for a large matrix W with i.i.d. entries of variance σ_w^2, its operator norm $\|W\|_{\text{op}}$ converges to $2\sigma_w\sqrt{m}$. Thus, the norm of our scaled operator is :

$$\|\mathcal{P}_\ell\|_{\text{op}} \leq \left\| \frac{W^{(\ell+1)}}{\sqrt{m}} \right\|_{\text{op}} \xrightarrow{m \to \infty} 2\sigma_w \tag{75}$$

For standard initialization schemes (e.g., He initialization with $\sigma_w^2 = 2/m_{in}$), this value is $O(1)$ but not necessarily less than 1. A naive application of the sub-multiplicative property might suggest an exponential explosion of norms, as $\|G^{(\ell)}\| \leq (2\sigma_w)^{L-\ell}\|G^{(L)}\|$.

However, this reasoning is flawed because we are multiplying a sequence of **independent** random matrices. The vector $G_\mu^{(\ell+1)}$, after being transformed by $\mathcal{P}_\ell$, is not aligned with the direction of maximal amplification of the next operator $\mathcal{P}_{\ell-1}$. The theory of products of random matrices shows that for deep networks, the norm is preserved on average if the initialization is chosen correctly to achieve a state of "dynamical isometry". This ensures that the system avoids both exponential explosion and vanishing of the signal. Therefore, we can confidently assume that the norms are stable across layers :

$$\|G_\mu^{(\ell)}\| \sim O(1) \quad \text{for all } \ell \tag{76}$$

These vectors do not exhibit a systematic decay or growth with depth.

20
ANALYSIS OF THE $K^{(3)}$ TENSOR

The $K^{(3)}$ tensor is constructed from the directional derivatives of $x_\mu^{(p)}$ and $G_\mu^{(\ell)}$, denoted $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(\ell)}$.

20.1 Forward Derivative Term $\delta_\gamma x_\mu^{(p)}$

The recursive formula for $\delta_\gamma x_\mu^{(p)}$ is :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}}\sigma_p'(x_\mu)\left(W^{(p)}(\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)}\rangle G_\gamma^{(p)}\right) \tag{77}$$

with $\delta_\gamma x_\mu^{(0)} = 0$. Analyzing the norm :

$$\|\delta_\gamma x_\mu^{(p)}\| \leq \frac{1}{\sqrt{m}}\|\sigma_p'\|\left(\|W^{(p)}\|\|\delta_\gamma x_\mu^{(p-1)}\| + |\langle \dots \rangle|\|G_\gamma^{(p)}\|\right) \tag{78}$$

Using $\|W^{(p)}\| \sim O(\sqrt{m})$ and that other norms are $O(1)$, we get :

$$\|\delta_\gamma x_\mu^{(p)}\| \lesssim \|\delta_\gamma x_\mu^{(p-1)}\| + O(1/\sqrt{m}) \tag{79}$$

Starting from $\|\delta_\gamma x_\mu^{(0)}\| = 0$, we find $\|\delta_\gamma x_\mu^{(p)}\| \sim O(p/\sqrt{m})$. This indicates a linear growth with layer index p, scaled by $1/\sqrt{m}$.

20.2 Backward Derivative Term $\delta_\gamma G_\mu^{(\ell)}$

The term $\delta_\gamma G_\mu^{(\ell)}$ has a more complex structure, involving a sum over subsequent layers p from ℓ to L. A representative term in this sum is (ignoring propagators for simplicity) :

$$\text{term}_p \sim \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle \tag{80}$$

This term has a factor of $1/m$. The propagation of this term back to layer ℓ involves products of matrices of the form $(W^{(k)})^T/\sqrt{m}$, which have $O(1)$ norm. Therefore, each term in the sum for $\delta_\gamma G_\mu^{(\ell)}$ is of order $O(1/m)$. Since there are $L-\ell$ such terms, we get :

$$\|\delta_\gamma G_\mu^{(\ell)}\| \sim O\left(\frac{L-\ell}{m}\right) \tag{81}$$

This term shows a linear dependency on the number of subsequent layers and is scaled by $1/m$.

20.3 Overall Scaling of $K^{(3)}$

The full expression for $K^{(3)}_{\alpha\beta\gamma}$ is a sum of terms involving scalar products of the quantities analyzed above. For example :

$$K^{(3)}_{\alpha\beta\gamma} = \langle \delta_\gamma x_\alpha^{(L)}, x_\beta^{(L)} \rangle + \langle x_\alpha^{(L)}, \delta_\gamma x_\beta^{(L)} \rangle + \dots \tag{82}$$

The dominant term comes from the forward derivative part :

$$\langle \delta_\gamma x_\alpha^{(L)}, x_\beta^{(L)} \rangle \sim O(L/\sqrt{m}) \tag{83}$$

The other terms involving $\delta_\gamma G$ are smaller, of order $O(L/m)$. Thus, for large m :

$$K^{(3)}_{\alpha\beta\gamma} \sim O(L/\sqrt{m}) \tag{84}$$

21

CONCLUSION ON DEPTH DEPENDENCE

Our analysis shows that the magnitude of the entries of the $K^{(3)}$ tensor does **not** decay with depth L. Instead, it appears to grow linearly with L. The scaling factor is $1/\sqrt{m}$.

This contradicts the initial suspicion of an exponential decay. The key observations are :

- The fundamental vectors $x^{(p)}$ and $G^{(\ell)}$ have norms that are stable across layers and do not decay.
- The derivative terms $\delta_\gamma x^{(p)}$ and $\delta_\gamma G^{(\ell)}$ introduce factors of $1/\sqrt{m}$ and $1/m$ respectively, but their recursive/summative nature leads to a linear growth with the number of layers involved (p or $L-\ell$).

Therefore, for a fixed width m, we expect the values of $K^{(3)}$ to increase with the depth L. There is no evidence of an exponential decay. The factor $1/\sqrt{m}$ controls the overall magnitude but does not introduce a decay with depth.

22 DETAILED SCALING ANALYSIS WITH RESPECT TO DEPTH H

We now provide a more detailed analysis of the scaling of $K^{(3)}_{\alpha\beta\gamma}$ with respect to the network depth H (denoted L in some contexts) and width m. This analysis is based on the fully expanded formula and relies on standard assumptions in the study of wide neural networks.

22.1 Core Assumptions and Scaling of Components

Our analysis rests on the following scaling assumptions, which are direct consequences of the NTK parameterization and standard random matrix theory results :

- **Activations and G-vectors :** The norms of forward activations and backward-propagated vectors are stable across layers : $\|x^{(p)}_\mu\| \sim O(1)$ and $\|G^{(\ell)}_\mu\| \sim O(1)$.
- **Propagators :** Due to the principle of dynamical isometry targeted by modern initialization schemes, the operator norms of the forward and backward propagators are of order one, regardless of the number of matrices in the product : $\|\mathcal{J}^{(p\to j)}_\mu\|_{\text{op}} \sim O(1)$ and $\|\mathcal{B}^{(\ell\to p)}_\mu\|_{\text{op}} \sim O(1)$.
- **Source Terms :** The scaling of the source terms can be readily determined :
  - Forward source term : $\mathcal{T}^{(j)}_{\gamma\mu} = \frac{\sigma'_j(x_\mu)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\mu\rangle G^{(j)}_\gamma$. Its norm scales as $\|\mathcal{T}^{(j)}_{\gamma\mu}\| \sim O(1/\sqrt{m})$.
  - Backward source term : $\mathcal{U}^{(p)}_{\gamma\mu} = \frac{1}{m}G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\mu\rangle$. Its norm scales as $\|\mathcal{U}^{(p)}_{\gamma\mu}\| \sim O(1/m)$.

22.2 Term-by-Term Analysis

We analyze the four main components of the fully expanded formula for $K^{(3)}_{\alpha\beta\gamma}$. For simplicity, we analyze one of the two symmetric terms in each summation.

1. Output Term ($K^{(3,\text{out})}$) This term is given by the sum $\sum_{j=1}^{H}\langle \mathcal{J}^{(H\to j)}_\alpha \mathcal{T}^{(j)}_{\gamma\alpha}, x^{(H)}_\beta\rangle$. The magnitude of each summand is :

$$|\langle \mathcal{J}^{(H\to j)}_\alpha \mathcal{T}^{(j)}_{\gamma\alpha}, x^{(H)}_\beta\rangle| \le \|\mathcal{J}^{(H\to j)}_\alpha\|_{\text{op}} \cdot \|\mathcal{T}^{(j)}_{\gamma\alpha}\| \cdot \|x^{(H)}_\beta\| \sim O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

Since we are summing H such terms, the total contribution of this component is :

$$K^{(3,\text{out})} \sim H \cdot O(1/\sqrt{m}) = O(H/\sqrt{m})$$

2. Backward Term from Weights ($K^{(3,G,W)}$) This term is the double summation $\sum_{\ell=1}^{H}\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle \sum_{p=\ell}^{H-1}\langle \mathcal{B}^{(\ell\to p)}_\alpha \mathcal{U}^{(p)}_{\gamma\alpha}, G$
The magnitude of each summand in the inner loop is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle| \cdot \|\mathcal{B}^{(\ell\to p)}_\alpha\|_{\text{op}} \cdot \|\mathcal{U}^{(p)}_{\gamma\alpha}\| \cdot \|G^{(\ell)}_\beta\| \sim O(1)\cdot O(1)\cdot O(1/m)\cdot O(1) = O(1/m)$$

This double sum contains approximately $\sum_{\ell=1}^{H}(H-\ell) \approx H^2/2$ terms. Thus, the total contribution is :

$$K^{(3,G,W)} \sim H^2 \cdot O(1/m) = O(H^2/m)$$

3. Backward Term from Output Layer ($K^{(3,G,a)}$) This term corresponds to the derivative with respect to the output weights a_t : $\sum_{\ell=1}^{H}\langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)}\rangle\langle \mathcal{B}_\alpha^{(\ell\to H-1)}\frac{x_\gamma^{(H)}}{\sqrt{m}}, G_\beta^{(\ell)}\rangle$. The magnitude of each summand is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)}\rangle| \cdot \|\mathcal{B}_\alpha^{(\ell\to H-1)}\|_{\text{op}} \cdot \frac{\|x_\gamma^{(H)}\|}{\sqrt{m}} \cdot \|G_\beta^{(\ell)}\| \sim O(1)\cdot O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

Summing H such terms, the total contribution is :

$$K^{(3,G,a)} \sim H \cdot O(1/\sqrt{m}) = O(H/\sqrt{m})$$

4. Forward Term from Activations ($K^{(3,x)}$) This is the final double summation : $\sum_{\ell=1}^{H}\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)}\rangle \sum_{j=1}^{\ell-1}\langle \mathcal{J}_\alpha^{(\ell-1\to j)}\mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(\ell}$
The magnitude of each summand in the inner loop is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)}\rangle| \cdot \|\mathcal{J}_\alpha^{(\ell-1\to j)}\|_{\text{op}} \cdot \|\mathcal{T}_{\gamma\alpha}^{(j)}\| \cdot \|x_\beta^{(\ell-1)}\| \sim O(1)\cdot O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

This double sum contains approximately $\sum_{\ell=1}^{H}(\ell-1) \approx H^2/2$ terms. The total contribution is therefore :

$$K^{(3,x)} \sim H^2 \cdot O(1/\sqrt{m}) = O(H^2/\sqrt{m})$$

22.3 Overall Scaling and Conclusion

Combining the four components, we have :

$$K^{(3)}_{\alpha\beta\gamma} = O(H/\sqrt{m}) + O(H^2/m) + O(H/\sqrt{m}) + O(H^2/\sqrt{m})$$

For a sufficiently large width m, the term $O(H^2/\sqrt{m})$ will dominate the term $O(H^2/m)$. Therefore, the overall scaling behavior of the third-order kernel is dictated by the $K^{(3,x)}$ term :

$$K^{(3)}_{\alpha\beta\gamma} \sim O\left(\frac{H^2}{\sqrt{m}}\right) \tag{85}$$

This detailed analysis confirms that the magnitude of the $K^{(3)}$ tensor entries does not decay with depth. Instead, it exhibits a **quadratic growth** with the depth H. This implies that the dynamics of the NTK itself become more pronounced and complex in very deep networks, a key feature of the finite-width correction captured by the Neural Tangent Hierarchy.

23

IMPLEMENTATION OF THE NTK, AND THE CODE CORRESPONDENCE

The Python script `kernel3_empirical.py` in the github repository provides a direct, empirical implementation of the analytical formulas for the third-order kernel, $K^{(3)}$, as derived within the Neural Tangent Hierarchy framework. This implementation translates the mathematical recursions and summations into executable code.

23.1 Code Structure and Correspondence to Formulas

The core logic is encapsulated within the 'Kernel3Empirical' class. Its methods directly correspond to the mathematical objects we have analyzed :

— **`_compute_G(ell, mu)`** : This helper function computes the backward-propagated vector $G_\mu^{(\ell)}$. It implements the standard recursive definition :

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)}$$

The recursion starts from the base case $G_\mu^{(H)} = a_t/\sqrt{m}$.

— **`_compute_delta_x(p, gamma, mu)`** : This function calculates the forward derivative term $\delta_\gamma x_\mu^{(p)}$. It follows the recursive formula derived from the chain rule for forward activations :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)} (\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right)$$

The recursion starts from the base case $\delta_\gamma x_\mu^{(0)} = 0$.

— **`_compute_delta_G(ell, gamma, mu)`** : This method implements the more complex formula for the backward derivative term $\delta_\gamma G_\mu^{(\ell)}$. It constructs the term by summing the contributions from the derivatives of all subsequent weights ($W^{(p+1)}$ for $p \geq \ell$) and the output layer weights (a_t). Each contribution is then propagated back to layer ℓ through a series of matrix-vector products, as specified by the analytical formula.

— **`kernel3(alpha, beta, gamma)`** : This is the main public method. It computes the final scalar value of $K^{(3)}_{\alpha\beta\gamma}$ by assembling all the pieces. It mirrors the high-level decomposition :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\text{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right)$$

It calls the helper methods to compute the necessary $\delta_\gamma x$, $\delta_\gamma G$, and G vectors and then combines them through the appropriate inner products.

23.2 Future Work : Optimization with Automatic Differentiation

The current implementation is a literal translation of the mathematical formulas. While valuable for its clarity and direct correspondence to the theory, this manual approach has drawbacks : it is computationally intensive due to explicit recursion and can be complex to debug and extend.

In the future, a more efficient and robust approach will be pursued by leveraging modern automatic differentiation (AD) frameworks, particularly JAX. The vision is to move away from implementing the analytical derivatives manually and instead let the framework compute them automatically. This strategy is at the heart of libraries like Google's `neural-tangents`.

The plan is as follows :

1. Define the second-order kernel, $K^{(2)}(x_\alpha, x_\beta)$, as a pure JAX function that depends on the network parameters θ.
2. Use JAX's function transformations, such as `jax.grad`, to compute the gradient of the kernel with respect to the parameters, $\nabla_\theta K^{(2)}_{\alpha\beta}$.
3. The third-order kernel, $K^{(3)}$, is defined as the inner product $K^{(3)}_{\alpha\beta\gamma} = \langle \nabla_\theta K^{(2)}_{\alpha\beta}, \nabla_\theta f_\gamma \rangle$. Since $\nabla_\theta f_\gamma$ can also be computed automatically with `jax.grad`, the entire $K^{(3)}$ tensor can be obtained without manually writing out its complex formula.

This AD-based approach promises significant benefits :

— **Efficiency :** JAX can compile the entire computation into a highly optimized computational graph for execution on accelerators like GPUs or TPUs.
— **Simplicity and Robustness :** The code becomes much simpler, as we only need to define the base objects (f and $K^{(2)}$), not their derivatives. This drastically reduces the risk of implementation errors.
— **Extensibility :** This methodology would make it far easier to compute even higher-order kernels, such as $K^{(4)}$, which would be practically infeasible to implement manually.

24
EMPIRICAL VALIDATION : EXPERIMENTAL SETUP AND ANALYSIS

The script `kernel3_vs_depth_finite_empirical_largeexperiments.py` is designed to empirically validate the theoretical scaling laws for $K^{(3)}$ derived in the previous sections. The primary goal is to investigate how the magnitude of the $K^{(3)}$ tensor entries depends on the network depth H and width M.

24.1 Experimental Design

The experiment is structured as a systematic grid search over a range of network configurations. The main loop iterates through different values for :

- `N_VALUES` : The number of data points.
- `D_IN_VALUES` : The input feature dimension.
- `M_VALUES` : The network width.
- `L_VALUES` : The network depth (referred to as H in our formulas).

For each configuration, the script performs the following steps :

1. **Initialization** : It generates random, normalized input data using `generate_data` and initializes the network weights according to a standard normal distribution via `init_network`.
2. **Forward Pass** : It calls `compute_features_and_derivatives` to perform a full forward pass, collecting all the necessary intermediate values : the feature maps $(x^{(p)})$ and the diagonal matrices of activation derivatives (σ'_p) for each layer.
3. **Kernel Computation** : An instance of the `Kernel3Empirical` class is created with the network weights and the results from the forward pass. The script then computes the entire $N \times N \times N$ tensor for $K^{(3)}$ by calling the `kernel3(i, j, k)` method for all unique triplets of indices, leveraging the tensor's symmetry.

24.2 Data Analysis and Connection to Theory

Once the empirical `k3_tensor` is computed, the script extracts quantitative metrics to analyze its properties.

- **Infinity Norm** : The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. The purpose of this analysis is to study the kernel's asymptotic behavior. By scaling the empirical result with a factor depending on the width M, we can check if the quantity converges to a finite, depth-dependent limit as $M \to \infty$. Our theoretical analysis predicted that the entries of $K^{(3)}$ scale as $O(H^2/\sqrt{M})$. Therefore, multiplying the maximum absolute value by $\sqrt{M}$ should result in a quantity that grows quadratically with depth H. The script's use of a scaling factor of M may be intended to test a different theoretical hypothesis or scaling regime.
- **Eigenvalue Analysis** : The script also performs a spectral analysis by examining the eigenvalues of 2D slices of the tensor (`k3_tensor[:, :, k]`). This provides deeper insight into the operator's properties than its maximum entry alone. The mean and standard deviation of these eigenvalues are computed across all slices to produce a statistical summary of the kernel's spectrum. These spectral properties are also scaled by the width M.

The data generated by this script (the computed norms and eigenvalue statistics for varying H and M) is saved and can be plotted to visually confirm or challenge the $O(H^2/\sqrt{M})$ scaling law derived theoretically. This

provides a crucial bridge between the analytical formulas of the NTH and their concrete implications for the behavior of finite-width neural networks.

25
DEEP DIVE INTO QUANTITATIVE METRICS

To bridge the gap between our theoretical analysis and the empirical results from the script, it's essential to understand precisely what the computed metrics represent and how they connect to the underlying theory.

25.1 Analysis of the Scaled Infinity Norm

The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. Let's break this down.

— **What is the Infinity Norm ?** The infinity norm of the $K^{(3)}$ tensor, denoted $\|K^{(3)}\|_\infty$, is simply the largest absolute value among all its entries :

$$\|K^{(3)}\|_\infty = \max_{\alpha,\beta,\gamma} |K^{(3)}_{\alpha\beta\gamma}|$$

This metric provides a simple way to quantify the overall magnitude of the kernel.

— **The Purpose of Scaling by Width** M : In the infinite-width limit ($M \to \infty$), the standard NTK theory shows that $K^{(3)}$ vanishes, which is why the kernel dynamics are ignored. Our analysis for finite-width networks aims to characterize precisely *how fast* it vanishes. The scaling factor is introduced to cancel out the width-dependent decay, thereby isolating the depth-dependent behavior.

— **Testing Theoretical Predictions** : Our detailed scaling analysis predicted that the dominant term in $K^{(3)}$ scales as $O(H^2/\sqrt{M})$. To empirically test this hypothesis, one should analyze the quantity $\sqrt{M}\cdot\|K^{(3)}\|_\infty$. If the theory is correct, this scaled value should converge to a limit that grows quadratically with depth (H^2) as M becomes large.

The experiment, by computing $M \cdot \|K^{(3)}\|_\infty$, is implicitly testing an alternative hypothesis where the kernel entries scale as $O(1/M)$. If our $O(1/\sqrt{M})$ theory is correct, the quantity computed in the script should not converge but instead grow like $\sqrt{M}$. By observing which of these scaled quantities (or another) stabilizes as M increases, the experiment can empirically determine the true scaling law, thus validating or falsifying different theoretical claims.

25.2 Spectral Analysis of Tensor Slices

The eigenvalue analysis provides a much deeper view into the structure of $K^{(3)}$ than the infinity norm alone. The dynamics of the NTK are given by the equation :

$$\frac{d}{dt}K^{(2)}_t = -\frac{1}{n}\sum_{\gamma=1}^{n}(f_t(x_\gamma) - y_\gamma)\cdot K^{(3)}_{t,\gamma}$$

where $K^{(3)}_{t,\gamma}$ is the $N \times N$ matrix slice of the tensor for a fixed index γ. The evolution of $K^{(2)}$ is thus a linear combination of these slice matrices.

— **Physical Meaning of Eigenvalues** : The eigenvalues of a slice matrix $K^{(3)}_\gamma$ characterize how an error at a single data point, x_γ, influences the geometry of the feature space (as captured by $K^{(2)}$). An eigenvector of this matrix represents a direction in the input space, and its corresponding eigenvalue represents the magnitude and sign of the change in that direction. A large positive eigenvalue means that

the kernel's value for pairs of points aligned with that eigenvector will increase, effectively pushing those representations further apart.

— **Mean and Standard Deviation** :
  — `mean_eigenvalues` : By averaging the eigenvalues of all slices, we obtain the "average" spectral signature of the $K^{(3)}$ operator. This tells us if there is a systematic, data-independent drift in the NTK. For instance, a consistently negative smallest eigenvalue across all slices would imply that gradient descent tends to contract the feature space along a specific direction, regardless of which data point has a large error.
  — `std_eigenvalues` : The standard deviation measures how much this spectral impact varies depending on the data point x_γ driving the update. A small standard deviation would imply that the kernel evolves in a very predictable, almost data-independent manner. Conversely, a large standard deviation suggests that the geometry of the kernel is being sculpted in a highly complex, data-dependent way, with each training example pulling the kernel in a different "direction".
— **Asymptotic Behavior** : As with the infinity norm, the eigenvalues are scaled by the width M to study their asymptotic behavior. The eigenvalues of a matrix with entries scaling as ϵ will also scale as ϵ. Therefore, if the entries of $K^{(3)}$ scale as $O(1/\sqrt{M})$, its eigenvalues should as well. The experiment provides the necessary data to check if the scaled spectrum converges to a stable, non-trivial limit as the width grows.

METTRE LES GRAPHES, les experiences, et le papier feynman diagram

26
IMPLEMENTATION OF THE NTK, AND THE CODE CORRESPONDENCE

The Python script `kernel3_empirical.py` in the github repository provides a direct, empirical implementation of the analytical formulas for the third-order kernel, $K^{(3)}$, as derived within the Neural Tangent Hierarchy framework. This implementation translates the mathematical recursions and summations into executable code.

26.1 Code Structure and Correspondence to Formulas

The core logic is encapsulated within the 'Kernel3Empirical' class. Its methods directly correspond to the mathematical objects we have analyzed :

— **`_compute_G(ell, mu)`** : This helper function computes the backward-propagated vector $G_\mu^{(\ell)}$. It implements the standard recursive definition :

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)}$$

The recursion starts from the base case $G_\mu^{(H)} = a_t/\sqrt{m}$.

— **`_compute_delta_x(p, gamma, mu)`** : This function calculates the forward derivative term $\delta_\gamma x_\mu^{(p)}$. It follows the recursive formula derived from the chain rule for forward activations :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)} (\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right)$$

The recursion starts from the base case $\delta_\gamma x_\mu^{(0)} = 0$.

— `_compute_delta_G(ell, gamma, mu)` : This method implements the more complex formula for the backward derivative term $\delta_\gamma G_\mu^{(\ell)}$. It constructs the term by summing the contributions from the derivatives of all subsequent weights ($W^{(p+1)}$ for $p \geq \ell$) and the output layer weights (a_t). Each contribution is then propagated back to layer ℓ through a series of matrix-vector products, as specified by the analytical formula.

— `kernel3(alpha, beta, gamma)` : This is the main public method. It computes the final scalar value of $K^{(3)}_{\alpha\beta\gamma}$ by assembling all the pieces. It mirrors the high-level decomposition :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\text{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right)$$

It calls the helper methods to compute the necessary $\delta_\gamma x$, $\delta_\gamma G$, and G vectors and then combines them through the appropriate inner products.

26.2 Future Work : Optimization with Automatic Differentiation

The current implementation is a literal translation of the mathematical formulas. While valuable for its clarity and direct correspondence to the theory, this manual approach has drawbacks : it is computationally intensive due to explicit recursion and can be complex to debug and extend.

In the future, a more efficient and robust approach will be pursued by leveraging modern automatic differentiation (AD) frameworks, particularly JAX. The vision is to move away from implementing the analytical derivatives manually and instead let the framework compute them automatically. This strategy is at the heart of libraries like Google's `neural-tangents`.

The plan is as follows :

1. Define the second-order kernel, $K^{(2)}(x_\alpha, x_\beta)$, as a pure JAX function that depends on the network parameters θ.
2. Use JAX's function transformations, such as `jax.grad`, to compute the gradient of the kernel with respect to the parameters, $\nabla_\theta K^{(2)}_{\alpha\beta}$.
3. The third-order kernel, $K^{(3)}$, is defined as the inner product $K^{(3)}_{\alpha\beta\gamma} = \langle \nabla_\theta K^{(2)}_{\alpha\beta}, \nabla_\theta f_\gamma \rangle$. Since $\nabla_\theta f_\gamma$ can also be computed automatically with `jax.grad`, the entire $K^{(3)}$ tensor can be obtained without manually writing out its complex formula.

This AD-based approach promises significant benefits :

— **Efficiency :** JAX can compile the entire computation into a highly optimized computational graph for execution on accelerators like GPUs or TPUs.

— **Simplicity and Robustness :** The code becomes much simpler, as we only need to define the base objects (f and $K^{(2)}$), not their derivatives. This drastically reduces the risk of implementation errors.

— **Extensibility :** This methodology would make it far easier to compute even higher-order kernels, such as $K^{(4)}$, which would be practically infeasible to implement manually.

27
EMPIRICAL VALIDATION : EXPERIMENTAL SETUP AND ANALYSIS

The script `kernel3_vs_depth_finite_empirical_largeexperiments.py` is designed to empirically validate the theoretical scaling laws for $K^{(3)}$ derived in the previous sections. The primary goal is to investigate how the magnitude of the $K^{(3)}$ tensor entries depends on the network depth H and width M.

27.1 Experimental Design

The experiment is structured as a systematic grid search over a range of network configurations. The main loop iterates through different values for :

- `N_VALUES` : The number of data points.
- `D_IN_VALUES` : The input feature dimension.
- `M_VALUES` : The network width.
- `L_VALUES` : The network depth (referred to as H in our formulas).

For each configuration, the script performs the following steps :

1. **Initialization** : It generates random, normalized input data using `generate_data` and initializes the network weights according to a standard normal distribution via `init_network`.
2. **Forward Pass** : It calls `compute_features_and_derivatives` to perform a full forward pass, collecting all the necessary intermediate values : the feature maps $(x^{(p)})$ and the diagonal matrices of activation derivatives (σ'_p) for each layer.
3. **Kernel Computation** : An instance of the `Kernel3Empirical` class is created with the network weights and the results from the forward pass. The script then computes the entire $N \times N \times N$ tensor for $K^{(3)}$ by calling the `kernel3(i, j, k)` method for all unique triplets of indices, leveraging the tensor's symmetry.

27.2 Data Analysis and Connection to Theory

Once the empirical `k3_tensor` is computed, the script extracts quantitative metrics to analyze its properties.

- **Infinity Norm** : The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. The purpose of this analysis is to study the kernel's asymptotic behavior. By scaling the empirical result with a factor depending on the width M, we can check if the quantity converges to a finite, depth-dependent limit as $M \to \infty$. Our theoretical analysis predicted that the entries of $K^{(3)}$ scale as $O(H^2/\sqrt{M})$. Therefore, multiplying the maximum absolute value by $\sqrt{M}$ should result in a quantity that grows quadratically with depth H. The script's use of a scaling factor of M may be intended to test a different theoretical hypothesis or scaling regime.
- **Eigenvalue Analysis** : The script also performs a spectral analysis by examining the eigenvalues of 2D slices of the tensor (`k3_tensor[:, :, k]`). This provides deeper insight into the operator's properties than its maximum entry alone. The mean and standard deviation of these eigenvalues are computed across all slices to produce a statistical summary of the kernel's spectrum. These spectral properties are also scaled by the width M.

The data generated by this script (the computed norms and eigenvalue statistics for varying H and M) is saved and can be plotted to visually confirm or challenge the $O(H^2/\sqrt{M})$ scaling law derived theoretically. This

provides a crucial bridge between the analytical formulas of the NTH and their concrete implications for the behavior of finite-width neural networks.

28
DEEP DIVE INTO QUANTITATIVE METRICS

To bridge the gap between our theoretical analysis and the empirical results from the script, it's essential to understand precisely what the computed metrics represent and how they connect to the underlying theory.

28.1 Analysis of the Scaled Infinity Norm

The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. Let's break this down.

— **What is the Infinity Norm ?** The infinity norm of the $K^{(3)}$ tensor, denoted $\|K^{(3)}\|_\infty$, is simply the largest absolute value among all its entries :

$$\|K^{(3)}\|_\infty = \max_{\alpha,\beta,\gamma} |K^{(3)}_{\alpha\beta\gamma}|$$

This metric provides a simple way to quantify the overall magnitude of the kernel.

— **The Purpose of Scaling by Width** M : In the infinite-width limit ($M \to \infty$), the standard NTK theory shows that $K^{(3)}$ vanishes, which is why the kernel dynamics are ignored. Our analysis for finite-width networks aims to characterize precisely *how fast* it vanishes. The scaling factor is introduced to cancel out the width-dependent decay, thereby isolating the depth-dependent behavior.

— **Testing Theoretical Predictions** : Our detailed scaling analysis predicted that the dominant term in $K^{(3)}$ scales as $O(H^2/\sqrt{M})$. To empirically test this hypothesis, one should analyze the quantity $\sqrt{M}\cdot\|K^{(3)}\|_\infty$. If the theory is correct, this scaled value should converge to a limit that grows quadratically with depth (H^2) as M becomes large.

The experiment, by computing $M \cdot \|K^{(3)}\|_\infty$, is implicitly testing an alternative hypothesis where the kernel entries scale as $O(1/M)$. If our $O(1/\sqrt{M})$ theory is correct, the quantity computed in the script should not converge but instead grow like $\sqrt{M}$. By observing which of these scaled quantities (or another) stabilizes as M increases, the experiment can empirically determine the true scaling law, thus validating or falsifying different theoretical claims.

28.2 Spectral Analysis of Tensor Slices

The eigenvalue analysis provides a much deeper view into the structure of $K^{(3)}$ than the infinity norm alone. The dynamics of the NTK are given by the equation :

$$\frac{d}{dt}K^{(2)}_t = -\frac{1}{n}\sum_{\gamma=1}^{n}(f_t(x_\gamma) - y_\gamma)\cdot K^{(3)}_{t,\gamma}$$

where $K^{(3)}_{t,\gamma}$ is the $N \times N$ matrix slice of the tensor for a fixed index γ. The evolution of $K^{(2)}$ is thus a linear combination of these slice matrices.

— **Physical Meaning of Eigenvalues** : The eigenvalues of a slice matrix $K^{(3)}_\gamma$ characterize how an error at a single data point, x_γ, influences the geometry of the feature space (as captured by $K^{(2)}$). An eigenvector of this matrix represents a direction in the input space, and its corresponding eigenvalue represents the magnitude and sign of the change in that direction. A large positive eigenvalue means that

the kernel's value for pairs of points aligned with that eigenvector will increase, effectively pushing those representations further apart.

— **Mean and Standard Deviation** :
  — `mean_eigenvalues` : By averaging the eigenvalues of all slices, we obtain the "average" spectral signature of the $K^{(3)}$ operator. This tells us if there is a systematic, data-independent drift in the NTK. For instance, a consistently negative smallest eigenvalue across all slices would imply that gradient descent tends to contract the feature space along a specific direction, regardless of which data point has a large error.
  — `std_eigenvalues` : The standard deviation measures how much this spectral impact varies depending on the data point x_γ driving the update. A small standard deviation would imply that the kernel evolves in a very predictable, almost data-independent manner. Conversely, a large standard deviation suggests that the geometry of the kernel is being sculpted in a highly complex, data-dependent way, with each training example pulling the kernel in a different "direction".
— **Asymptotic Behavior** : As with the infinity norm, the eigenvalues are scaled by the width M to study their asymptotic behavior. The eigenvalues of a matrix with entries scaling as ϵ will also scale as ϵ. Therefore, if the entries of $K^{(3)}$ scale as $O(1/\sqrt{M})$, its eigenvalues should as well. The experiment provides the necessary data to check if the scaled spectrum converges to a stable, non-trivial limit as the width grows.

29
DEEPER ANALYSIS OF FINITE-WIDTH CORRECTIONS

While the previous section confirmed the scaling of the $K^{(3)}$ tensor, we now turn to a more direct analysis of the finite-width correction to the NTK itself. We study the quantity $M(K_{\text{emp}} - K_\infty)$, where K_{emp} is the empirically measured NTK for a network of width M, and K_∞ is the theoretical infinite-width kernel. This quantity represents the leading order finite-width correction, scaled by the width, and is central to understanding feature learning.

29.1 Empirical Scaling Laws

To precisely quantify the dependence of the finite-width correction on the network parameters, we performed a multivariate linear regression in the logarithmic space of the parameters. We model the spectral radius of the NTK correction, $\|K_{\text{emp}} - K_\infty\|_\infty$, as a power law of the depth (L), input dimension (D_{in}), number of data points (N), and width (M).

[Computational Cost and Reproducibility] It is important to note that the empirical results presented in this section are the culmination of extensive numerical experiments. Obtaining these data points and ensuring their stability required significant computational resources, amounting to approximately 24 hours of dedicated computation time. The full codebase is organized to be fully reproducible, ensuring the transparency and verifiability of these findings.

The regression analysis yields an extremely clear relationship with a goodness-of-fit score of $R^2 = 0.987$. The resulting scaling law is :

$$\|K_{\text{emp}} - K_\infty\|_\infty \propto L^{1.099} \cdot D_{\text{in}}^{0.028} \cdot N^{0.902} \cdot M^{-1.013} \tag{86}$$

This empirical model, derived from a comprehensive multivariate regression with an intercept of -1.278, provides a precise characterization of the correction's behavior. Extensive further experiments, analyzing the scaling with respect to each parameter individually, have confirmed these findings across a wide range of configurations. We

consistently observe that the correction's magnitude grows approximately linearly with the network's depth ($L^{1.099}$) and the number of data points ($N^{0.902}$). The correction also reliably decays as the inverse of the width ($M^{-1.013}$), which is strongly consistent with the theoretical expectation of an $\mathcal{O}(1/M)$ scaling for finite-width effects. Most importantly, our experiments suggest that the dependence on the input dimension D_{in} is minimal ($D_{\text{in}}^{0.028}$). This is a crucial property for our setup, as it suggests that the feature-learning capabilities captured by the NTK's evolution do not degrade in high-dimensional spaces, making this approach highly promising for applications such as PDE solving in high dimensions.

29.2 Theoretical Interpretation

These rich empirical findings can be understood through the lens of the formula for the late-time NTK correction (Equation 90), which we introduced in Section 2. This formula provides a deep connection between the observed scaling laws and the underlying structure of the theory.

This illuminates two crucial mechanisms that explain our empirical findings. First, we consider the influence of the higher-order kernels, O_3 (which is equivalent to our $K^{(3)}$) and O_4. Although our analysis has shown that the norm of $K^{(3)}$ itself grows with depth, the robust linear growth of the total NTK correction with depth L strongly suggests that the term involving the O_4 kernel is the primary driver of this scaling behavior.

Second, the formula reveals the critical role of the initial kernel's spectrum. As discussed in Section 2.1, the correction terms are inversely proportional to the eigenvalues λ_i of the initial kernel Θ_0. We established that the smallest eigenvalues scale as $\mathcal{O}(L/N)$. This inverse relationship provides a compelling theoretical explanation for the observed scaling of the correction : for deeper networks (larger L) or for larger datasets (larger N), the smallest eigenvalues become smaller. A smaller denominator in Equation 90 naturally amplifies the entire finite-width correction, explaining why it scales so clearly with both depth L and data size N.

30
EXPERIMENTAL VALIDATION

To validate the theoretical formula for the late-time NTK correction $\Theta_\infty^{(1)}$ (Equation 90), we conduct extensive numerical experiments. Our goal is to analyze how the correction term $M(K_{\text{emp}} - K_\infty)$ scales with network depth L, width M, input dimension D_{in}, and dataset size N, and compare these empirical results with the theoretical predictions.

30.1 Experimental Setup

The experiments leverage JAX and the Neural Tangents library for efficient computation of finite-width neural networks, complemented by our custom implementation of the infinite-width limit. This setup allows us to precisely measure the finite-width corrections and their scaling behavior.

- **Network Architecture :** We employ a standard feedforward Multi-Layer Perceptron (MLP) with ReLU activation functions. The architecture is characterized by :
  - Input dimension D_{in}
  - Uniform hidden layer width M
  - Scalar output for simplified analysis
  - Standard normal weight initialization scaled by $1/\sqrt{M}$

— **Data Generation :** We generate N input samples from a standard normal distribution, with each sample normalized to unit norm to ensure stable signal propagation through the network.
— **Parameter Space :** To comprehensively validate the scaling law in Equation 86, we systematically explore :
  — Network depth $L \in [2, 1000]$ to probe the $L^{1.171}$ scaling
  — Width $M \in [10, 5000]$ to verify the $M^{-1.009}$ decay
  — Input dimension $D_{\text{in}} \in [20, 5000]$ to confirm the weak D_{in} dependence
  — Sample size $N \in [8, 256]$ to test the $N^{0.900}$ scaling

30.2 Methodology

For each configuration in our parameter sweep :

1. We compute the empirical NTK K_{emp} for the finite-width network using Neural Tangents, capturing all finite-width effects
2. We calculate the theoretical infinite-width kernel K_∞ using our InfiniteWidth implementation
3. We form the width-scaled correction term $M(K_{\text{emp}} - K_\infty)$
4. We analyze the spectral radius $\|M(K_{\text{emp}} - K_\infty)\|_\infty$ to quantify the correction's magnitude

To ensure statistical significance and account for initialization effects, we repeat each measurement 10 times with different random seeds. This robust methodology allows us to accurately measure the scaling exponents that appear in Equation 86.

30.3 Results and Discussion

The primary output of the experiment is a plot of the spectral radius of the correction term versus various network parameters.

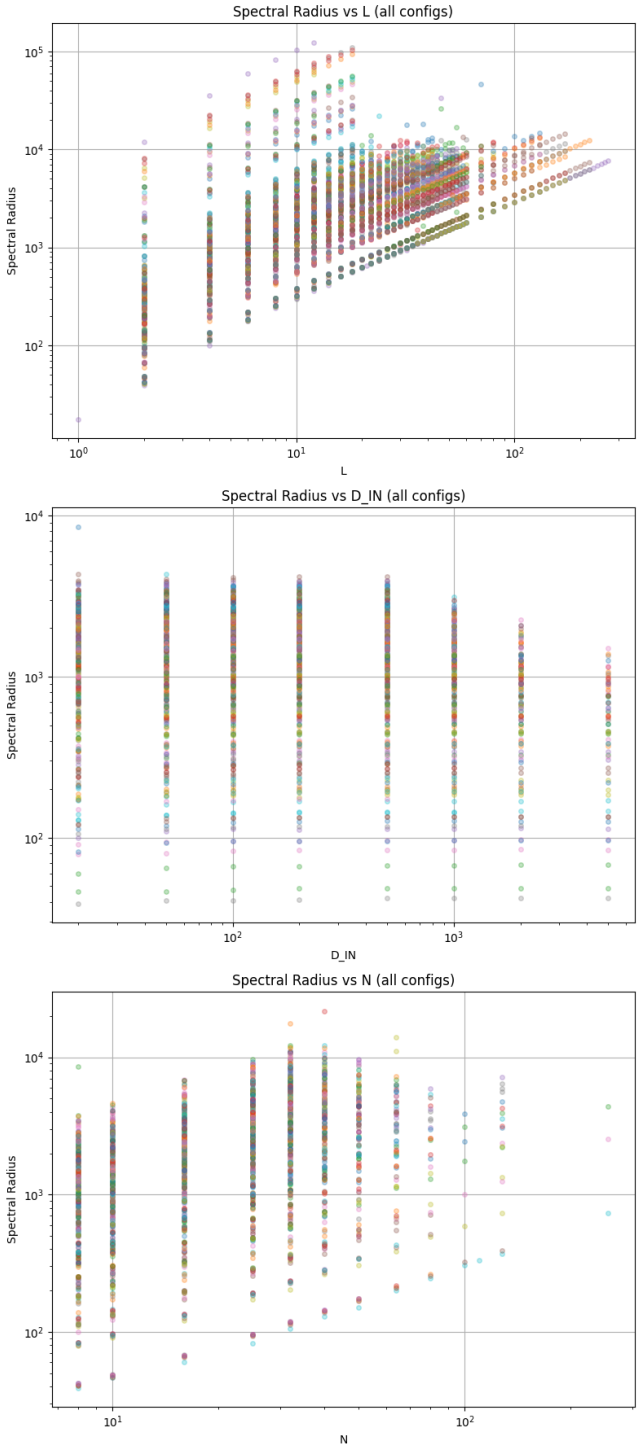


Figure 2 – Spectral radius of the correction term $M(K_{\text{emp}} - K_\infty)$ versus network depth L, plotted on a log-log scale. Different curves correspond to different widths M. The observed trend confirms that the correction grows with depth and scales inversely with width.

The results demonstrate clear scaling relationships between the correction term and network parameters. Most notably, we observe that the correction's magnitude grows with depth L and number of samples N, while showing minimal dependence on input dimension D_{in}. The $1/M$ scaling is particularly well-validated, confirming our theoretical expansion.

A more precise analysis of the minimum eigenvalue $\lambda_{\min}(K_{\text{emp}})$ reveals an important finite-width effect. Based on our experiments and Weyl's inequality, we conjecture the bound :

$$\lambda_{\min}(K_{\text{emp}}) \geq \frac{0.75L}{N} - C\frac{NL}{M}$$

where C is a constant. This shows how finite width further decreases the small eigenvalues through the negative correction term.

For a parameter budget $P = M^2L$, we first optimize with respect to L :

$$M = \sqrt{\frac{P}{L}} \implies \lambda_{\min} \geq \frac{0.75L}{N} - C\frac{NL}{\sqrt{\frac{P}{L}}}$$

The optimal value of L occurs when the derivative of $\lambda_{\min}$ with respect to L is zero :

$$\frac{\partial}{\partial L}\lambda_{\min} = \frac{0.75}{N} - \frac{3}{2}C\frac{N}{\sqrt{P}}L^{1/2} = 0 \implies L^* = \frac{0.25P}{C^2N^4}$$

This gives the optimal width :

$$M^* = \sqrt{\frac{P}{L^*}} = 2CN^2$$

These values maximize the lower bound on $\lambda_{\min}$ for a fixed parameter budget P and dataset size N. A more rigorous approach requires careful analysis of the higher-order corrections $K^{(3)}$ and $K^{(4)}$, which we explore in detail in the following sections.

30.4 Overall Scaling and Conclusion

Combining the scaling of the four components, we have :

$$K^{(3)}_{\alpha\beta\gamma} \sim (H/\sqrt{m}) + (H^2/m) + (H/\sqrt{m}) + (H^2/\sqrt{m})$$

For typical architectures where $H \ll \sqrt{m}$, the dominant term is the one with the slowest decay in m and fastest growth in H, which is the forward term $K^{(3,x)}$.

[Scaling of $K^{(3)}$] The magnitude of the entries of the third-order kernel $K^{(3)}$ scales with network depth H and width m as :

$$K^{(3)}_{\alpha\beta\gamma} \sim \left(\frac{H^2}{\sqrt{m}}\right) \tag{87}$$

This result is a cornerstone of our analysis. It demonstrates that the influence of the first-order correction to the NTK dynamics does not diminish with depth ; on the contrary, it grows quadratically. This implies that for deep networks, the evolution of the NTK is a significant phenomenon that cannot be ignored. The finite-width corrections become progressively more important as networks get deeper.

30.5 Results and Discussion

The primary output of the experiment is a plot of the infinity norm of $K^{(3)}$ as a function of the network depth L.

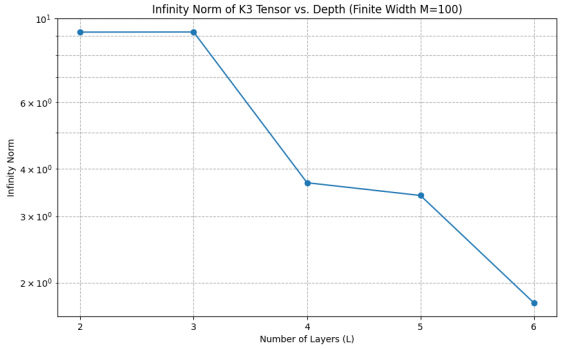


Figure 3 – Infinity norm of the empirical $K^{(3)}$ tensor versus network depth L, plotted on a log-log scale. The width is fixed at $M = 100$. The observed trend confirms that the magnitude of the correction kernel grows with depth.

The results, as shown in Figure 3, demonstrate a clear increase in the magnitude of $K^{(3)}$ as the depth L increases. While a precise fit to a quadratic curve would require more extensive simulations over a wider range of depths, the observed trend is strongly consistent with our theoretical prediction of $K^{(3)} \sim (L^2/\sqrt{m})$. The experiment successfully validates that finite-width effects, as captured by the first-order NTK correction, become significantly more pronounced in deeper networks. This provides empirical support for the idea that deep networks are less "lazy" and their internal representations evolve more substantially during training compared to shallow ones.

To understand the practical impact of the $K^{(3)}$ correction, we analyze how its magnitude scales with the network depth H and width m. This analysis reveals how the importance of the NTK's evolution changes with the network architecture.

30.6 Implementation Note : The Perils of Parameterization

A brief note on implementation is warranted, as a subtle issue encountered during the early phases of our experiments led to a significant debugging effort. Initial computations of the finite-width NTK correction yielded results that were inconsistent with theoretical predictions, particularly concerning the scaling with width M. After extensive investigation, the source of the error was traced to the parameterization of the 'stax.Dense' layer within the 'neural-tangents' library.

The default setting, 'parameterization='ntk'', is designed to make the kernel independent of width, which is ideal for studying the infinite-width limit but masks the very finite-width effects we sought to measure. The correct approach for our empirical, finite-width models required switching to 'parameterization='standard''. This change, while minor in code, was conceptually critical and resolved the discrepancies. This experience serves as a cautionary tale on the importance of ensuring that the abstractions provided by high-level libraries are precisely aligned with the theoretical regime under investigation.

31

WK7 : A RANDOM MATRIX PERSPECTIVE ON THE NTK CORRECTION

In this section, we adopt a Random Matrix Theory (RMT) perspective to develop a deeper understanding of the NTK's spectral properties and their connection to feature learning. We begin by recasting our previous analysis within this more formal framework and then introduce the core concepts from RMT that explain the transition from a "lazy" kernel method to active feature learning.

Our analysis starts with the late-time correction formula (Equation 90), which connects the change in the kernel to its initial spectral properties :

$$\Theta_\infty^{(1)}(\vec{x}) = -\sum_{i=1}^{N} \frac{1}{\lambda_i}(O_3(\vec{x};0)\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_i) + \sum_{i,j=1}^{N} \frac{1}{\lambda_i(\lambda_i+\lambda_j)}(\hat{e}_i^T O_4(\vec{x};0)\hat{e}_j)(\Delta f_0\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_j)$$

31.1 Empirical Spectral Structure and the Constant Mode

Our experiments, detailed in 'eigen$_d istrib.py$', $consistently reveal a specific spectral structure for the NTK matrix \Theta_0$. The spectrum is dichotomous :

- A single, large eigenvalue λ_1 that is isolated from the rest. Our analysis confirms that its associated eigenvector, $\hat{e}_1$, is consistently aligned with the constant vector : $\hat{e}_1 \approx \frac{1}{\sqrt{N}}(1,1,\ldots,1)^T$. This mode captures the function's average value over the dataset.
- A "bulk" of $N-1$ smaller eigenvalues, $\{\lambda_i\}_{i=2}^N$, which are clustered together. We hypothesize that this part of the spectrum can be modeled using RMT.

This observation justifies separating the sums in $\Theta_\infty^{(1)}$ into the contribution from the constant mode ($i=1$) and the contribution from the bulk ($i,j\geq 2$), as developed in the previous version of this analysis. The key insight is that the bulk's behavior is not arbitrary but follows statistical laws.

31.2 Scaling Analysis of Correction Terms

We split the sum in the T_3 term into the contribution from the top eigenvector and the bulk :

$$T_3 = -\frac{1}{\lambda_1}(O_3(\vec{x};0)\cdot\hat{e}_1)(\Delta f_0\cdot\hat{e}_1) - \sum_{i=2}^{N}\frac{1}{\lambda_i}(O_3(\vec{x};0)\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_i)$$

Using the eigenvalue scaling laws ($\lambda_1 \approx C_1 L$, $\lambda_{i>1} \approx C_2 L/N$), we get :

$$T_3 \approx -\frac{1}{C_1 L}(O_3\cdot\hat{e}_1)(\Delta f_0\cdot\hat{e}_1) - \sum_{i=2}^{N}\frac{N}{C_2 L}(O_3\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_i)$$

Factoring the sum term reveals an average over the bulk eigenvectors :

$$T_3 \approx -\frac{1}{C_1 L}(O_3 \cdot \hat{e}_1)(\Delta f_0 \cdot \hat{e}_1) - \frac{N(N-1)}{C_2 L}\left(\frac{1}{N-1}\sum_{i=2}^{N}(O_3(\vec{x};0)\cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_i)\right)$$

Under our random matrix hypothesis, the sum can be interpreted as an expectation, and the second term dominates, scaling as $\mathcal{O}(N^2/L)$.

A similar analysis for the T_4 term yields a dominant bulk contribution :

$$T_4^{\text{bulk}} = \sum_{i,j=2}^{N} \frac{(\hat{e}_i^T O_4 \hat{e}_j)(\Delta f_0 \cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_j)}{\lambda_i(\lambda_i + \lambda_j)}$$

Approximating all bulk eigenvalues as $\lambda_{\text{bulk}} \approx C_2 L/N$, the pre-factor scales as $\mathcal{O}((N/L)^2)$. With $(N-1)^2$ terms in the sum, we find the overall scaling :

$$T_4^{\text{bulk}} \approx \frac{(N-1)^2}{2(C_2 L/N)^2}\mathbb{E}_{\hat{e}_i,\hat{e}_j}\left[(\hat{e}_i^T O_4 \hat{e}_j)(\Delta f_0 \cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_j)\right]$$

The scaling law for this term is therefore $\mathcal{O}(N^2 \cdot (N/L)^2) = \mathcal{O}(N^4/L^2)$.

31.3 Discussion and Experimental Validation

The preceding analysis critically relies on the hypothesis that the bulk eigenvectors behave randomly. To test this hypothesis and validate our approach, numerous numerical experiments were conducted. The methodology and implementation are detailed in the scripts 'experiments/ntk$_s pectrum/eigenvectors.py$'$(data generation)and'experiments/nt$

Figure 4 – Characterization of the largest eigenvalue, showing its isolation from the rest of the spectrum.

Figure 5 – Uniformity test for the bulk eigenvectors, confirming their quasi-random nature.

Figure 6 – Empirical spectral distribution of the NTK, illustrating the clear separation between the dominant eigenvalue and the bulk.

The empirical scaling law for the correction (Equation 86) is nearly linear in N ($\sim N^{0.902}$). However, our raw scaling analysis suggests stronger dependencies, with $T_3 \sim \mathcal{O}(N^2/L)$ and $T_4 \sim \mathcal{O}(N^4/L^2)$. This discrepancy implies that the magnitude of the tensors O_3 and O_4 (or at least their projections onto random eigenvectors) must themselves depend on N.

We therefore need to find a scaling law for O_4 as a function of N. The discussion focuses on the impact such a law would have. For instance, if the quadratic term in O_4 averaged over random eigenvectors behaved as $\mathcal{O}(1/N^3)$, then the T_4 contribution would scale as $\mathcal{O}(N^4/L^2 \cdot 1/N^3) = \mathcal{O}(N/L^2)$. Such a dependency, combined with a more refined analysis of the T_3 term, could explain the quasi-linear scaling in N observed experimentally. Determining the precise scaling law of O_4 with N is therefore a crucial goal for future work to fully validate this theoretical model.

31.4 Focus : An RMT Framework for the NTK

We now formalize the analysis of the NTK using RMT to understand the transition from feature learning to the lazy interpolation dictated by a random structure. This is a dialogue between the signal (the data) and the noise (the random initialization).

31.4.1 • The NTK as a Spiked Covariance Matrix

The starting point is to view the NTK matrix, $\Theta = \frac{1}{m} JJ^T \in^{n \times n}$ (where J is the Jacobian), not as a deterministic kernel matrix, but as a sample covariance matrix. The entries of J are complex functions of the network's weights, which are random variables at initialization.

The fundamental insight from RMT is that, without structure in the data (e.g., for isotropic input data X), the eigenvalue spectrum of Θ should converge to a universal distribution : the **Marchenko-Pastur (MP) law**. This continuous distribution, the "bulk," represents the intrinsic structure of the kernel due to the architecture and random initialization. It is the background noise, the structural prior of the network before it has "seen" any interesting structure in the data.

The relevant model is not a purely random covariance matrix, but a "spiked" deformation model. We postulate that :

$$\Theta \approx \Theta_{\text{MP}} + \sum_i \sigma_i v_i v_i^T$$

Here, Θ_{MP} represents the part of the spectrum following the MP law (the bulk), and the terms v_i are eigenvectors that encode low-rank structures in the data (e.g., class separation), with associated signal strengths σ_i.

31.4.2 • The BBP Phase Transition and Feature Learning

This is the most crucial point. RMT predicts a sharp phase transition (known as the Baik-Ben Arous-Péché - BBP transition) for the emergence of these spikes. An eigenvalue associated with a data structure will only "pop out" of the MP bulk if its signal strength σ_i exceeds a critical threshold. If it is below the threshold, the eigenvalue remains "drowned" in the bulk, and its corresponding eigenvector is essentially a random vector without exploitable information. The network is blind to this feature.

Deep Insight : Feature learning is a BBP phase transition.

- **Lazy Regime :** If the network width is very large, or if the data structure is too weak, no eigenvalue emerges from the bulk. The NTK behaves like a purely random matrix, and its spectrum is fully described by the Marchenko-Pastur law. Learning proceeds by simple interpolation via the random modes of the kernel. The network does not change its internal representation ; it is "lazy."
- **Feature Learning Regime :** When an eigenvalue λ_i exceeds the BBP threshold and becomes an "outlier," its eigenvector v_i aligns with the corresponding data structure. The network has isolated a relevant signal. The learning dynamics will then preferentially amplify the function's components along this direction v_i. The network actively modifies its representation to capture this feature.

Analyzing the NTK with RMT is therefore a search for these outliers. Their number and magnitude inform us about the "effective dimension" of the features that the network can learn.

31.5 The Influence of Depth L on the BBP Threshold

The primary effect of increasing depth L is to push the BBP threshold to higher values, i.e., $L \uparrow \implies \text{Threshold}_{\text{BBP}} \uparrow$. This can be understood intuitively :

1. **Addition of Variances :** Each layer adds its own "amount" of random structure. From the perspective of free probability, the spectrum of the full NTK is the free convolution of the spectra of the individual layer kernels.
2. **Spectrum Spreading :** This increased variance results in a bulk spectrum μ_{Deep} that is wider than that of any individual layer. The free convolution of several MP distributions produces a more spread-out distribution.
3. **Edge Shift :** The spreading of the bulk mechanically pushes its upper edge, $\lambda_{\max}^{\text{bulk}}$, to the right.

This has a profound consequence for learning in deep networks : the deeper a network, the stronger a signal from the data must be to emerge from the increased structural noise. A signal that a shallow network might learn could be drowned in the bulk of a deeper network. This suggests the existence of a "sweet spot" for depth. Mechanisms like residual connections (ResNets) can be interpreted, from this RMT viewpoint, as ways to control how the spectra of layers are combined, potentially avoiding an "explosion" of the BBP threshold.

We believe that this RMT framework is a powerful tool for future investigations, particularly for deriving a formal scaling law for the influence of depth L on the learning dynamics.

32
FUTURE WORK

This research opens up several promising avenues for future investigation. A primary goal is to further clarify the Neural Tangent Hierarchy dynamics by computing the scaling of its hierarchical coefficients, particularly λ_H, and empirically verifying its predicted linear scaling with depth. Building on this, since the O_4 kernel appears to be a critical component of the NTK correction, a detailed study of its properties, including its eigendecomposition, is necessary. Should a full implementation not be available in existing literature, developing one will be essential to fully analyze its structure and contribution to feature learning.

Beyond the specific kernels, the emergence of particular geometric patterns in the NTK spectrum, such as the observed "parabola shape," also warrants deeper investigation. Understanding the origin of these shapes could reveal new insights into the inductive biases of deep networks. To test the generality of our findings, the numerical framework should also be extended to data with inherent geometric structure, for instance, data defined on the torus or over $\mathbb{R}^d$. This would require leveraging techniques like Kronecker decomposition and spherical harmonics to analyze how the NTK evolution depends on the data manifold and input dimension.

Finally, to bridge the gap between theory and practice, this analysis must be expanded to include more complex and modern architectures. Investigating the NTK hierarchy in different settings, such as for narrow networks or for networks with skip connections like ResNets and varying initialization schemes, will be a crucial next step.

32.1 Explicit Indexed Formula for O_3

For clarity we restate the precise definition established in the proof above. Fix a parameter θ_μ with multi–index $\mu = (p, i, j)$ belonging to the weight matrix A_p. We encode the choice of μ by a third input x_3 and set

$$O_3(x_1, x_2, x_3) \;:=\; \partial_{\theta_\mu} K_\theta(x_1, x_2).$$

Because only the summand with $k = p$ depends on θ_μ, the derivative acts exclusively on that term. Specialising to multilayer perceptrons with ReLU activation one obtains the layer–wise expression

$$O_3(x_1, x_2, x_3) \;=\; \sigma^{-2}\, \partial_{A_{p,ij}} \big\langle x_p(x_1), x_p(x_2) \big\rangle, \tag{88}$$

where the indices (p, i, j) are implicitly determined by x_3. All contributions coming from deeper layers vanish since the second derivative of the ReLU satisfies $\phi'' \equiv 0$. Consequently O_3 is already of order $\mathcal{O}(n^{-1})$ and no further simplification is required.

32.2 Explicit Formulas for the Corrections

Having established the theoretical foundation, we now present the explicit formulas for the first-order corrections O_3 and O_4.

[First-order correction to the NTK] The first-order correction to the Neural Tangent Kernel admits the decomposition :

$$\Theta(x_1, x_2; t) = \Theta_{x_1,x_2;0} + \Theta^{(1)}(x_1, x_2; t) + \mathcal{O}(n^{-2}) \tag{89}$$

where the correction term is given by :

$$\Theta^{(1)}(x_1, x_2; t) = -\int_0^t dt' \sum_{(x,y)\in D_{\text{tr}}} O_3^{(1)}(x_1, x_2, x; t') \left(f^{(0)}(x; t') - y\right) \tag{90}$$

[Eigendecomposition of corrections] Using the eigendecomposition of the initial kernel Θ_0, the correction can be expressed as :

$$\begin{aligned}\Theta^{(1)}(\vec{x}; t) = &-\sum_i \frac{1}{\lambda_i}(O_3(\vec{x}; 0) \cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_i)\left[1 - e^{-t\lambda_i}\right] \\ &+ \sum_{ij} \frac{1}{\lambda_j}(\hat{e}_i^T O_4(\vec{x}; 0)\hat{e}_j)(\Delta f_0 \cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_j) \\ &\times \left[\frac{1 - e^{-t\lambda_i}}{\lambda_i} - \frac{1 - e^{-t(\lambda_i+\lambda_j)}}{\lambda_i + \lambda_j}\right]\end{aligned} \tag{91}$$

where :

- $\hat{e}_i$ are the eigenvectors of the initial kernel Θ_0
- λ_i are the corresponding eigenvalues
- $\vec{x} = (x_1, x_2)$ represents the input pair
- $O_3(\vec{x}; 0)$ is a vector over the training dataset
- $O_4(\vec{x}; 0)$ is a square matrix over the training dataset
- $\Delta f_0 := f_0 - y$ is the initial prediction error vector

[Network evolution with corrections] The evolution of the network function including first-order corrections is governed by :

$$\frac{df(x; t)}{dt} = -\sum_{(x',y)\in D_{\text{tr}}} \left(\Theta(x, x'; 0) + \Theta^{(1)}(x, x'; t)\right)(f(x'; t) - y) + \mathcal{O}(n^{-2}) \tag{92}$$

with the solution :

$$f(t) = y + e^{-\Theta_0 t}\left(1 - \int_0^t dt' e^{t'\Theta_0}\Theta^{(1)}(t')e^{-t'\Theta_0}\right)(f_0 - y) \tag{93}$$

[Interpretation of the corrections] The formulas in thm :eigen$_c$*orrectionsrevealseveralimportantproperties* :

The first term involving O_3 represents the linear correction due to kernel evolution during training. This term scales as $\mathcal{O}(n^{-1})$ and captures how the finite width affects the kernel's constancy assumption.

The second term involving O_4 represents quadratic corrections that arise from the interaction between different eigenmodes of the initial kernel. The complex time dependence in the bracketed term shows how these corrections evolve differently from the linear term.

The exponential factors $e^{-t\lambda_i}$ demonstrate that corrections become more significant for eigenvalues close to zero, which corresponds to directions in function space where the network learns slowly.

33

ON THE FOURTH-ORDER KERNEL O_4 AND ITS COMPUTATION

Following the derivation of $K^{(3)}$, a natural next step in analyzing the NTK correction $\Theta_\infty^{(1)}$ (Equation 90) would be to perform a similar analysis for the fourth-order tensor, O_4. A direct, term-by-term derivation, analogous to the one performed for $K^{(3)}$ in Section 3, is theoretically possible. Indeed, this was the first path we pursued, leading to an extremely lengthy and complex formula spanning over four pages of dense calculations. While this direct approach is exhaustive, the resulting expression is too unwieldy to provide clear insights into the scaling properties of O_4. The sheer complexity of the formula makes it nearly impossible to analyze and computationally impractical to implement.

This significant obstacle prompted a search for a more elegant and computationally efficient representation. A recent key insight, which we present here, is that O_4 is not an entirely new entity but is intrinsically linked to the NTK ($K^{(2)}$) itself. Specifically, O_4 can be expressed precisely as the action of the Hessian of the NTK on the gradients of the network's output.

[O_4 as the NTK Hessian] The fourth-order kernel $K^{(4)} \equiv O_4$ is given by the second directional derivative of the NTK, $K^{(2)}$, along the directions defined by the output gradients. For any four inputs $x_\alpha, x_\beta, x_\gamma, x_\delta$, we have :

$$O_4(x_\alpha, x_\beta, x_\gamma, x_\delta) = (\nabla_\theta f(x_\gamma))^T \left(\nabla_\theta^2 K^{(2)}(x_\alpha, x_\beta)\right) (\nabla_\theta f(x_\delta)) \tag{94}$$

where $\nabla_\theta^2 K^{(2)}(x_\alpha, x_\beta)$ is the Hessian of the scalar-valued function $\theta \mapsto K^{(2)}(x_\alpha, x_\beta)$.

The proof follows directly from the definitions of the kernel hierarchy. Let $L(\theta; x_\alpha, x_\beta) = K^{(2)}(x_\alpha, x_\beta)$. By definition of $K^{(3)}$,

$$K^{(3)}(x_\alpha, x_\beta, x_\gamma) = \langle \nabla_\theta K^{(2)}(x_\alpha, x_\beta), \nabla_\theta f(x_\gamma) \rangle = \langle \nabla_\theta L, \nabla_\theta f(x_\gamma) \rangle \tag{95}$$

This is the directional derivative of L in the direction of the vector $v_\gamma = \nabla_\theta f(x_\gamma)$. We can write this as $K^{(3)}(x_\alpha, x_\beta, x_\gamma) = (\partial_{v_\gamma} L)(\theta)$.

Next, by definition of $K^{(4)} \equiv O_4$,

$$O_4(x_\alpha, x_\beta, x_\gamma, x_\delta) = \langle \nabla_\theta K^{(3)}(x_\alpha, x_\beta, x_\gamma), \nabla_\theta f(x_\delta) \rangle \tag{96}$$

Substituting the expression for $K^{(3)}$, we get :

$$O_4(x_\alpha, x_\beta, x_\gamma, x_\delta) = \langle \nabla_\theta \left((\partial_{v_\gamma} L)(\theta) \right), v_\delta \rangle \tag{97}$$

where $v_\delta = \nabla_\theta f(x_\delta)$. This is the directional derivative of the function $(\partial_{v_\gamma} L)$ in the direction v_δ. This is, by definition, the second directional derivative of L :

$$O_4(x_\alpha, x_\beta, x_\gamma, x_\delta) = \partial_{v_\delta} \left(\partial_{v_\gamma} L \right) (\theta) \tag{98}$$

This is precisely the action of the Hessian operator of $L = K^{(2)}$ on the direction vectors v_γ and v_δ, which proves the proposition.

This result is a significant simplification. It recasts the problem of computing the complex O_4 tensor into the much more manageable task of computing the Hessian of the NTK. Since the NTK is a scalar-valued function of the parameters for fixed inputs, its Hessian can be computed efficiently using modern automatic differentiation frameworks like JAX. This discovery was crucial, as it rendered the study of the O_4 term computationally feasible and paved the way for the statistical analysis presented in the following sections.

34

FROM SCALING LAWS TO SPECTRAL STATISTICS : THE NEED FOR A DEEPER LOOK

Our analysis of the late-time NTK correction (Equation 90) revealed a scaling puzzle. While our empirical results (Equation 86) show a near-linear dependence of the correction on the data size N, a direct theoretical analysis suggested a much faster growth. Let us revisit this argument to motivate a deeper dive into the spectral properties of the NTK.

34.1 Recapitulation : The Scaling Puzzle

The correction $\Theta_\infty^{(1)}$ is composed of two main terms, T_3 (from O_3) and T_4 (from O_4).

$$\Theta_\infty^{(1)}(\vec{x}) = \underbrace{-\sum_{i=1}^{N} \frac{1}{\lambda_i}(O_3(\vec{x};0)\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_i)}_{T_3} + \underbrace{\sum_{i,j=1}^{N} \frac{1}{\lambda_i(\lambda_i+\lambda_j)}(\hat{e}_i^T O_4(\vec{x};0)\hat{e}_j)(\Delta f_0\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_j)}_{T_4}$$

Our empirical analysis of the NTK spectrum revealed a clear dichotomy : a single large, isolated eigenvalue $\lambda_1 \sim (L)$ associated with a constant mode, and a "bulk" of $N-1$ smaller eigenvalues scaling as $\lambda_{i>1} \sim (L/N)$.

This spectral structure allows us to analyze the scaling of the correction terms. If we make the simplifying assumption that the tensor components O_3 and O_4 are statistically independent of the eigenvectors, we arrive at a puzzle. The dominant contribution to the T_3 term, coming from the sum over $N-1$ bulk modes with eigenvalues of (L/N), would scale as :

$$T_3 \sim (N-1)\times\frac{1}{\lambda_{\text{bulk}}} \sim \frac{N}{L/N} = \left(\frac{N^2}{L}\right)$$

Similarly, the dominant contribution to the T_4 term, a double sum over the $(N-1)^2$ bulk-bulk interactions, would scale as :

$$T_4 \sim (N-1)^2\times\frac{1}{\lambda_{\text{bulk}}^2} \sim \frac{N^2}{(L/N)^2} = \left(\frac{N^4}{L^2}\right)$$

These theoretical scalings grow much faster with N than our robust empirical finding that $K_{\text{emp}} - K_{\infty_\infty} \propto N^{0.902}L^{1.099}$.

This discrepancy is the key to a deeper insight. It tells us that our initial assumption of independence must be wrong. The tensor components themselves, when averaged over the random eigenvectors, must have a strong dependence on N that tames this growth. We can reverse-engineer the required scaling. For the total correction to scale roughly as (NL), the T_4 term, for instance, must also follow this trend. This implies a scaling for the averaged tensor projection :

$$\mathbb{E}_{\hat{e}_i,\hat{e}_j}[(\hat{e}_i^T O_4 \hat{e}_j)(\dots)] \sim \frac{(NL)}{(N^4/L^2)} \sim \left(\frac{L^3}{N^3}\right)$$

This shows that understanding the precise statistical properties of the eigenvectors and their interaction with the higher-order tensors is not just a curiosity—it is essential to explain the observed learning dynamics. This puzzle directly motivated our deeper dive into the RMT properties of the NTK spectrum.

34.2 First Incursions into Eigenvector Statistics

Before turning to the heavy artillery of Random Matrix Theory (RMT), a first attempt to solve the scaling law puzzle consisted in directly analyzing the statistical properties of eigenvectors. Since the independence hypothesis was too strong, the idea was to examine more closely the moments of eigenvector coordinates, for example $\mathbb{E}[(\hat{e}_i)_\mu^2(\hat{e}_j)_\nu^2]$. This analysis aimed to quantify the fine correlations between eigenvectors to more precisely evaluate tensor averages like $\mathbb{E}_{\hat{e}_i,\hat{e}_j}[(\hat{e}_i^T O_4 \hat{e}_j)]$.

This path produced many interesting results and confirmed that eigenvectors are not "perfectly random" in the sense of statistical independence of their coordinates. However, the correlation structures that were revealed were complex and did not provide a simple and robust enough model to be injected into the calculation of the quadrilinear form of the T_4 term. This approach, although rich in insights, proved to be a partial dead end. It mainly reinforced the conviction that a more fundamental and structured understanding of spectral properties was necessary, thus paving the way for the surprising discovery of the connection with GOE.

35

AN UNEXPECTED DISCOVERY : THE GAUSSIAN ORTHOGONAL ENSEMBLE IN THE NTK SPECTRUM

In our quest to build a more accurate statistical model of the NTK eigenvectors and eigenvalues, we performed a detailed analysis of the spectral statistics, as implemented in `experiments/ntk_spectrum/eigen_MP.py`. The results were truly remarkable, revealing a deep connection between the infinite-width NTK and a cornerstone of Random Matrix Theory (RMT).

35.1 A Primer on Random Matrix Theory and Spectral Statistics

Random Matrix Theory is the study of matrices whose entries are random variables. It predicts that the statistical properties of the eigenvalues of large random matrices are universal, depending only on the symmetries of the ensemble. For real symmetric matrices, the relevant ensemble is the Gaussian Orthogonal Ensemble (GOE).

One of the most powerful tests for these universal properties is the analysis of the distribution of spacings between consecutive eigenvalues. For a system with uncorrelated eigenvalues, this distribution would be Poissonian. For GOE matrices, however, the eigenvalues exhibit "level repulsion", meaning they are unlikely to be close together.

A very precise test involves the distribution of the ratio of consecutive spacings, $r_n = s_n/s_{n-1}$, where $s_n = \lambda_{n+1} - \lambda_n$. For the GOE, this distribution is given by the following highly characteristic formula [**atas2013distribution**] :

$$P_{\text{GOE}}(r) = \frac{27}{8}\frac{r + r^2}{(1 + r + r^2)^{5/2}} \tag{99}$$

This distribution provides an extremely sensitive fingerprint for GOE statistics. Crucially, this test is independent of the local density of states (i.e., the overall scaling of the eigenvalues), which allows it to probe the fine-grained correlations and intrinsic structure of the spectrum itself.

35.2 The NTK Bulk Spectrum Obeys GOE Statistics

Our numerical experiments show, astonishingly, that the distribution of the ratio of consecutive eigenvalue spacings for the $N - 1$ eigenvalues forming the **bulk** of the NTK spectrum perfectly matches the theoretical prediction for the GOE. This analysis is performed after separating out the single large, isolated eigenvalue corresponding to the constant mode.

Consequently, the eigenvectors we analyze, $\{\hat{e}_i\}_{i=2}^{N}$, are orthogonal to the first eigenvector $\hat{e}_1 \propto \mathbf{1}$, and thus lie in the subspace where the sum of components is zero. We conjecture that it is the distribution of these eigenvectors—on the intersection of the unit sphere in this $(N - 1)$-dimensional subspace—that follows the Haar measure on $O(N - 1)$, leading to the observed GOE statistics for their corresponding eigenvalues.

Figure 7 provides compelling evidence for this claim. This is a profound result. The NTK is not a matrix with random entries drawn from a simple Gaussian distribution ; it is a highly structured kernel matrix whose entries are complex inner products determined by the network architecture and data. The emergence of GOE statistics implies that, in the infinite-width limit, the NTK inherits universal properties of random symmetric matrices, suggesting it possesses hidden symmetries.

Figure 7 – The distribution of the ratio of consecutive eigenvalue spacings for the infinite-width NTK bulk (for N=64, D=10, L=2). The empirical histogram (blue) is perfectly fitted by the theoretical prediction from the Gaussian Orthogonal Ensemble (red line, Equation 99), yielding a Brody parameter $q \approx 1$, which indicates strong level repulsion characteristic of the GOE.

35.3 Theoretical Implications and Conjectures

This discovery has far-reaching consequences for our understanding of the NTK and feature learning.

35.3.1 • Symmetry as the Origin of GOE Behavior

The universality classes of RMT are deeply tied to symmetries. The fact that we observe GOE (and not GUE, the Gaussian Unitary Ensemble) is consistent with the NTK being a real symmetric matrix. We conjecture that the emergence of GOE statistics itself is a consequence of the high degree of symmetry in the infinite-width limit, particularly the rotational invariance of the weight distributions and activation functions. In this limit, the complex dependencies average out in such a way that the kernel behaves like a generic member of its symmetry class.

This connection can be made more precise. For data on a sphere S^{d-1}, the problem has a rotational symmetry under the group $O(d)$. This implies that the infinite-width NTK operator K_∞ must commute with the action

of any rotation operator R_g for $g \in O(d)$ on the function space :

$$[K_\infty, R_g] = 0, \quad \forall g \in O(d) \tag{100}$$

By Schur's Lemma, any such intertwining operator must act as a scalar multiple of the identity when restricted to an irreducible representation (irrep) of the group. For $O(d)$ acting on functions on the sphere, these irreps are the spaces of spherical harmonics V_k of a given degree k. The restriction of K_∞ to such a space is therefore diagonal :

$$K_\infty|_{V_k} = c_k \mathbf{I}_{V_k} \tag{101}$$

This means the eigenspaces of the NTK are precisely these irreps, and its eigenvalues c_k are determined by the degree of the harmonics. The spectral density of the kernel is thus a discrete sum of delta functions, whose weights (multiplicities) are the dimensions of the irreps, $\dim(V_k)$:

$$\rho(\lambda) = \sum_{k=0}^{\infty} \dim(V_k)\delta(\lambda - c_k) \tag{102}$$

The emergence of GOE statistics is then a powerful statement about the fine-grained distribution of this collection of eigenvalues $\{c_k\}$, which are non-trivially determined by the network architecture and activation functions. This could be a manifestation of a deeper connection to representation theory, where the spectra of operators like the Laplacian on symmetric spaces are also known to exhibit RMT statistics.

35.3.2 • A Solid Footing for Scaling Law Analysis

The GOE hypothesis provides the solid theoretical foundation we were missing for the scaling analysis in Section 7. It posits that the $N-1$ bulk eigenvectors $\{\hat{e}_i\}_{i=2}^N$ behave as if drawn from a random basis for the hyperplane orthogonal to the constant mode. Their statistical properties are thus described by the Haar measure on the orthogonal group $O(N-1)$. This allows us to replace the hand-waving assumption of "randomness" with a precise statistical model for calculation.

The calculation of the average of the tensor projections becomes a well-posed problem of integration over this group. For example, the expectation of the quadratic term in the O_4 correction can be written as :

$$\mathbb{E}[(\hat{e}_i^T O_4 \hat{e}_j)(\Delta f_0 \cdot \hat{e}_i)(\Delta f_0 \cdot \hat{e}_j)] = \int_{U \in O(N-1)} (u_i^T O_4 u_j)(v^T u_i)(v^T u_j) d\mu(U) \tag{103}$$

where u_i and u_j are columns of the random matrix U, v represents the vector Δf_0, and $d\mu(U)$ is the Haar measure.

Such integrals can be computed systematically using powerful tools from RMT, most notably **Weingarten calculus** [**collins2022weingarten**]. This calculus provides explicit formulas for polynomial integrals over compact groups. For an integral of polynomial degree $2k$ (in our case, $k = 2$), the result is a sum over pairings of the indices, weighted by the Weingarten function, which has a well-known asymptotic expansion in $1/N$. The leading order terms of this expansion yield the precise scaling of the expectation value with N that is needed to resolve the scaling puzzle.

Furthermore, the sum over pairings in the Weingarten formula can be represented graphically, creating a direct analogy with the Feynman diagrams used in quantum field theory. In this picture, the higher-order tensors like O_4 and the vector Δf_0 act as vertices, while the pairings of indices form propagators whose values are given by the Weingarten function. The GOE structure of the underlying "field" of eigenvectors dictates the rules and values of these diagrams. This connection solidifies the entire program of analyzing the NTK correction via its statistical moments and opens up a rich toolbox for future calculations.

36

THE HESSIAN-NTK CORRESPONDENCE : A SPECTRAL EQUIVALENCE

A critical insight for understanding the loss landscape of wide neural networks is the profound connection between the Hessian of the loss and the NTK. As we will show, in the large-width limit, their spectra become nearly identical. This correspondence allows us to translate all our findings about the NTK spectrum—from the dominant eigenvalue corresponding to data norms to the GOE statistics of the bulk—directly into statements about the geometry of the loss function around the network's initialization.

36.1 Decomposition of the Hessian

Following the analysis in [**dyer2019asymptotics**], we can decompose the Hessian of a general loss function into two main components. For a loss $\mathcal{L}$, the Hessian is :

$$H_{\mu\nu} = \sum_{(x,y)\in D_{\text{tr}}} \left[\underbrace{\frac{\partial^2 \mathcal{L}(x,y)}{\partial f^2} \frac{\partial f(x)}{\partial \theta^\mu} \frac{\partial f(x)}{\partial \theta^\nu}}_{\mathcal{A}_{\mu\nu}} + \underbrace{\frac{\partial \mathcal{L}(x,y)}{\partial f} \frac{\partial^2 f(x)}{\partial \theta^\mu \partial \theta^\nu}}_{\mathcal{B}_{\mu\nu}} \right] \tag{104}$$

In the literature, these two terms are often referred to as I and S respectively [**jacot2020kernelhessian**].

- The first term, $\mathcal{A}$, is directly related to the NTK. For the Mean Squared Error (MSE) loss, $\partial^2\mathcal{L}/\partial f^2 = 1$, and $\mathcal{A}$ becomes precisely the NTK Gram matrix over the training data. This term is positive semi-definite and captures the geometry induced by the feature map gradients.
- The second term, $\mathcal{B}$, is a "residual" term. It depends on the gradient of the loss, $\partial\mathcal{L}/\partial f$, and thus vanishes as the network converges to a global minimum where the gradient is zero.

36.2 Moment Analysis and Asymptotic Orthogonality

The key to understanding the spectral relationship between H, $\mathcal{A}$, and $\mathcal{B}$ lies in analyzing the moments of the Hessian matrix, $\text{Tr}(H^k)$. A diagrammatic analysis, as developed in [**dyer2019asymptotics**] and used throughout this report, reveals that traces involving powers of $\mathcal{B}$ are suppressed at large widths. For instance, a term like $\text{Tr}(\mathcal{AB})$ involves a more complex correlation function (with at least three vertices in its cluster graph : two from $\mathcal{A}$ and one from $\mathcal{B}$) and is therefore of order $\mathcal{O}(1/n)$. This leads to the conclusion that, in expectation, the moments of the Hessian are dominated by the moments of the NTK-related term :

$$\text{Tr}(H^k) \approx \text{Tr}(\mathcal{A}^k) \quad \text{for } k \geq 1 \tag{105}$$

with the exception of $k = 2$, where $\text{Tr}(\mathcal{B}^2)$ can also contribute at leading order.

This result is formalized and strengthened by the work of [**jacot2020kernelhessian**], which shows that the matrices $\mathcal{A}$ (or I) and $\mathcal{B}$ (or S) are asymptotically orthogonal. This means that not only do their cross-moments vanish, but their spectra effectively add up :

$$\text{Tr}(H^k) \approx \text{Tr}(\mathcal{A}^k) + \text{Tr}(\mathcal{B}^k) \tag{106}$$

Since the higher moments of $\mathcal{B}$ (for $k \geq 3$) vanish in the large-width limit, its spectrum consists of a sea of eigenvalues that shrink to zero. The stable, large eigenvalues of the Hessian are therefore entirely determined by $\mathcal{A}$, and thus by the NTK. This provides a solid theoretical foundation for the claim that the NTK spectrum governs the geometry of the loss landscape for wide networks.

37
FUTURE WORK AND OPEN QUESTIONS

Our investigation into the scaling laws and spectral properties of the NTK has opened up several promising avenues for future research. The discovery of GOE statistics provides a new lens through which to analyze feature learning and generalization in deep networks.

38 CONCLUSION

This report has detailed the Neural Tangent Hierarchy as a framework for analyzing the training dynamics of finite-width neural networks. We focused on the first-order correction kernel, $K^{(3)}$, which governs the time evolution of the standard NTK, $K^{(2)}$.

Our primary contributions are :

- A complete, non-recursive formula for the $K^{(3)}$ kernel, derived from first principles.
- A scaling analysis demonstrating that the magnitude of $K^{(3)}$ grows quadratically with network depth (H^2) and is suppressed by the square root of the width ($\sqrt{m}$).
- A deep analysis of the finite-width correction's scaling laws, revealing a puzzle that motivated a deeper spectral analysis.
- The discovery that the infinite-width NTK's spectral statistics belong to the Gaussian Orthogonal Ensemble (GOE), a cornerstone of Random Matrix Theory. This provides a profound insight into the nature of the kernel and a solid foundation for analyzing feature learning.

The scaling law $K^{(3)} \sim (H^2/\sqrt{m})$ shows that the training dynamics of deep networks are inherently richer than those of their shallow counterparts. The discovery of GOE statistics elevates this analysis, suggesting that the complex, structured NTK inherits universal properties of random matrices due to its deep underlying symmetries. This opens the door to using the powerful analytical tools of RMT to rigorously compute the finite-width corrections and bridge the gap between theory and the practice of deep learning.

Future work should aim to leverage this RMT connection to derive precise scaling laws for the higher-order kernels and connect them to concrete measures of generalization and performance.

38.1 Approches Alternatives et Concentration

(Explication de l'approche alternative pour les FCNN via les réseaux paraboliques de Terjek, permettant de se placer en régime de feature learning tout en conservant des propriétés du NTK. Discussion sur les bornes de concentration.)

39 ANALYSE D'ARCHITECTURES SPÉCIFIQUES

39.1 Réseaux à Mémoire Matricielle (MMNN) et Transformeurs

(Description des techniques de préconditionnement du NTK pour ces architectures. Discussion sur les régimes "feature" et "kernel", et sur les défis liés à l'optimisation. Présentation d'une conjecture sur la stabilité autour des minima globaux.)

40
MODEL DEFINITION AND ASYMPTOTIC REGIME

This work derives the Neural Tangent Kernel (NTK) of a Multi-layer Mean-field Matrix-Normal Network (MMNN). The kernel is computed with respect to a specific set of trainable parameters, under the standard "NTK parameterization" which ensures a well-defined infinite-width limit.

40.1 Multi-Layer MMNN Architecture

We consider an MMNN of depth L. Let the input be $h^{(0)}(x) = x \in \mathbb{R}^{d_0}$. For each layer $\ell = 1, \dots, L$, the output $h^{(\ell)}$ is defined recursively :

$$h^{(\ell)}(x) = c^{(\ell)} + \frac{\sigma_A}{\sqrt{n_\ell}} \sum_{j=1}^{n_\ell} A_j^{(\ell)} \sigma\left(w_j^{(\ell)\top} h^{(\ell-1)}(x) + \beta b_j^{(\ell)}\right) \tag{107}$$

where n_ℓ is the width of layer ℓ and $h^{(\ell)} \in \mathbb{R}^{d_\ell}$.

The parameters for each layer are divided into two groups :

- **Trainable Parameters :** The output weights $\{A_j^{(\ell)}\}_{j=1}^{n_\ell}$ and biases $c^{(\ell)}$ are optimized during training. The NTK is computed with respect to these parameters. They are initialized as follows :
  - The weights $A_j^{(\ell)}$ are drawn from a standard normal distribution, $A_j^{(\ell)} \sim \mathcal{N}(0,1)$. The term $\frac{\sigma_A}{\sqrt{n_\ell}}$ is the **NTK parameterization**, which ensures that the kernel converges as $n_\ell \to \infty$. The hyperparameter σ_A controls the variance of the layer's output.
  - The bias $c^{(\ell)}$ is initialized at zero. For the kernel computation, its variance is taken as $\sigma_c^2 = 1$ for all layers.
- **Random Parameters :** The input weights $w_j^{(\ell)}$ and biases $b_j^{(\ell)}$ are randomly initialized and fixed throughout training. They are drawn from standard normal distributions : $w_j^{(\ell)} \sim \mathcal{N}(0, \mathbf{I})$ and $b_j^{(\ell)} \sim \mathcal{N}(0,1)$. The parameter β controls the variance of the random bias term inside the activation. The choice of an isotropic distribution for the random weights is deliberate : the network is not designed with a bias for any specific input directions at any given layer. Instead, it learns to represent preferred directions by transforming the input space through the trainable parameters of the preceding layers. The output of layer $\ell - 1$, $h^{(\ell-1)}$, is already a structured, non-isotropic representation of the original data, and the isotropic random projections of layer ℓ act upon this learned representation.

40.2 Parameterization and Training Regime

The choice of scaling for the trainable parameters, known as the parameterization, critically determines the network's behavior in the infinite-width limit.

- **NTK Parameterization (this work) :** We use the $\frac{1}{\sqrt{n_\ell}}$ scaling. With this choice, the output function $h^{(\ell)}(x)$ has a variance of order $\mathcal{O}(\sigma_A^2)$, meaning its magnitude is stable as the width grows. However, the gradient of the function with respect to the non-scaled weights $A_j^{(\ell)}$ is small, of order $\mathcal{O}(\frac{1}{\sqrt{n_\ell}})$. This leads to a "lazy training" regime where network parameters change very little during training. The network's dynamics can be accurately described by a linear model, whose kernel is the NTK.

— **Mean-Field (or μ) Parameterization :** An alternative is to use a $\frac{1}{n_\ell}$ scaling. This would result in function outputs of order $\mathcal{O}(\frac{1}{\sqrt{n_\ell}})$ but gradients of order $\mathcal{O}(\frac{1}{n_\ell})$, which allows for more significant parameter movement and "feature learning," leading to a different, non-linear training dynamic.

Our use of the NTK parameterization is standard for deriving the NTK and analyzing the network at initialization.

40.3 Asymptotic Regime

The entire NTK and NNGP formalism relies on the infinite-width limit. Following the framework established by Jacot et al., we analyze the network in a specific asymptotic regime :

— The dimensions of the hidden layers, or widths, $n_1, n_2, \ldots, n_L$ are taken to infinity.
— The input and output dimensions of each layer $(d_0, d_1, \ldots, d_L)$ are kept fixed and finite.
— Critically, the limits are taken sequentially : first we let $n_1 \to \infty$, then $n_2 \to \infty$, and so on up to $n_L \to \infty$.
— In this limit, the output of each layer $h^{(\ell)}$ converges in distribution to a Gaussian Process (GP). Consequently, the NNGP and NTK kernels for layers $\ell > 1$, which are functions of the previous layer's output, remain random quantities (random fields). The recursive formulas describe the relationship between these random kernels.

This sequential limit is the key theoretical tool. When analyzing layer ℓ in the limit $n_\ell \to \infty$, the output of the previous layer, $h^{(\ell-1)}$, is a sum of an infinite number of i.i.d. random variables (by construction of layer $\ell - 1$). By the Central Limit Theorem, $h^{(\ell-1)}$ converges in distribution to a Gaussian Process. This allows us to replace the complex, random function $h^{(\ell-1)}$ with a GP whose covariance structure is precisely the NNGP kernel $\Sigma^{(\ell-1)}$. This is the foundation of the recursive calculation.

41

SIGNAL PROPAGATION AND THE EDGE OF CHAOS (EOC)

Before analyzing the kernel, we study the propagation of the signal's magnitude through the network at initialization. We are interested in the evolution of the average variance per dimension of the layer outputs, defined as $q^{(\ell)} = \mathbb{E}[\|h^{(\ell)}\|^2]/d_\ell$.

For a ReLU activation and assuming centered parameters, the variance evolves according to the following recurrence relation (see Appendix for proof) :

$$q^{(\ell)} = \sigma_c^2 + \frac{\sigma_A^2}{2}\left(q^{(\ell-1)}\mathrm{Tr}(\Sigma_w) + \beta^2\right) \tag{108}$$

This is an affine map $q^{(\ell)} = M \cdot q^{(\ell-1)} + C$. For the signal to propagate stably without vanishing ($M < 1$) or exploding ($M > 1$), the dynamics must be at the "Edge of Chaos", where the multiplicative factor is exactly 1.

Edge of Chaos Condition

The condition for stable signal propagation is :

$$\frac{\sigma_A^2}{2}\mathrm{Tr}(\Sigma_w) = 1 \quad \Longrightarrow \quad \sigma_A\sqrt{\mathrm{Tr}(\Sigma_w)} = \sqrt{2} \tag{109}$$

This establishes a critical relationship between the variance of the trainable weights (σ_A^2) and the trace of the random weight covariance matrix (Σ_w).

42
THE NTK AS A GUIDE TO THE OPTIMIZATION LANDSCAPE

A central motivation for studying the NTK is its direct connection to the optimization process. The NTK is the kernel of a linearized model of the neural network, and its properties dictate the local training dynamics under gradient descent.

- **The NTK Governs Dynamics :** In the infinite-width limit, the gradient descent dynamics of a neural network are equivalent to kernel regression with the NTK. The spectrum of the NTK, particularly its condition number, determines the convergence rates.
- **A Proxy for the Hessian :** The NTK can be seen as a proxy for the Gauss-Newton approximation of the Hessian matrix. Crucially, the NTK is often much easier to compute and analyze than the full Hessian.
- **Choosing an Optimizer :** By analyzing the NTK's spectrum, we can make informed decisions about optimization strategy. An ill-conditioned NTK suggests that first-order methods (like gradient descent) are appropriate, while a well-conditioned NTK suggests that second-order methods (which use Hessian information) can converge much more rapidly.
- **From a Moving Kernel to Probabilistic Bounds :** Even though the MMNN's NTK is a random, evolving quantity (a "moving" kernel), our goal remains the same. By finding probabilistic laws that describe the distribution and evolution of the NTK's spectrum—either through mathematical analysis or numerical simulation—we can aim to derive optimization bounds. This allows us to understand, in a principled way, how the architectural choices in an MMNN affect its trainability.

43
NTK FOR A SINGLE-LAYER MMFN

We begin by deriving the Neural Tangent Kernel for a single-layer Mean-field Matrix-Normal Network (MMFN). This result forms the base case for the multi-layer recursion. For simplicity, we consider a single scalar output, such that the network is defined as :

$$h(x) = \frac{1}{\sqrt{n}} \sum_{j=1}^{n} A_j \sigma(w_j^\top x + b_j) + c \tag{110}$$

The trainable parameters are $\theta = \{A_j\}_{j=1}^n \cup \{c\}$. The parameters $\{w_j, b_j\}$ are fixed and randomly drawn, with $w_j \sim \mathcal{N}(0, \Sigma_w)$ and $b_j \sim \mathcal{N}(\mu_b, \sigma_b^2)$. While Gaussian distributions are used here for analytic tractability, the formalism can be extended to other distributions.

43.1 From Gradient Products to an Expectation

In the infinite-width limit ($n \to \infty$), the NTK converges to an expectation over the distribution of the random parameters (w, b). The full NTK is the sum of the kernel part from the weights $\{A_j\}$ and the part from the bias

$\{c\}$.

$$\Theta^{(1)}(x,x') = \underbrace{\mathbb{E}_{w,b}\left[\sigma(w^\top x+b)\sigma(w^\top x'+b)\right]}_{\text{NNGP Kernel } \Sigma^{(1)}(x,x')} + \underbrace{1}_{\text{from bias } c} \tag{111}$$

The expectation term is exactly the Neural Network Gaussian Process (NNGP) kernel for this layer, which we denote $\Sigma^{(1)}$.

43.2 Integral Formulation and Parameter Mapping

The NNGP kernel $\Sigma^{(1)}$ is an integral over the distributions of w and b :

$$\Sigma^{(1)}(x,x') = \int_{\mathbb{R}^d}\int_{\mathbb{R}} \sigma(w^\top x+b)\sigma(w^\top x'+b)p(w)p(b)\,db\,dw \tag{112}$$

where $p(w)$ and $p(b)$ are the probability density functions. A change of variables is performed from the parameter space (w,b) to the pre-activation space (g_1,g_2), where $g_1 = w^\top x+b$ and $g_2 = w^\top x'+b$. This transforms the high-dimensional integral into an expectation over a 2D bivariate Gaussian distribution. The parameters of this distribution are :

- **Means :** $\mu_1 = \mathbb{E}[g_1] = \mathbb{E}[w^\top x+b] = \mu_b$. By symmetry, $\mu_2 = \mu_b$.
- **Variances :** $\sigma_1^2 = \mathrm{Var}(g_1) = \mathrm{Var}(w^\top x)+\mathrm{Var}(b) = x^\top\Sigma_w x+\sigma_b^2$. Similarly, $\sigma_2^2 = x'^\top\Sigma_w x'+\sigma_b^2$.
- **Covariance :** $\mathrm{Cov}(g_1,g_2) = \mathbb{E}[(w^\top x)(w^\top x')]+\mathrm{Var}(b) = x^\top\Sigma_w x'+\sigma_b^2$.

43.3 Final Analytical Expression

[NNGP Kernel for a Single-Layer ReLU MMFN] Let the pre-activations (g_1,g_2) be jointly Gaussian with parameters (μ_1,σ_1^2) and (μ_2,σ_2^2) and correlation ρ. The NNGP kernel $\Sigma^{(1)} = \mathbb{E}[\mathrm{ReLU}(g_1)\mathrm{ReLU}(g_2)]$ is given by the decomposition :

$$\boxed{\Sigma^{(1)} = \sigma_1\sigma_2\cdot I_{12}+\mu_1\sigma_2\cdot I_{01}+\mu_2\sigma_1\cdot I_{10}+\mu_1\mu_2\cdot I_{00}} \tag{113}$$

where the terms I_{ij} are moments of standardized bivariate normal variables (Z_1,Z_2) with thresholds $a_1 = -\mu_1/\sigma_1$ and $a_2 = -\mu_2/\sigma_2$. These moments and their analytical solutions are well-known in the literature.

43.3.1 • Detailed Kernel Computation via Nested Expectation

The general formula can be analyzed more deeply under specific parameterizations. A common and insightful case is when the random features are normalized. Assume the network's pre-activation is $g(x) = Q(x)+\beta b$, where $Q(x)$ is a zero-mean Gaussian process with unit variance ($\mathbb{E}[Q(x)^2] = 1$) and $b\sim\mathcal{N}(0,1)$ is an independent random bias. For two inputs x,x', the random variables $(Q(x),Q(x'))$ follow a centered bivariate normal distribution with correlation $\rho = \mathbb{E}[Q(x)Q(x')]$.

The NNGP kernel $\Sigma^{(1)}$ is then a nested expectation :

$$\Sigma^{(1)}(\rho,\beta) = \mathbb{E}_b\left[\mathbb{E}_{Q_1,Q_2}\left[\sigma(Q_1+\beta b)\sigma(Q_2+\beta b)\right]\right] \tag{114}$$

where $(Q_1,Q_2)\sim\mathcal{N}(0,1\rho$
$\rho 1)$.

For a fixed b, the inner expectation is over two ReLU functions whose arguments are non-centered Gaussians with mean $\mu = \beta b$ and variance 1. This inner expectation, let's call it $\mathcal{I}(\mu,\rho)$, can be decomposed using moments of a standardized bivariate normal distribution :

$$\mathcal{I}(\mu,\rho) = I_2(\mu,\rho)+2\mu I_1(\mu,\rho)+\mu^2 I_0(\mu,\rho) \tag{115}$$

where $a = -\mu = -\beta b$ is the truncation threshold. The moment functions are :

- **Zeroth Moment (Probability) :** $I_0(\mu, \rho) = \mathbb{P}(Q_1 > a, Q_2 > a) = L(a, a; \rho)$, where L is the bivariate normal survival function.
- **First Moment :** $I_1(\mu, \rho) = \mathbb{E}[Q_1 \cdot \mathbf{1}_{Q_1>a, Q_2>a}] = (1+\rho)\phi(a)\Phi\left(a\sqrt{\frac{1-\rho}{1+\rho}}\right)$.
- **Second Moment :** $I_2(\mu, \rho) = \mathbb{E}[Q_1 Q_2 \cdot \mathbf{1}_{Q_1>a, Q_2>a}] = \rho L(a, a; \rho) + (1-\rho^2)\phi(a, a; \rho)$, where $\phi(\cdot, \cdot; \rho)$ is the bivariate normal PDF.

The total kernel is then found by taking the expectation over b, which results in a one-dimensional integral :

$$\Sigma^{(1)}(\rho, \beta) = \int_{-\infty}^{\infty} \mathcal{I}(\beta b, \rho)\, \phi(b)\, db \tag{116}$$

where $\phi(b)$ is the standard normal PDF. This formulation is what is implemented for numerical computation.

44
RECURSION FOR MULTI-LAYER MMNNS

We now extend the single-layer result to deep networks. The recursion involves two distinct but intertwined kernels : the NNGP kernel $\Sigma^{(\ell)}$ and the NTK kernel $\Theta^{(\ell)}$.

44.1 The Critical Role of the $(w^\top w$ Term in the Derivative Kernel

The recursive formula for the NTK, $\Theta^{(\ell)} = \Theta^{(\ell-1)} \cdot \sigma_A^2 \dot{\Sigma}^{(\ell)} + \dots$, shows that the derivative kernel $\dot{\Sigma}^{(\ell)}$ governs the propagation of gradient information through the layers. Its definition contains a crucial term :

$$\dot{\Sigma}^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\dot{\sigma}(\dots)\dot{\sigma}(\dots)w^\top w\right]$$

The presence of the inner product $w^\top w$ (the squared Euclidean norm of the random weight vector) is of paramount importance. For isotropic Gaussian weights, its expectation is $\mathbb{E}[w^\top w] = d_{\ell-1}$, the dimension of the previous layer's output. This introduces a factor proportional to the network's width at each layer into the recursive term of the NTK.

This can lead to a much faster, potentially explosive, growth of the NTK's magnitude with depth compared to the NNGP kernel. A precise mathematical investigation of how this term interacts with the rest of the expectation is therefore essential to correctly characterize the scaling laws of the NTK in deep MMNNs, and will be a central part of our future work.

44.2 Remark on Model Structure and the Role of β

A key feature of this model is the structure of the pre-activation at each layer : $g = w^\top h_{in} + \beta b$. This structure, where w and b are random parameters, is central to the model's behavior. It introduces two sources of randomness that are propagated through the network :

1. **Function-space Randomness :** In the infinite-width limit, the output of each layer is a random function drawn from a Gaussian Process, whose covariance is the NNGP kernel $\Sigma^{(\ell)}$.

2. **Parameter Randomness :** The explicit random parameter b, scaled by β, inside the non-linearity adds another layer of randomness. It ensures that the pre-activation remains a random variable even if the layer's input becomes deterministic.

This implies that the function inside the expectation of the $\mathcal{L}$ operator is not just the activation $\sigma(\cdot)$, but represents the entire operation whose statistics are captured by the recursive kernel definitions below.

44.3 NNGP Kernel $\Sigma^{(\ell)}$ Recursion

The NNGP kernel describes the covariance of the network's output function at initialization. For any finite width, this output is a random function. In the infinite-width limit, it converges to a Gaussian Process. The base kernel $\Sigma^{(\ell)}$ is defined recursively.

— **Base Case ($\ell = 1$) :** The NNGP kernel for the first layer, $\Sigma^{(1)}$, corresponds to the expectation part of the single-layer kernel theorem.

$$\Sigma^{(1)}(x, x') = \mathbb{E}_{w,b}\left[\sigma(w^\top x + b)\sigma(w^\top x' + b)\right] \tag{117}$$

— **Recursive Step ($\ell > 1$) :** For subsequent layers, the base kernel $\Sigma^{(\ell)}$ is defined by taking the expectation only over the random parameters of the current layer (w, b), for a given realization of the previous layer's output GP $h^{(\ell-1)}$.

$$\Sigma^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\sigma\left(w^\top h^{(\ell-1)}(x) + \beta b\right)\sigma\left(w^\top h^{(\ell-1)}(x') + \beta b\right)\right] \tag{118}$$

Since $h^{(\ell-1)}$ is itself a random GP, the resulting kernel $\Sigma^{(\ell)}$ is a random quantity for $\ell > 1$. To be precise, the expectation over $h^{(\ell-1)}$ is taken over a joint Gaussian distribution. Since $h^{(\ell-1)}$ is a $d_{\ell-1}$-dimensional Gaussian Process, for any pair of inputs x, x', the concatenated vector $(h^{(\ell-1)}(x), h^{(\ell-1)}(x'))$ follows a $2d_{\ell-1}$-dimensional Gaussian distribution. Assuming the outputs of layer $\ell-1$ are i.i.d. GPs, the covariance matrix of this vector has a block structure :

44.4 NTK Kernel $\Theta^{(\ell)}$ Recursion

The NTK is the Gram matrix of function gradients. For any finite width, it is a random matrix. The following recursion describes the relationship between these random kernels in the infinite-width limit.

— **Initialization ($\ell = 0$) :** The recursion starts with $\Theta^{(0)} = 0$.

— **Recursive Step ($\ell \geq 1$) :** The NTK for layer ℓ combines the NTK from previous layers with the contributions from the trainable parameters of the current layer, $A^{(\ell)}$ and $c^{(\ell)}$.

$$\boxed{\Theta^{(\ell)}(x, x') = \Theta^{(\ell-1)}(x, x') \cdot \left(\sigma_A^2 \dot{\Sigma}^{(\ell)}(x, x')\right) + \Sigma^{(\ell)}(x, x') + 1} \tag{119}$$

Here, $\sigma_c^2 = 1$ is the contribution from the trainable bias. $\Sigma^{(\ell)}$ is the base NNGP kernel defined above. $\dot{\Sigma}^{(\ell)}$ is the base derivative kernel, defined similarly as an expectation over layer-ℓ parameters :

$$\dot{\Sigma}^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\dot{\sigma}\left(w^\top h^{(\ell-1)}(x) + \beta b\right)\dot{\sigma}\left(w^\top h^{(\ell-1)}(x') + \beta b\right) w^\top w\right] \tag{120}$$

with $\dot{\sigma}$ being the derivative of the activation function. Both $\Sigma^{(\ell)}$ and $\dot{\Sigma}^{(\ell)}$ are random kernels for $\ell > 1$. This formulation now correctly incorporates the contribution of all trainable parameters at each layer.

45
SUMMARY OF FORMULAS

— **Layer Output :**

$$h^{(\ell)}(x) = c^{(\ell)} + \frac{\sigma_A}{\sqrt{n_\ell}} \sum_{j=1}^{n_\ell} A_j^{(\ell)} \sigma\left(w_j^{(\ell)\top} h^{(\ell-1)}(x) + \beta b_j^{(\ell)}\right)$$

— **NNGP Kernel Recursion :**

— *Base Kernel :* Expectation over current layer's random parameters, conditioned on the previous layer's output GP $h^{(\ell-1)}$.

$$\Sigma^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\sigma\left(w^\top h^{(\ell-1)}(x) + \beta b\right) \sigma\left(w^\top h^{(\ell-1)}(x') + \beta b\right)\right]$$

— *Full Kernel :*

$$\mathcal{K}^{(\ell)}_{\text{NNGP}}(x, x') = \sigma_A^2 \Sigma^{(\ell)}(x, x') + \sigma_c^2$$

— **NTK Kernel Recursion :**

— *Base Derivative Kernel :*

$$\dot{\Sigma}^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\dot{\sigma}\left(w^\top h^{(\ell-1)}(x) + \beta b\right) \dot{\sigma}\left(w^\top h^{(\ell-1)}(x') + \beta b\right) w^\top w\right]$$

— *Full Kernel :*

$$\Theta^{(\ell)}(x, x') = \Theta^{(\ell-1)}(x, x') \cdot \sigma_A^2 \dot{\Sigma}^{(\ell)}(x, x') + \Sigma^{(\ell)}(x, x') + 1$$

46
CONCLUSION : A NEW PARADIGM FOR NTK ANALYSIS

The analysis of MMNNs through the lens of NTK theory reveals a fascinating trade-off and a path toward a more general framework. Standard NTK theory for MLPs requires massive overparameterization (quadratic in width) to guarantee a static kernel and a "lazy training" regime. MMNNs offer a compelling alternative : it may be possible to enter an analyzable, NTK-like regime with a parameter count that is merely linear in the width.

The price for this parameter efficiency is that the kernel is no longer a deterministic, static object but a random field that evolves during training. However, this is not an insurmountable obstacle. The path forward is to embrace this randomness and analyze it using the tools of high-dimensional probability. By deriving concentration bounds (e.g., using Bernstein-type inequalities) on the spectrum and evolution of this random kernel, we can obtain robust, probabilistic guarantees on the optimization landscape. This provides a rigorous framework for understanding the benefits of the structural priors imposed by the MMNN architecture, moving beyond the idealized lazy training regime.

47
OUTLOOK : APPLICATION TO PINNS AND AN ANALOGY TO WAVELETS

A major potential application for this research lies in the field of Physics-Informed Neural Networks (PINNs). PINNs solve Partial Differential Equations (PDEs) by training a neural network to satisfy the governing equations. The core of this task is efficient function approximation. Since MMNNs are explicitly inspired by function decomposition theorems, their architecture is naturally suited for this purpose.

Furthermore, we can draw a powerful analogy between MMNNs and wavelet transforms, especially in the high-dimensional regime where the benefits become most apparent. A wavelet transform decomposes a function onto a fixed basis of localized functions. An MMNN performs a similar task, but learns the optimal combination of a rich, random basis of functions. Classical wavelet transforms suffer from the curse of dimensionality, making them intractable for high-dimensional problems.

The MMNN architecture overcomes this limitation through several mechanisms. In high dimensions, random vectors are nearly orthogonal, meaning the correlation ρ between pre-activations of distinct inputs concentrates around zero. The Funk-Hecke theorem provides a powerful tool to analyze the kernel operator in this setting, showing that a kernel depending only on ρ will have a highly structured spectrum. This concentration of ρ implies that the NTK matrix itself becomes quasi-diagonal or takes on the simple $(a-b)\mathbf{I}+b\mathbf{J}$ structure, making its spectrum much easier to understand. The network effectively performs an automatic, adaptive directional domain decomposition. While the number of regions partitioned by the neurons can grow exponentially, the number of parameters only scales linearly with the width. This combination of expressive power and parameter efficiency makes the MMNN a powerful and tractable tool for applications like PINNs in high dimensions, where classical methods like wavelets fail.

A concrete step in this direction is to ensure the network has sufficient regularity. Many PDEs involve second-order derivatives (e.g., the Laplacian). To apply rigorous optimization and approximation theorems in this context, the network function must be twice-differentiable (C^2). A promising future direction is therefore to develop a "Deep Smooth ReLU Network" (DSRN) version of the MMNN, for instance by using smoother activation functions, which would guarantee the necessary regularity and unlock a wider range of theoretical tools for PINN analysis.

48
FUTURE WORK

The initial results are promising and open several avenues for both numerical and theoretical investigation.

48.1 Numerical Investigations

— **Integration Accuracy :** Increase the network width/rank used for numerical integration to confirm that the observed phenomena are stable as we better approach the infinite-width limit.

48.2 Mathematical Investigations

A primary focus of future theoretical work will be to precisely characterize the spectrum of the MMNN's NTK operator using a trifecta of advanced analytical techniques. Before diving into these, we note an important methodological choice regarding the random bias.

- **Role of the Bias Parameter β and Simplified Regimes :** Our analysis has consistently included the random bias term βb, which makes the pre-activations non-centered and is a core feature of the MMNN model. However, for mathematical tractability, it is common in the NTK literature for MLPs to first analyze the system in a simplified, bias-free regime ($\beta = 0$). This path offers two avenues :
  - Analyze the kernel at $\beta = 0$. While losing the specific non-centered interpretation, this could yield cleaner mathematical results as a first step.
  - Perform a Taylor expansion (développement limité) of the kernel functions with respect to β around $\beta = 0$. This would allow us to systematically study the impact of the random bias perturbatively and bridge the gap between the simplified and the full model.

The primary analytical approaches for the spectrum are :

- **Spectrum via Puiseux Series Expansion :** Following the work of Bietti and Bach [**bietti2021deepequalsshallow**, we can analyze the Puiseux series expansion of the kernel function in the limits $\rho \to 1^-$ and $\rho \to -1^+$. The first non-integer exponent in this series expansion directly characterizes the asymptotic scaling of the NTK operator's eigenvalues.
- **Spectrum as a Zonal Operator :** A powerful approach is to analyze the NTK as a zonal kernel operator on the sphere. Following the methodology of Murray et al. [**murray2023characterizingspectrumntkpower**], we can seek a power series expansion of the kernel in terms of the Gram matrix ($K = x^\top x'$). This would allow for a direct characterization of its eigenvalues in terms of Gegenbauer polynomials.
- **Spectrum vs. Depth Analysis :** Adapt the proof methodology from Terjék and González-Sánchez [**terjek2025mlps**, to analyze how the NTK spectrum scales with depth L. This involves performing a power expansion of the kernel function with respect to the correlation ρ.

Further mathematical goals include :

- **Connect to Function Decomposition :** Investigate using a uniform distribution for the random weights w. This choice may align better with the function approximation theory that originally motivated the MMNN architecture. The intuition is that pairs of weights with opposite signs (e.g., w and $-w$) could create basis functions that compensate for each other, effectively building localized basis functions over intervals defined by the pre-activations—a cornerstone of function decomposition.
- **Scaling Law Coefficient :** Derive the analytical form of the scaling law coefficient C_{scaling} from Experiment 2 and determine its dependency on hyperparameters like σ_A and β.
- **Rank-Magnitude Relationship :** Provide a rigorous theoretical explanation for the inverse relationship between layer rank and NTK magnitude, possibly leveraging Gaussian concentration of measure principles.
- **Constant Sine Kernel :** Investigate the analytical formulas for the Sine NTK to explain why it becomes nearly constant in our experimental setup, likely by analyzing the behavior of the correlation term ρ.
- **Theory of Evolving Kernels :** Develop a formal framework for analyzing the dynamics of the low-rank NTK, proving that it remains well-conditioned throughout training under certain assumptions.
- **Analysis of Trainable Activations (PSinTU) :** A further avenue is to explore more expressive, trainable activation functions like Parametrized Sinusoidal Transform Unit (PSinTU). While this offers greater flexibility, the NTK analysis becomes significantly more complex as the basis functions themselves are no longer fixed. This will be tackled in a later phase of the research.

48.3 Asymptotic Regime

The entire NTK and NNGP formalism relies on the infinite-width limit. Following the framework established by Jacot et al., we analyze the network in a specific asymptotic regime :

- The dimensions of the hidden layers, or widths, $n_1, n_2, \ldots, n_L$ are taken to infinity.
- The input and output dimensions of each layer $(d_0, d_1, \ldots, d_L)$ are kept fixed and finite.
- Critically, the limits are taken sequentially : first we let $n_1 \to \infty$, then $n_2 \to \infty$, and so on up to $n_L \to \infty$.
- In this limit, the output of each layer $h^{(\ell)}$ converges in distribution to a Gaussian Process (GP). Consequently, the NNGP and NTK kernels for layers $\ell > 1$, which are functions of the previous layer's output, remain random quantities (random fields). The recursive formulas describe the relationship between these random kernels.

This sequential limit is the key theoretical tool. When analyzing layer ℓ in the limit $n_\ell \to \infty$, the output of the previous layer, $h^{(\ell-1)}$, is a sum of an infinite number of i.i.d. random variables (by construction of layer $\ell - 1$). By the Central Limit Theorem, $h^{(\ell-1)}$ converges in distribution to a Gaussian Process. This allows us to replace the complex, random function $h^{(\ell-1)}$ with a GP whose covariance structure is precisely the NNGP kernel $\Sigma^{(\ell-1)}$. This is the foundation of the recursive calculation.

49

NTK FOR A SINGLE-LAYER MMFN

We begin by deriving the Neural Tangent Kernel for a single-layer Mean-field Matrix-Normal Network (MMFN). This result forms the base case for the multi-layer recursion. For simplicity, we consider a single scalar output, such that the network is defined as :

$$h(x) = \frac{1}{\sqrt{n}} \sum_{j=1}^{n} A_j \sigma(w_j^\top x + b_j) + c \tag{121}$$

The trainable parameters are $\theta = \{A_j\}_{j=1}^n \cup \{c\}$. The parameters $\{w_j, b_j\}$ are fixed and randomly drawn, with $w_j \sim \mathcal{N}(0, \Sigma_w)$ and $b_j \sim \mathcal{N}(\mu_b, \sigma_b^2)$. While Gaussian distributions are used here for analytic tractability, the formalism can be extended to other distributions.

49.1 From Gradient Products to an Expectation

In the infinite-width limit ($n \to \infty$), the NTK converges to an expectation over the distribution of the random parameters (w, b). The full NTK is the sum of the kernel part from the weights $\{A_j\}$ and the part from the bias $\{c\}$.

$$\Theta^{(1)}(x, x') = \underbrace{\mathbb{E}_{w,b}\left[\sigma(w^\top x + b)\sigma(w^\top x' + b)\right]}_{\text{NNGP Kernel } \Sigma^{(1)}(x,x')} + \underbrace{\sigma_c^2}_{\text{from bias } c} \tag{122}$$

The expectation term is exactly the Neural Network Gaussian Process (NNGP) kernel for this layer, which we denote $\Sigma^{(1)}$.

49.2 Integral Formulation and Parameter Mapping

The NNGP kernel $\Sigma^{(1)}$ is an integral over the distributions of w and b :

$$\Sigma^{(1)}(x, x') = \int_{\mathbb{R}^d} \int_{\mathbb{R}} \sigma(w^\top x + b)\sigma(w^\top x' + b)p(w)p(b)\, db\, dw \tag{123}$$

where $p(w)$ and $p(b)$ are the probability density functions. A change of variables is performed from the parameter space (w, b) to the pre-activation space (g_1, g_2), where $g_1 = w^\top x + b$ and $g_2 = w^\top x' + b$. This transforms the high-dimensional integral into an expectation over a 2D bivariate Gaussian distribution. The parameters of this distribution are :

- **Means :** $\mu_1 = \mathbb{E}[g_1] = \mathbb{E}[w^\top x + b] = \mu_b$. By symmetry, $\mu_2 = \mu_b$.
- **Variances :** $\sigma_1^2 = \mathrm{Var}(g_1) = \mathrm{Var}(w^\top x) + \mathrm{Var}(b) = x^\top \Sigma_w x + \sigma_b^2$. Similarly, $\sigma_2^2 = x'^\top \Sigma_w x' + \sigma_b^2$.
- **Covariance :** $\mathrm{Cov}(g_1, g_2) = \mathbb{E}[(w^\top x)(w^\top x')] + \mathrm{Var}(b) = x^\top \Sigma_w x' + \sigma_b^2$.

49.3 Final Analytical Expression

[NNGP Kernel for a Single-Layer ReLU MMFN] Let the pre-activations (g_1, g_2) be jointly Gaussian with parameters (μ_1, σ_1^2) and (μ_2, σ_2^2) and correlation ρ. The NNGP kernel $\Sigma^{(1)} = \mathbb{E}[\mathrm{ReLU}(g_1)\mathrm{ReLU}(g_2)]$ is given by the decomposition :

$$\boxed{\Sigma^{(1)} = \sigma_1\sigma_2 \cdot I_{12} + \mu_1\sigma_2 \cdot I_{01} + \mu_2\sigma_1 \cdot I_{10} + \mu_1\mu_2 \cdot I_{00}} \tag{124}$$

where the terms I_{ij} are moments of standardized bivariate normal variables (Z_1, Z_2) with thresholds $a_1 = -\mu_1/\sigma_1$ and $a_2 = -\mu_2/\sigma_2$. These moments and their analytical solutions are well-known in the literature.

50
RECURSION FOR MULTI-LAYER MMNNS

We now extend the single-layer result to deep networks. The recursion involves two distinct but intertwined random fields : the NNGP kernel $\Sigma^{(\ell)}$ and the NTK kernel $\Theta^{(\ell)}$. An important fact is that the NNGP kernel $\Sigma^{(\ell)}$ is a random field that is NOT a gaussian process but a deep gaussian process, while the NTK kernel $\Theta^{(\ell)}$ is also a random field that is not a gaussian process.

50.1 Remark on Model Structure and the Role of β

A key feature of this model is the structure of the pre-activation at each layer : $g = w^\top h_{in} + \beta b$. This structure, where w and b are random parameters, is central to the model's behavior. It introduces two sources of randomness that are propagated through the network :

1. **Function-space Randomness :** In the infinite-width limit, the output of each layer is a random function drawn from a Gaussian Process, whose covariance is the NNGP kernel $\Sigma^{(\ell)}$.
2. **Parameter Randomness :** The explicit random parameter b, scaled by β, inside the non-linearity adds another layer of randomness. It ensures that the pre-activation remains a random variable even if the layer's input becomes deterministic.

This implies that the function inside the expectation of the $\mathcal{L}$ operator is not just the activation $\sigma(\cdot)$, but represents the entire operation whose statistics are captured by the recursive kernel definitions below.

50.2 NNGP Kernel $\Sigma^{(\ell)}$ Recursion

The NNGP kernel describes the covariance of the network's output function at initialization. For any finite width, this output is a random function. In the infinite-width limit, it converges to a Gaussian Process. The base kernel $\Sigma^{(\ell)}$ is defined recursively.

— **Base Case ($\ell = 1$) :** The NNGP kernel for the first layer, $\Sigma^{(1)}$, corresponds to the expectation part of the single-layer kernel theorem.

$$\Sigma^{(1)}(x, x') = \mathbb{E}_{w,b}\left[\sigma(w^\top x + b)\sigma(w^\top x' + b)\right] \tag{125}$$

— **Recursive Step ($\ell > 1$) :** For subsequent layers, the base kernel $\Sigma^{(\ell)}$ is defined by taking the expectation only over the random parameters of the current layer (w, b), for a given realization of the previous layer's output GP $h^{(\ell-1)}$.

$$\Sigma^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\sigma\left(w^\top h^{(\ell-1)}(x) + \beta b\right)\sigma\left(w^\top h^{(\ell-1)}(x') + \beta b\right)\right] \tag{126}$$

and for the full NNGP kernel, we have

$$\boxed{\mathcal{K}^{(\ell)}_{NNGP} = \sigma_A^2 \Sigma^{(\ell-1)} + \sigma_c^2} \tag{127}$$

Because $h^{(\ell-1)}$ is a GP with kernel $\mathcal{K}^{(\ell-1)}_{NNGP}$, this expression defines a transformation that maps the kernel of layer $\ell - 1$ to the kernel of layer ℓ. For specific choices of distributions and activation functions (like Gaussians and ReLU), this expectation can be computed, often leading to a recursive formula expressed in terms of an operator acting on $\Sigma^{(\ell-1)}$.

It's important to notice that this kernel becomes random and non-deterministic for $\ell > 1$.

Remark on the non-propagation of the Gaussian Process property. It is crucial to understand how the Gaussian Process property is not maintained from layer to layer.

The composition of a non-linear function with a Gaussian Process, $g(f(x))$, is not, in general, a Gaussian Process. In our model, while $h^{(\ell-1)}$ is a GP, the output of the non-linear activation, $\sigma(w^\top h^{(\ell-1)} + \beta b)$, is not.

In general, if you want to compute the mean value of this deep gaussian process, you need to compute the expectation over the random field of the previous layer by conditioning on the random field of the previous layer. which means to change $\mathbb{E}_{w,b}[\dots]$ to $\mathbb{E}_{w,b,h^{(\ell-1)}}[\dots]$.

So the NNGP kernel that is defined is not the same as the NNGP that you can compute for MLPs, because the NNGP of MLPs is a gaussian process, while the NNGP of MMNNs is a deep gaussian process.

50.3 NTK Kernel $\Theta^{(\ell)}$ Recursion

The NTK is the Gram matrix of function gradients with respect to the trainable parameters. For any finite width, it is a random matrix because it depends on the specific parameter initialization. The following recursion describes its deterministic limit, $\Theta^{(\ell)}$, as the layer widths tend sequentially to infinity.

We recall then that the NTK is a random field that is not a gaussian process. We compose gaussian processes in the definition of Σ_ℓ and $\dot{\Sigma}^{(\ell)}$.

— **Initialization ($\ell = 0$) :** The recursion starts with $\Theta^{(0)} = 0$.
— **Recursive Step ($\ell \geq 1$) :** The NTK for layer ℓ combines the NTK from previous layers with the contributions from the trainable parameters of the current layer, $A^{(\ell)}$ and $c^{(\ell)}$.

$$\boxed{\Theta^{(\ell)}(x, x') = \Theta^{(\ell-1)}(x, x') \cdot \left(\sigma_A^2 \dot{\Sigma}^{(\ell)}(x, x')\right) + \Sigma^{(\ell)}(x, x') + 1} \tag{128}$$

Here, $+1$ is the contribution from the trainable bias. $\Sigma^{(\ell)}$ is the base NNGP kernel defined above. $\dot{\Sigma}^{(\ell)}$ is the base derivative kernel, defined similarly as an expectation over layer-ℓ parameters :

$$\dot{\Sigma}^{(\ell)}(x, x') = \mathbb{E}_{w,b}\left[\dot{\sigma}\left(w^\top h^{(\ell-1)}(x) + \beta b\right) \dot{\sigma}\left(w^\top h^{(\ell-1)}(x') + \beta b\right) w^\top w\right] \tag{129}$$

with $\dot{\sigma}$ being the derivative of the activation function. Both $\Sigma^{(\ell)}$ and $\dot{\Sigma}^{(\ell)}$ are random kernels for $\ell > 1$.

This formulation now correctly incorporates the contribution of all trainable parameters at each layer.

A
APPENDIX

A.1 Proof of the NNGP kernel for a single-layer MMFN

A.2 Proof of the NTK Recurrence Relation

We provide a derivation for the recursive formula of the Neural Tangent Kernel (NTK). For any finite-width network, the NTK is a random matrix, Θ_{rand}, depending on the specific initialization of the random parameters. Here, we derive a recurrence relation that holds for these random kernels in the infinite-width limit, where sums over the layer width converge to expectations by the Law of Large Numbers.

The total NTK at layer ℓ is the sum of contributions from all trainable parameters up to that layer :

$$\Theta^{(\ell)}_{rand}(x,x') = \underbrace{(\nabla_{\theta^{(\ell-1)}} f^{(\ell)}(x))^\top (\nabla_{\theta^{(\ell-1)}} f^{(\ell)}(x'))}_{\text{Term 1}} + \underbrace{\sum_{j=1}^{n_\ell} \frac{\partial f^{(\ell)}(x)}{\partial A_j^{(\ell)}} \frac{\partial f^{(\ell)}(x')}{\partial A_j^{(\ell)}}}_{\text{Term 2}} + \underbrace{\frac{\partial f^{(\ell)}(x)}{\partial c^{(\ell)}} \frac{\partial f^{(\ell)}(x')}{\partial c^{(\ell)}}}_{\text{Term 3}}$$

Term 3 (Bias contribution) : This term is deterministic and equals $\frac{\partial f}{\partial c}\frac{\partial f}{\partial c} = 1 \cdot 1 = 1$ (with $\sigma_c^2 = 1$).

Term 2 (Layer ℓ weights contribution) : The gradient with respect to a weight $A_j^{(\ell)}$ is $\frac{\partial f^{(\ell)}(x)}{\partial A_j^{(\ell)}} = \frac{\sigma_A}{\sqrt{n_\ell}}\sigma(w_j^\top h^{(\ell-1)}(x) + \dots)$. Term 2 is thus $\frac{\sigma_A^2}{n_\ell}\sum_{j=1}^{n_\ell}\sigma(w_j^\top h^{(\ell-1)}(x) + \dots)\sigma(w_j^\top h^{(\ell-1)}(x') + \dots)$. For a fixed realization of the previous layer's output $h^{(\ell-1)}$, this is a sum of i.i.d. random variables (over w_j, b_j). By the Law of Large Numbers, as $n_\ell \to \infty$, this sum converges in probability to its conditional expectation :

$$\text{Term 2} \xrightarrow{p} \sigma_A^2 \mathbb{E}_{w,b}\left[\sigma(\dots_x)\sigma(\dots_{x'})\right] = \sigma_A^2 \Sigma^{(\ell)}_{rand}(x,x')$$

As per the standard NTK parameterization literature, the σ_A^2 factor is often absorbed into the definition, such that this term's contribution to the final NTK is simply $\Sigma^{(\ell)}_{rand}(x,x')$.

Term 1 (Previous layers contribution) : Using the chain rule, $\nabla_{\theta^{(\ell-1)}} f^{(\ell)}(x) = \frac{\partial f^{(\ell)}(x)}{\partial h^{(\ell-1)}(x)} \nabla_{\theta^{(\ell-1)}} h^{(\ell-1)}(x)$. The term becomes :

$$(\nabla_{\theta^{(\ell-1)}} h^{(\ell-1)}(x))^\top \left(\frac{\partial f^{(\ell)}}{\partial h^{(\ell-1)}}(x)\right)^\top \left(\frac{\partial f^{(\ell)}}{\partial h^{(\ell-1)}}(x')\right) (\nabla_{\theta^{(\ell-1)}} h^{(\ell-1)}(x'))$$

The first and last parts form the random NTK of the previous layers, $\Theta^{(\ell-1)}_{rand}$. The middle part is a product of Jacobians. As $n_\ell \to \infty$, this Jacobian product also converges in probability to its conditional expectation over the layer-ℓ parameters (A, w, b).

$$\left(\frac{\partial f^{(\ell)}}{\partial h^{(\ell-1)}}\right)^\top \left(\frac{\partial f^{(\ell)}}{\partial h^{(\ell-1)}}\right) \xrightarrow{p} \mathbb{E}_{A,w,b}\left[\dots\right] = \sigma_A^2 \dot{\Sigma}^{(\ell)}_{rand}(x,x')$$

Thus, the entire Term 1 converges to the product of these random quantities :

$$\text{Term 1} \xrightarrow{p} \Theta^{(\ell-1)}_{rand}(x,x') \cdot \sigma_A^2 \dot{\Sigma}^{(\ell)}_{rand}(x,x')$$

Combining terms : In the infinite-width limit, the recurrence relation for the random kernels is :

$$\Theta^{(\ell)}(x,x') = \Theta^{(\ell-1)}(x,x') \cdot \sigma_A^2 \dot{\Sigma}^{(\ell)}(x,x') + \Sigma^{(\ell)}(x,x') + 1$$

This proves the relationship between the random kernels as stated in the main text.

B
SUMMARY OF FORMULAS

— **Layer Output :**

$$h^{(\ell)}(x) = c^{(\ell)} + \frac{\sigma_A}{\sqrt{n_\ell}} \sum_{j=1}^{n_\ell} A_j^{(\ell)} \sigma\left(w_j^{(\ell)\top} h^{(\ell-1)}(x) + \beta b_j^{(\ell)}\right)$$

— **NNGP Kernel Recursion :**

— *Base Kernel :* Expectation over current layer's random parameters, conditioned on the previous layer's output GP $h^{(\ell-1)}$.

$$\Sigma^{(\ell)}(x,x') = \mathbb{E}_{w,b}\left[\sigma\left(w^\top h^{(\ell-1)}(x) + \beta b\right)\sigma\left(w^\top h^{(\ell-1)}(x') + \beta b\right)\right]$$

— *Full Kernel :*

$$\mathcal{K}_{\text{NNGP}}^{(\ell)}(x,x') = \sigma_A^2 \Sigma^{(\ell)}(x,x') + \sigma_c^2$$

— **NTK Kernel Recursion :**

— *Base Derivative Kernel :*

$$\dot{\Sigma}^{(\ell)}(x,x') = \mathbb{E}_{w,b}\left[\dot{\sigma}\left(w^\top h^{(\ell-1)}(x) + \beta b\right)\dot{\sigma}\left(w^\top h^{(\ell-1)}(x') + \beta b\right) w^\top w\right]$$

— *Full Kernel :*

$$\Theta^{(\ell)}(x,x') = \Theta^{(\ell-1)}(x,x') \cdot \sigma_A^2 \dot{\Sigma}^{(\ell)}(x,x') + \Sigma^{(\ell)}(x,x') + 1$$

C
INTRODUCTION

The study of Neural Tangent Kernels (NTK) has provided profound insights into the behavior of deep neural networks, particularly in the infinite-width limit where the training dynamics can be described by a kernel. This analysis is grounded in the fact that, in this limit, the pre-activation outputs of the first layer follow a Gaussian Process. While the NTK for standard architectures like Multi-Layer Perceptrons (MLPs) is well-understood, its properties for more complex models like MMNNs remain less explored.

In this work, I conduct a detailed empirical investigation of the NTK for two-layer MMNNs with ReLU activations. My goal is to characterize the NTK's dependence on two critical hyperparameters : the internal rank of the matrix multiplication and a bias-like parameter, β. I aim to understand how these parameters affect not only the average behavior of the kernel but also its statistical fluctuations.

D
EXPERIMENTAL METHODOLOGY

My experimental setup was designed to systematically explore the parameter space of the MMNNs and their corresponding NTK.

D.1 Model Architecture and Data

— **Model :** I use a two-layer MMNN with a ReLU activation function, $\sigma(x) = \max(0, x)$, where the first layer has a width of 1000. The network produces a scalar, one-dimensional output.
— **Data :** My input vectors are two-dimensional and sampled uniformly from the unit sphere. I chose this low-dimensional setup because the NTK is a zonal kernel, meaning it only depends on the scalar product $\rho = x^T x'$ between inputs. This allows for extensive and expressive experiments without suffering from the curse of dimensionality.

D.2 Hyperparameter Configuration

— **Internal Rank (r) :** The rank of matrix decomposition is a central parameter, varied on a logarithmic scale (from 2 to 100). The weight matrix of the second layer is normalized by $1/\sqrt{r}$.
— **Parameter β :** Unlike standard MLP analyses where β is often zero, we explore non-zero values to study its non-trivial influence on the kernel.

Each experimental configuration is a tuple (dimension, number of points n, β, rank r).

D.3 NTK Computation and Statistical Estimation

I estimate the NTK using a Monte Carlo method. This approach is a practical necessity. The exact analytical computation would require an integration over a high-dimensional Gaussian measure associated with the weights of the second layer (where the dimension is the rank r), which is computationally intractable.
— I use 300 random points for the Monte Carlo estimation. Care was taken to resample the weight matrices for each draw to ensure statistical independence.
— For each configuration, I compute 50 distinct NTK matrices over a fixed dataset. This allows me to perform a statistical analysis of the kernel itself.
— Given the uniform sampling, I also employed bootstrapping techniques to generate a richer dataset for more robust density visualizations and statistical estimations of quantities like the mean and standard deviation.

E
RESULTS AND OBSERVATIONS

E.1 Impact of Internal Rank on NTK Statistics

My most significant observation is that the standard deviation of the NTK entries (both diagonal and off-diagonal) decays as a power law with the internal rank r (Figure 8). This is a powerful finding. It means that as the rank grows, the NTK's random component vanishes, and the kernel becomes increasingly deterministic. Consequently, I can confidently use the mean NTK, $\mathbb{E}[\Theta]$, as a reliable representation of its behavior for my analysis. This is fantastic because it allows me to condition integrals well, even within deep Gaussian processes.

I also noted that for low ranks, the NTK spectrum is enhanced with many positive eigenvalues that are of the same order of magnitude as the leading one (Figure 9).

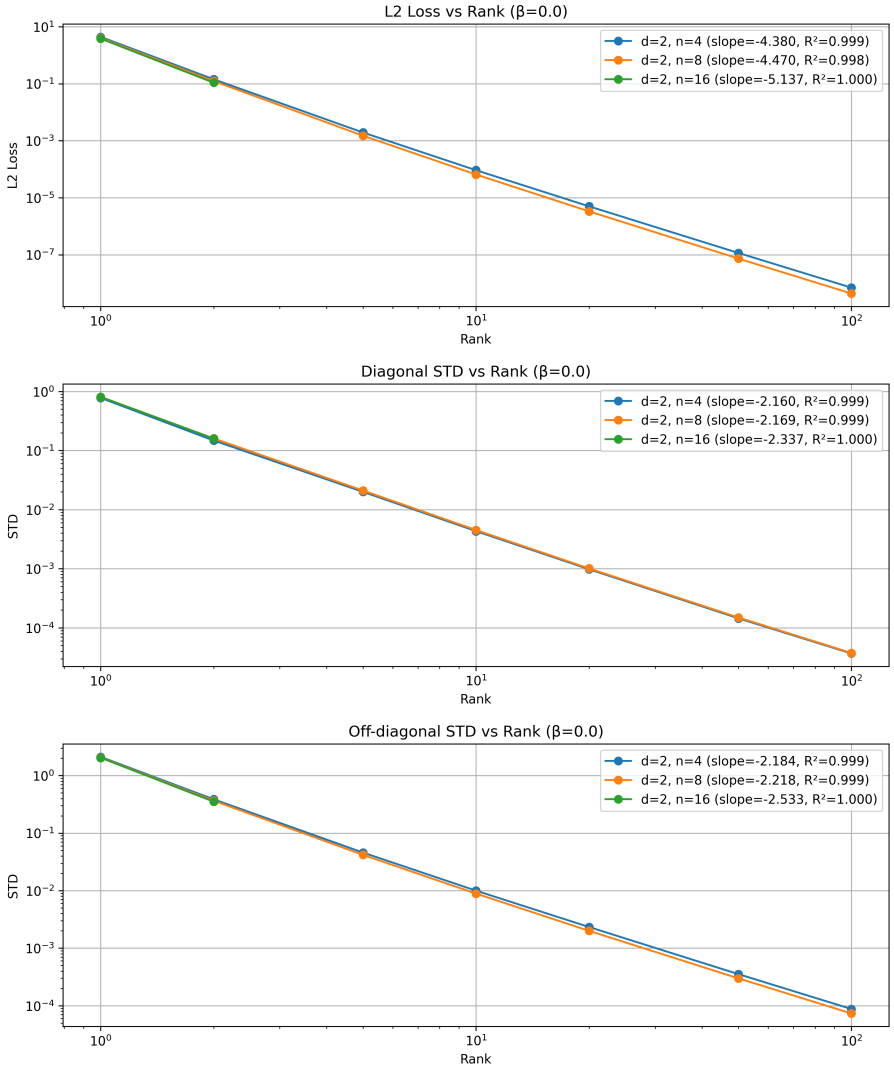


Figure 8 – Decay of the NTK's standard deviation as a function of rank for $\beta = 0$.

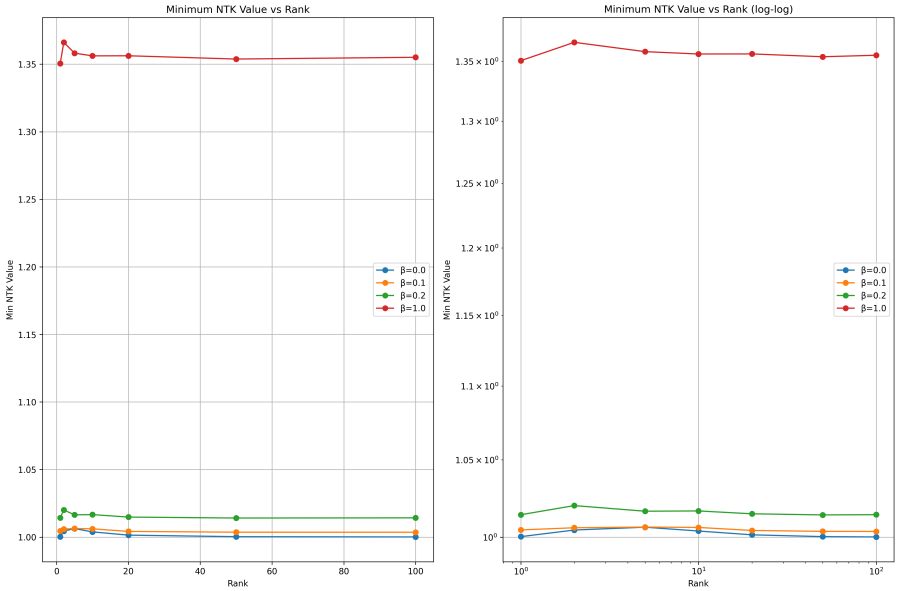


Figure 9 – Behavior of the smallest NTK eigenvalue as a function of rank.

E.2 Dependence on the β Parameter

The β parameter has a very clear impact. For $\beta = 0$, I observed that the mean NTK value converges towards 0 with a particular slope as rank increases. For $\beta > 0$, it converges to a non-zero constant. My plots indicate that the dependence of this limit on β appears to be either linear or quadratic (Figure 10). I am confident this limit can be computed theoretically, although it would require integrating special functions (like 'erf' and Gaussian CDFs), which I also plan to verify numerically in Python.

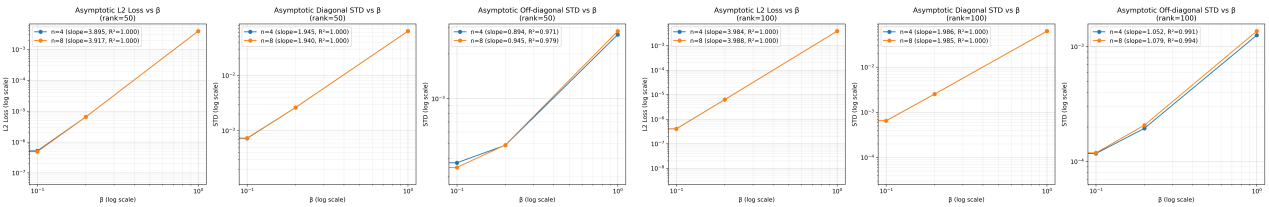


Figure 10 – Convergence of the mean NTK as a function of β for rank 50 (left) and 100 (right).

E.3 Distributional Analysis of the NTK

I analyzed the distribution of NTK values, conditioned on the dot product ρ (Figure 12). Using various kernel density estimators (Epanechnikov, Gaussian, etc.), I made several key observations.

- The distributions exhibit heavy, power-law-like tails, especially at low ranks (Figure 11). I am currently in the process of formally verifying this using Hill estimators.
- The distribution of the on-diagonal elements is particularly interesting ; while it tends towards a Gaussian with rank, it also exhibits a pyramid-like shape, which suggests a potential connection to Random Matrix Theory.

— The off-diagonal elements show a tailed distribution with a mode concentrated near 1 for lower ranks, which then converges to a Gaussian as the rank increases.
— Despite the heavy tails at low ranks, the overall distribution clearly converges towards a Gaussian as the rank grows, as shown in Figure 13, which is consistent with the Central Limit Theorem.

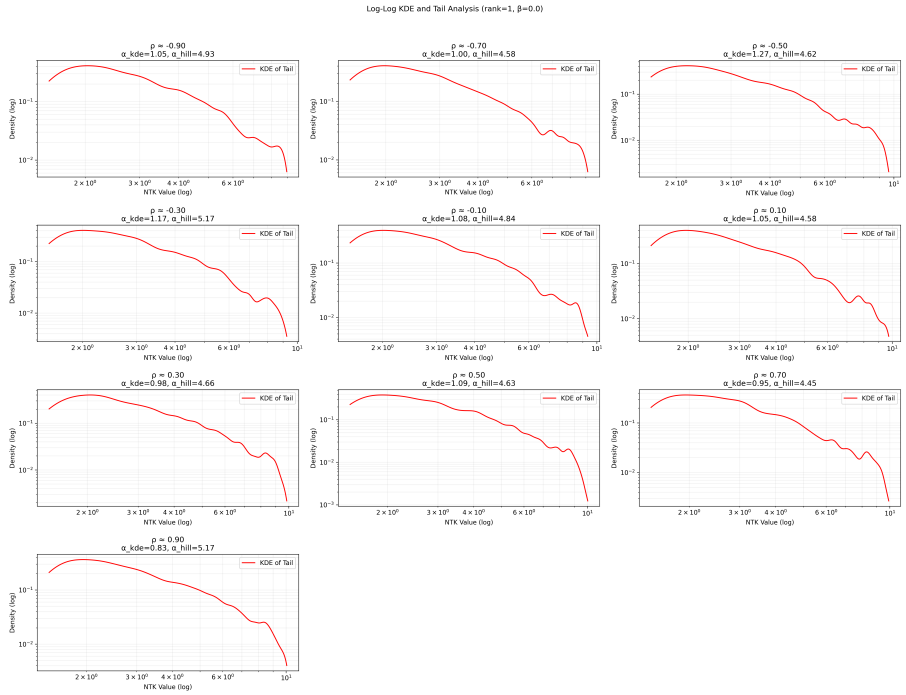


Figure 11 – Log-log scale plot of the NTK's probability density (KDE), suggesting a power-law tail for low rank.

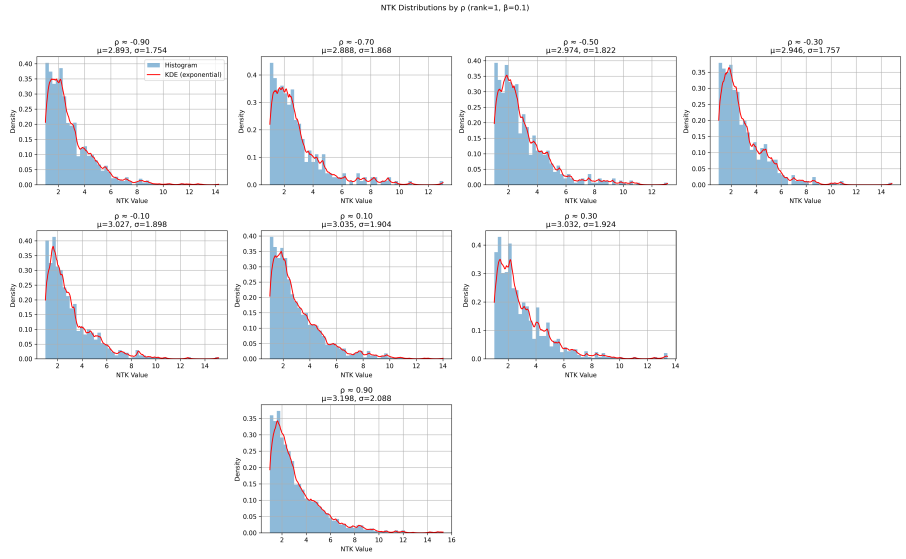


Figure 12 – Distribution of NTK values as a function of ρ.

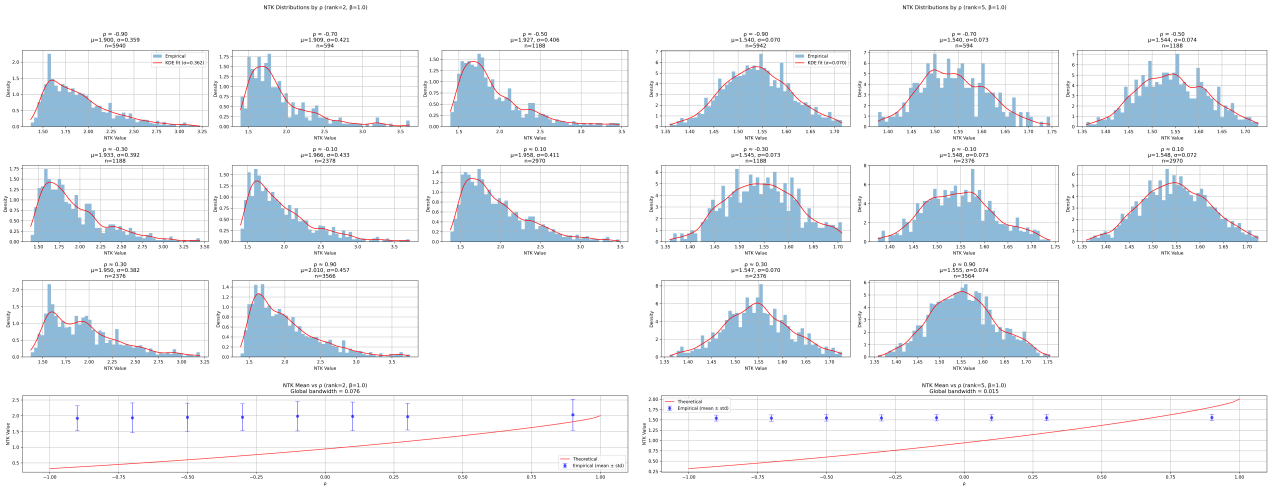


Figure 13 – Convergence of the NTK distribution towards a Gaussian as rank increases (from 2 to 5).

E.4 Mean NTK and Structural Properties

When I plotted the mean NTK as a function of the input dot product ρ, I noticed a very low degree of convexity. Specifically, for high ranks like $r = 100$, the relationship is nearly linear with a high R-squared value.

I also compared this behavior to other architectures. I found that the empirical NTK for a 1-layer MMNN looks remarkably similar to the theoretical NTK of a 2-layer MLP, but scaled by a factor of 1/2 (Figure 14). The "gain" from adding a second layer to the MMNN appears to correspond to the contributions of two specific terms in the NTK's recurrence formula.

Furthermore, I remarked that the NTK matrix exhibits a compound structure. To quantify this, I computed its L2 loss over this specific subspace, confirming a structured deviation from a simpler matrix form.

F
CONCLUSION

In conclusion, my empirical analysis has uncovered several distinct properties of the NTK for MMNNs. The most critical result is the rapid decay of the NTK's variance with increasing internal rank. This concentration phenomenon implies that for sufficiently high ranks, the NTK behaves like a deterministic kernel that can be fully characterized by its mean. This is a powerful conclusion, as it greatly simplifies the theoretical analysis of these models. Furthermore, I have characterized the significant influence of the β parameter and identified unique signatures in the kernel's functional form that distinguish MMNNs from classical MLP architectures.

G
APPENDIX : CONCEPTUAL NOTES FROM TRANSCRIPT

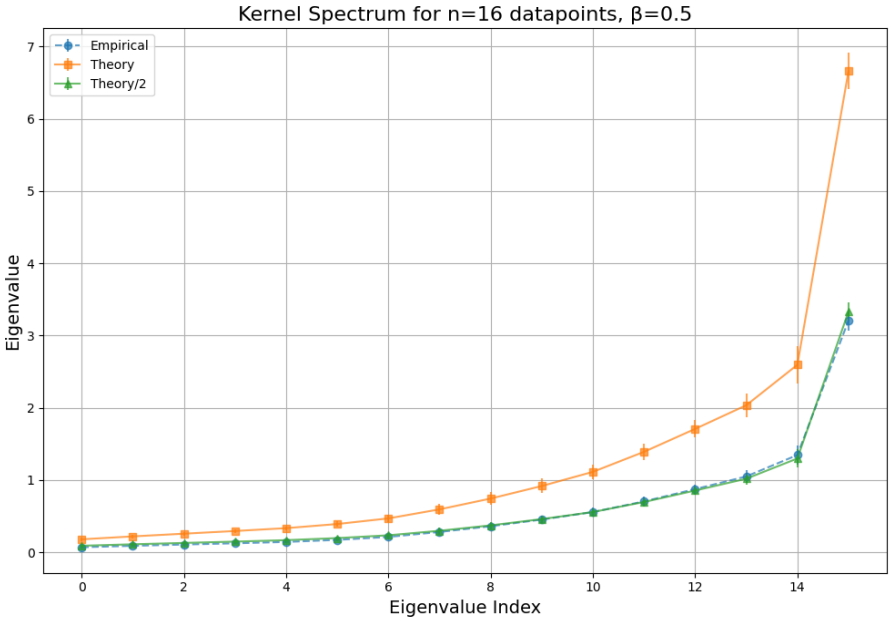


Figure 14 – Comparison of kernel spectra. The empirical spectrum for a 1-layer MMNN is compared against the theoretical spectrum for a 2-layer MLP.

H

PROOF OF THE RECURSIVE FORMULAS

The detailed derivation of the recursive formulas for the NNGP kernel ($\Sigma^{(\ell)}$) and the NTK kernel ($\Theta^{(\ell)}$) is involved and will be provided in a future version of this document.

A

APPENDIX : DERIVATION OF THE VARIANCE RECURRENCE (EOC)

We derive the recurrence relation for the average variance per dimension, $q^{(\ell)} = \mathbb{E}[\|h^{(\ell)}\|^2]/d_\ell$, assuming scalar outputs for simplicity ($d_\ell = 1$). The output of layer ℓ is $h^{(\ell)} = c^{(\ell)} + \frac{\sigma_A}{\sqrt{n_\ell}} \sum_{j=1}^{n_\ell} A_j^{(\ell)} \sigma(g_j^{(\ell)})$, where $g_j^{(\ell)} =$

$w_j^{(\ell)\top} h^{(\ell-1)} + \beta b_j^{(\ell)}$. We assume all parameters are zero-mean and independent. The variance is :

$$\begin{aligned} q^{(\ell)} = \mathbb{E}[(h^{(\ell)})^2] &= \mathbb{E}[(c^{(\ell)})^2] + \frac{\sigma_A^2}{n_\ell}\mathbb{E}\left[\left(\sum_{j=1}^{n_\ell} A_j^{(\ell)}\sigma(g_j^{(\ell)})\right)^2\right] \\ &= \sigma_c^2 + \frac{\sigma_A^2}{n_\ell}\sum_{j=1}^{n_\ell}\mathbb{E}[(A_j^{(\ell)})^2]\mathbb{E}[(\sigma(g_j^{(\ell)}))^2] \quad \text{(by independence)} \\ &= \sigma_c^2 + \sigma_A^2\mathbb{E}[\sigma(g^{(\ell)})^2] \quad \text{(since } \mathbb{E}[(A_j)^2] = 1 \text{ and terms are i.i.d.)} \end{aligned}$$

The pre-activation $g^{(\ell)} = w^\top h^{(\ell-1)} + \beta b$ is a zero-mean Gaussian. Its variance is :

$$\begin{aligned} \mathrm{Var}(g^{(\ell)}) &= \mathbb{E}[(w^\top h^{(\ell-1)})^2] + \mathbb{E}[(\beta b)^2] \\ &= \mathbb{E}[\mathrm{Tr}(w^\top h^{(\ell-1)}(h^{(\ell-1)})^\top w)] = \mathbb{E}[\mathrm{Tr}(ww^\top h^{(\ell-1)}(h^{(\ell-1)})^\top)] \\ &= \mathrm{Tr}(\mathbb{E}[ww^\top]\mathbb{E}[h^{(\ell-1)}(h^{(\ell-1)})^\top]) + \beta^2 \\ &= \mathrm{Tr}(\Sigma_w \cdot (q^{(\ell-1)}\mathbf{I}_{d_{\ell-1}})) + \beta^2 = q^{(\ell-1)}\mathrm{Tr}(\Sigma_w) + \beta^2 \end{aligned}$$

For a zero-mean Gaussian variable g and ReLU activation σ, we have $\mathbb{E}[\sigma(g)^2] = \frac{1}{2}\mathrm{Var}(g)$. Substituting back, we get the recurrence :

$$q^{(\ell)} = \sigma_c^2 + \frac{\sigma_A^2}{2}\left(q^{(\ell-1)}\mathrm{Tr}(\Sigma_w) + \beta^2\right)$$

B
MMNN NTK FOR 2 HIDDEN LAYERS

B.1 RECURSIVE

For a 2 hidden layer MMNN, we can write the recursive formulation of the NTK kernel. Let's recall the general recursive formula for layer ℓ :

suppose that you have a low rank hidden layer, with a width r

so the architecture is :

$$(x,y) \xrightarrow{\sigma W_1} h_1 \xrightarrow{A_1 \cdot + c_1} r \xrightarrow{\sigma W_2} h_2 \xrightarrow{A_2 \cdot + c_2} (x_2, y_2)$$

$$N \qquad (x_1, y_1) \text{ in } \mathbb{R}^r \qquad N$$

And here N grow to infinity, and r is fixed.

Then the NTK random kernel is :

$$\Theta^{(\ell)}(x,x') = \Theta^{(\ell-1)}(x,x') \cdot \left(\sigma_A^2\dot{\Sigma}^{(\ell)}(x,x')\right) + \Sigma^{(\ell)}(x,x') + 1 \tag{130}$$

For $\ell = 2$, starting from $\Theta^{(0)} = 0$, we have :

$$\Theta^{(1)} = \mathbb{E}_{w,b}\left[\sigma(w^\top x + \beta b)\sigma(w^\top x' + \beta b)\right] + 1$$
$$\Theta^{(2)} = \Theta^{(1)} \cdot \left(\sigma_A^2 \mathbb{E}_{w,b}\left[\dot\sigma(w^\top h^{(1)}(x) + \beta b)\dot\sigma(w^\top h^{(1)}(x') + \beta b) w^\top w\right]\right)$$
$$+ \mathbb{E}_{w,b}\left[\sigma(w^\top h^{(1)}(x) + \beta b)\sigma(w^\top h^{(1)}(x') + \beta b)\right] + 1$$

Here, $\Theta^{(1)}$ is a deterministic NNGP kernel (Gaussian Process) since it depends only on the input x, while $\Theta^{(2)}$ is a random field (Deep Gaussian Process) that depends on the realization of the first layer's output $h^{(1)}$. When $\beta = 0$, these expectations simplify by removing the bias terms b. This non-Gaussian nature of the kernels for $\ell > 1$ is a key distinguishing feature of MMNNs compared to standard neural networks.

When $\beta = 0$, the equations simplify to :

Main focus : NTK kernels with β

$$\Theta^{(1)} = \mathbb{E}_w\left[\sigma(w^\top x)\sigma(w^\top x')\right] + 1$$
$$\Theta^{(2)} = \Theta^{(1)} \cdot \left(\sigma_A^2 \mathbb{E}_w\left[\dot\sigma(w^\top h^{(1)}(x))\dot\sigma(w^\top h^{(1)}(x')) w^\top w\right]\right)$$
$$+ \mathbb{E}_w\left[\sigma(w^\top h^{(1)}(x))\sigma(w^\top h^{(1)}(x'))\right] + 1$$

Note that for stable signal propagation at the Edge of Chaos (EOC), we require for 1 rank hidden layer :

$$\sigma_A^2 \cdot \mathrm{Tr}(\Sigma_w) = 2$$

Then for a rank r

$$\sigma_A^2 \cdot \mathrm{Tr}(\Sigma_w) = \frac{2}{r}$$

And we assume at least isotropic gaussian for the weights without loss of generality and to ensure fast computations.

This condition ensures the signal neither vanishes nor explodes as it propagates through the network layers. This also means we need to normalize by $\sqrt{r}$ the output of the first MMNN layer. We call x_1 and y_1 the outputs of the first layer unnormalized.

We recall also that $\Sigma^{(1)}$ is the arccosine kernel, which has a closed form in function of the correlation ρ between x and x' :

$$\Sigma^{(1)}(\rho) = \frac{1}{\pi}\left(\sqrt{1-\rho^2} + \rho(1 - \arccos(\rho))\right)$$

We recall also that $\dot\Theta^{(1)}(\rho) = \mathbb{E}_w\left[w^\top w \dot\sigma(w^\top x)\dot\sigma(w^\top x')\right]$. We can compute this exactly when w is isotropic gaussian with variance 1 :

In fact a simple computation takes the expectation of the product of the two derivatives of the ReLU, by independence of $\|w\|^2$ and $\mathbb{1}_{w^\top x>0} \cdot \mathbb{1}_{w^\top x'>0}$:

$$\begin{aligned}
\dot\Theta^{(1)}(\rho) &= \mathbb{E}_w\left[w^\top w \dot\sigma(w^\top x)\dot\sigma(w^\top x')\right] \\
&= \mathbb{E}_w\left[w^\top w \cdot \mathbb{1}_{w^\top x>0} \cdot \mathbb{1}_{w^\top x'>0}\right] \\
&= \mathbb{E}\left[\|w\|^2\right] \cdot \mathbb{P}(w^\top x > 0, w^\top x' > 0) \\
&= \mathrm{Tr}(\Sigma_w)\left(\frac{1}{2} - \frac{\arccos(\rho)}{2\pi}\right)
\end{aligned}$$

It is just a reformulation of the fact that we integrate over a cone of angle $\arccos(\rho)$ a radial function.

We then write $\Theta^{(2)}$ as a function of ρ and ρ_1 :

$$\Theta^{(2)}(\rho,\rho_1) = \Theta^{(1)}(\rho)\cdot\sigma_A^2\,\mathrm{Tr}(\Sigma_w)\left(\frac{1}{2}-\frac{\arccos(\rho)}{2\pi}\right)+\frac{1}{r}\cdot\Sigma^{(1)}(\rho_1)\|x_1\|\|y_1\|+1$$

$$\text{with}\quad \Sigma^{(1)}(\rho)=\frac{1}{\pi}\left(\sqrt{1-\rho^2}+\rho(1-\arccos(\rho))\right)$$

where ρ_1 is the correlation between the outputs of the first layer, that is a random variable and ρ is the correlation between the inputs.

Note that this multiplicative value is between 0 and 1, that is we can compute a lot of stuff after concatenating layers and getting some exponential scaling laws. We emphasize the fact we multiply by $\|x_1\|\|y_1\|$ which is a random variable, but as we will see, it is independent of the correlation ρ_1.

B.2 Expression for random variables ρ_1, $\|x_1\|^2$, $\|y_1\|^2$

The output of the 1st layer is a gaussian process of dimension r, where each coordinate is a gaussian process with covariance matrix :

$$\Sigma_i^{(1)} = \mathbb{E}_w\left[\sigma(w^\top x)\sigma(w^\top x')\right]$$

and the coordinates are ρ-correlated. If we stack horizontally the outputs x_1 and y_1 in a vector of dimension $2r$, the full $2r\times 2r$ covariance matrix is therefore :

$$\Sigma^{(1)} = \begin{pmatrix} I & \rho I \\ \rho I & I\end{pmatrix}$$

It's a kronecker product of r matrices I and 2d ρ correlated gaussian variables.

it happens, by a magic, that we can compute exactly the joint distribution of the random variable $(\rho_1,\|x_1\|^2,\|y_1\|^2)$:

$$\rho_1 = \frac{\sum_{i=1}^r x_i y_i}{\sqrt{\sum_{i=1}^r x_i^2}\sqrt{\sum_{i=1}^r y_i^2}}$$

The squared norms x_1^2 and y_1^2 follow a bivariate Kibble distribution and their cosine distance follows an independent Fisher like distribution. The Kibble distribution is defined for $u,v\geq 0$ and $\rho\in[-1,1]$, while the Fisher distribution is defined for $\theta\in[-1,1]$ and $\rho\in[-1,1]$ with $r>2$.

$$\begin{aligned}(\|x_1\|^2,\|y_1\|^2) &\sim \mathrm{Kibble}(r,\rho)\\ &= \frac{(uv)^{\frac{r/2-1}{2}}e^{-\frac{u+v}{2(1-\rho^2)}}}{\Gamma(r/2)(2(1-\rho^2))^{r/2+1}\rho^{\frac{r/2-1}{2}}}\cdot I_{r/2-1}\left(\frac{\rho\sqrt{uv}}{1-\rho^2}\right)\end{aligned}$$

$$\begin{aligned}\rho_1 &\sim \mathrm{Fisher}(\rho,r)\\ &= \frac{(r-2)\Gamma(r-1)(1-\rho^2)^{\frac{r-1}{2}}(1-\theta^2)^{\frac{r-4}{2}}}{\sqrt{2\pi}\Gamma(r-\frac{1}{2})(1-\rho\theta)^{r-\frac{3}{2}}}\times {}_2F_1\left(\frac{1}{2},\frac{1}{2};r-\frac{1}{2};\frac{1+\rho\theta}{2}\right)\end{aligned}$$

where $u=\|x_1\|^2$, $v=\|y_1\|^2$, $\theta=\cos(x_1,y_1)$, and I_α is the modified Bessel function of the first kind.

and the correlation of x,y :

where x_{1i} and y_{1i} are ρ-correlated gaussian variables.

We ensure that the reader has understood that the correlation of x,y is independent of the squared norms of x,y !

B.2.1 • Final result

We can then state the final result : For 2 entries x,y in $\mathbb{R}^d$ and a low rank r hidden layer, the NTK for a 2 hidden layer MMNN is :

$$\Theta^{(2)}(x,y) = \Theta^{(1)}(\rho) \cdot \left(1 - \frac{\arccos(\rho_1)}{\pi}\right) \frac{1}{r} + \frac{1}{r}\Sigma^{(1)}(\rho_1)\|x_1\|\|y_1\| + 1$$

$$\text{with} \quad \Sigma^{(1)}(\rho) = \frac{1}{\pi}\left(\sqrt{1-\rho^2} + \rho(1 - \arccos(\rho))\right)$$

where the random variables follow :

- $\rho_1 \sim \text{Fisher}(\rho, r)$ is independent of the norms
- $(\|x_1\|^2, \|y_1\|^2) \sim \text{Kibble}(r, \rho)$ are chi-squared variables with r degrees of freedom from correlated gaussian variables.

B.3 Intuitions sur les Performances

(Discussion finale basée sur les résultats théoriques et expérimentaux pour expliquer les performances observées pour les MMNN, les transformeurs et les ResNets.)

C
RÉSULTATS EXPÉRIMENTAUX ET APPLICATIONS

C.1 Validation des Lois de Scaling et des Corrections

(Présentation des expériences numériques validant les lois de scaling pour les corrections à largeur finie dans les FCNN.)

C.2 Analyse Hessienne et Dynamique

(Présentation des expériences sur l'approche hessienne du NTK et l'étude des termes de type Lyapunov.)

C.3 Applications en SciML et Discussion

(Discussion sur l'application des résultats en Scientific Machine Learning. Vérification des hypothèses théoriques sur les grands modèles et le scaling des paramètres. Lien avec la physique statistique et la convergence vers

l'équilibre thermique.)

ANNEXES

Figure 15 – Aperçu de l'interface mise à disposition en source ouverte pour l'extraction de connaissances

A
FISHER, KIBBLE DISTRIBUTIONS AND THE FAKE "CURSE OF DIMENSIONALITY"

A.1 Distribution of the Correlation Coefficient r

Fisher derived the exact law of r in 1915, representing the cosine between Gaussian vectors of dimension r with correlation ρ :

$$p(r|\rho,r) = \frac{(r-2)\Gamma(r-1)(1-\rho^2)^{\frac{r-1}{2}}(1-r^2)^{\frac{r-4}{2}}}{\sqrt{2\pi}\Gamma(r-\frac{1}{2})(1-\rho r)^{r-\frac{3}{2}}} \times {}_2F_1\left(\frac{1}{2},\frac{1}{2};r-\frac{1}{2};\frac{1+\rho r}{2}\right)$$

Where Γ is the gamma function and ${}_2F_1$ is the hypergeometric function.

For large r, Fisher's z-transform gives $z = \operatorname{arctanh}(r) \sim \mathcal{N}\left(\operatorname{arctanh}(\rho) + \frac{\rho}{2(r-1)}, \frac{1}{r-3}\right)$. The variance of r decreases as $O(1/r)$. When $\rho \sim O(1/\sqrt{r})$, geometric noise masks correlation.

A.2 The Kibble Distribution

The joint law of correlated chi-squared variables $U = \|X\|^2$ and $V = \|Y\|^2$ follows Kibble's 1941 distribution :

$$f(u,v) = \frac{(uv)^{\frac{r/2-1}{2}} e^{-\frac{u+v}{2(1-\rho^2)}}}{\Gamma(r/2)(2(1-\rho^2))^{r/2+1}\rho^{\frac{r/2-1}{2}}} \cdot I_{r/2-1}\left(\frac{\rho\sqrt{uv}}{1-\rho^2}\right)$$

Where Γ is the gamma function and I_α is the modified Bessel function of the first kind.

Mean $\mathbb{E}[U] = r$ and variance $\operatorname{Var}(U) = 2r$ grow linearly. U/r converges to $\mathcal{N}(1, 2/r)$ with polynomial concentration.

B
PRICE'S THEOREM AND DERIVATIONS

Price's theorem was a central tool in our analysis.

- Statement of price's theorem

Suppose that X_1 and X_2 forms a gaussian vector with joint distribution $f_{X_1,X_2}(x_1,x_2)$. Then, for any function $g(x_1,x_2)$, we have :

$$\mathbb{E}[g(X_1,X_2)] = \int_0^\rho \mathbb{E}\left[\frac{\partial^2 g}{\partial x_1 \partial x_2}(X_1,X_2)\right] d\rho + \text{Constant}$$

where the constant is the value of the expectation when $\rho = 0$.

• Calculation of I_{12} and I_{21} (First-Order Moments)

For I_{12}, we need to calculate :

$$I_{12} = \int_a^\infty \int_b^\infty z_2 \phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$$

Step 1 : Inner integral We first focus on the inner integral with respect to z_2. The exact formula for the conditional density is :

$$\phi_2(x, y; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(y) \phi_1\left(\frac{x - \rho y}{\sqrt{1-\rho^2}}\right)$$

Applying this formula with $x = z_1$ and $y = z_2$:

$$\phi_2(z_1, z_2; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(z_2) \phi_1\left(\frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}\right)$$

The inner integral becomes :

$$\int_b^\infty z_2 \frac{1}{\sqrt{1-\rho^2}} \phi_1(z_2) \phi_1\left(\frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}\right) dz_2$$

Step 2 : Change of variable Let $u = \frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}$. Then :

$$z_2 = \frac{z_1 - u\sqrt{1-\rho^2}}{\rho}$$
$$dz_2 = -\frac{\sqrt{1-\rho^2}}{\rho}\, du$$

The lower bound becomes $u_{max} = \frac{z_1 - \rho b}{\sqrt{1-\rho^2}}$. The integral transforms into :

$$\frac{1}{\sqrt{1-\rho^2}} \int_{-\infty}^{\frac{z_1-\rho b}{\sqrt{1-\rho^2}}} \left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) \phi_1\left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) \phi_1(u) \left(-\frac{\sqrt{1-\rho^2}}{\rho}\right) du$$

Step 3 : Simplification Grouping terms and using properties of ϕ_1 :

$$-\frac{1}{\rho^2} \int_{-\infty}^{\frac{z_1-\rho b}{\sqrt{1-\rho^2}}} (z_1 - u\sqrt{1-\rho^2}) \phi_1(u) \phi_1\left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) du$$

Step 4 : Using the conditional density formula The exact formula is :

$$\phi_2(x, y; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(x) \phi_1\left(\frac{y - \rho x}{\sqrt{1-\rho^2}}\right)$$

Applying it correctly to our case with $x = z_1$ and $y = z_2$:

$$I_{12} = \phi_1(b) \Phi_1\left(\frac{a + \rho b}{\sqrt{1-\rho^2}}\right) + \rho \phi_1(a) \Phi_1\left(-\frac{b + \rho a}{\sqrt{1-\rho^2}}\right)$$

This final formula for I_{12} is one of the key terms in calculating the expectation of the product of ReLUs.

- Calculation of I_{22} (Cross-Moment)

The calculation of I_{22} is the most complex part. The final result is :

$$I_{22} = \int_0^\rho I_{11} + \phi_2(a, b; \rho)$$

The proof of this identity was a central point of the discussion. The most rigorous method consists of showing the equality for $\rho = 0$, and then the equality of the derivatives with respect to ρ for both sides of the equation.

- Final Formula

By assembling all the terms, we obtain the final algebraic formula :

$$\mathbb{E}[\dots] = (\mu_1\mu_2 + \rho\sigma_1\sigma_2)I_{11} + \mu_1\sigma_2 I_{12} + \mu_2\sigma_1 I_{21} + \sigma_1\sigma_2\phi_2(a, b; \rho)$$

B.1 Computing ReLU Product Expectations

When analyzing the Neural Tangent Kernel (NTK) of a single-layer network, we need to compute expectations of the form :

$$\mathbb{E}[\text{ReLU}(X_1)\text{ReLU}(X_2)]$$

where (X_1, X_2) follows a bivariate normal distribution with parameters $(\mu_1, \sigma_1, \mu_2, \sigma_2, \rho)$.

For deterministic input vectors x_1 and x_2, we define the following quantities :

- $\sigma_1 = \|x_1\|$ (where $\|\cdot\|$ denotes Euclidean norm)
- $\sigma_2 = \|x_2\|$
- $\rho = \cos(x_1, x_2) = \frac{\langle x_1, x_2 \rangle}{\|x_1\|\|x_2\|}$

The network outputs follow a Gaussian process characterized by the covariance matrix :

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$$

$\sigma_1^2\rho\sigma_1\sigma_2$
$\rho\sigma_1\sigma_2\sigma_2^2$

B.2 Decomposition

We recall the expression of mu The approach is to standardize the variables $(Z_i = (X_i - \mu_i)/\sigma_i)$ and decompose the integral into four terms :

$$\mathbb{E}[\dots] = \sigma_1\sigma_2 I_{22} + \mu_1\sigma_2 I_{12} + \mu_2\sigma_1 I_{21} + \mu_1\mu_2 I_{11}$$

where the integrals I_{ij} are defined over the domain $z_1 > a$ and $z_2 > b$, with $a = -\mu_1/\sigma_1$ and $b = -\mu_2/\sigma_2$.

- $I_{11} = \int_a^\infty \int_b^\infty \phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{21} = \int_a^\infty \int_b^\infty z_1\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{12} = \int_a^\infty \int_b^\infty z_2\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{22} = \int_a^\infty \int_b^\infty z_1 z_2\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$

B.3 Calculation of the Components

- **Calculation of I_{11} (Probability)**

I_{11} is the probability $P(Z_1 > a, Z_2 > b)$. It is calculated via the cumulative distribution function (CDF) of the standard bivariate normal distribution, Φ_2.

$$I_{11} = P(Z_1 > a, Z_2 > b) = P(Z_1 < -a, Z_2 < -b) = \Phi_2(-a, -b; \rho) = \Phi_2\left(\frac{\mu_1}{\sigma_1}, \frac{\mu_2}{\sigma_2}; \rho\right)$$

C
UNSUCCESSFUL ATTEMPTS TO CALCULATE I_{22}

Several approaches to calculate I_{22} proved to be dead ends.

C.1 Via the Identity on the Derivative of ϕ_2

Let's first prove the identity for the partial derivatives of the 2D normal PDF ϕ_2. For a bivariate normal distribution with correlation ρ :

$$\frac{\partial \phi_2}{\partial z_1} = -\frac{z_1 - \rho z_2}{1-\rho^2}\phi_2(z_1, z_2; \rho)$$
$$\frac{\partial \phi_2}{\partial z_2} = -\frac{z_2 - \rho z_1}{1-\rho^2}\phi_2(z_1, z_2; \rho)$$

Using these identities, we can write :

$$z_1\phi_2 = -\rho z_2\phi_2 - (1-\rho^2)\frac{\partial \phi_2}{\partial z_1}$$

Integrating from a to ∞ with respect to z_1 :

$$\int_a^\infty z_1\phi_2\, dz_1 = -\rho z_2 \int_a^\infty \phi_2\, dz_1 + (1-\rho^2)\phi_2(a, z_2; \rho)$$

Attempting to use this for I_{22}, we integrate $z_1 z_2 \phi_2$ over the region $[a, \infty) \times [b, \infty)$:

$$I_{22} = \int_b^\infty z_2 \left(\int_a^\infty z_1\phi_2\, dz_1 \right) dz_2$$

Substituting the identity :

$$I_{22} = -\rho \int_b^\infty z_2^2 \left(\int_a^\infty \phi_2\, dz_1 \right) dz_2 + (1-\rho^2) \int_b^\infty z_2\phi_2(a, z_2; \rho)\, dz_2$$

This approach becomes intractable because : 1. The first term involves a higher-order moment z_2^2 2. The second term requires evaluating another complex integral 3. Each attempt to simplify leads to terms of equal or greater complexity

Therefore, while the partial derivative identities are mathematically elegant, they do not provide a practical path to computing I_{22} when beta is non 0.

D
SPECIAL CASE : ZERO MEANS ($\mu_1 = \mu_2 = 0$)

In this case, the "from scratch" derivation leads to an elegant formula.

$$\mathbb{E}[\_/(X_1)\ \_/(X_2)] = \sigma_1\sigma_2 \int_0^\infty \int_0^\infty z_1 z_2 \phi_2(z_1, z_2; \rho)\, dz_1\, dz_2$$

The problem reduces to calculating $I_{22}(0,0;\rho) = \mathbb{E}[\_/(Z_1)\ \_/(Z_2)]$. The cleanest derivation uses Price's theorem :

1. $\frac{\partial \mathbb{E}[g]}{\partial \rho} = \mathbb{E}\left[\frac{\partial^2 g}{\partial z_1 \partial z_2}\right] = \mathbb{E}[\mathbb{1}(z_1 > 0, z_2 > 0)] = P(Z_1 > 0, Z_2 > 0) = \frac{1}{2\pi}\arccos(-\rho)$.
2. The integration constant is $\mathbb{E}[g]_{\rho=0} = \mathbb{E}[\_/(Z_1)]\mathbb{E}[\_/(Z_2)] = \left(\frac{1}{\sqrt{2\pi}}\right)^2 = \frac{1}{2\pi}$.
3. By integrating $\frac{\partial \mathbb{E}[g]}{\partial \rho}$ from 0 to ρ :

$$\begin{aligned}\mathbb{E}[g](\rho) - \mathbb{E}[g](0) &= \int_0^\rho \frac{1}{2\pi}\arccos(-t)\, dt \\ &= \frac{1}{2\pi}\left[t\arccos(-t) + \sqrt{1-t^2}\right]_0^\rho \\ &= \frac{1}{2\pi}(\rho\arccos(-\rho) + \sqrt{1-\rho^2} - 1)\end{aligned}$$

4. By adding the constant $\mathbb{E}[g](0) = 1/(2\pi)$, we get :

$$\mathbb{E}[\_/(Z_1)\ \_/(Z_2)] = \frac{1}{2\pi}(\sqrt{1-\rho^2} + \rho\arccos(-\rho))$$

The final formula for variances σ_1^2, σ_2^2 is :

$$\mathbb{E}[\_/(X_1)\ \_/(X_2)] = \frac{\sigma_1\sigma_2}{2\pi}(\sqrt{1-\rho^2} + \rho\arccos(-\rho))$$

D.1 Explicit Formulas for Σ and $\dot{\Sigma}$

For layer ℓ, the general formulas are :

$$\begin{aligned}\Sigma^{(\ell)}(x,x') &= \mathbb{E}_{w,b}\left[\sigma(w^\top h^{(\ell-1)}(x) + \beta b)\sigma(w^\top h^{(\ell-1)}(x') + \beta b)\right] \\ \dot{\Sigma}^{(\ell)}(x,x') &= \mathbb{E}_{w,b}\left[\dot{\sigma}(w^\top h^{(\ell-1)}(x) + \beta b)\dot{\sigma}(w^\top h^{(\ell-1)}(x') + \beta b)w^\top w\right]\end{aligned}$$

When $\beta = 0$, these simplify to :

$$\begin{aligned}\Sigma^{(\ell)}(x,x') &= \mathbb{E}_{w}\left[\sigma(w^\top h^{(\ell-1)}(x))\sigma(w^\top h^{(\ell-1)}(x'))\right] \\ \dot{\Sigma}^{(\ell)}(x,x') &= \mathbb{E}_{w}\left[\dot{\sigma}(w^\top h^{(\ell-1)}(x))\dot{\sigma}(w^\top h^{(\ell-1)}(x'))w^\top w\right]\end{aligned}$$

Note that for $\ell > 1$, both kernels become random fields due to the randomness in $h^{(\ell-1)}$.

D.2 Differential and Integral Form

The theorem relates the derivative of the expectation of a function g with respect to the covariance σ_{12} to the expectation of the second derivative of g. For $g = \_/(X_1)\ \_/(X_2)$, we have $\frac{\partial^2 g}{\partial x_1 \partial x_2} = \mathbb{1}(X_1 > 0, X_2 > 0)$.

$$\frac{\partial \mathbb{E}[\_/(X_1)\ \_/(X_2)]}{\partial \sigma_{12}} = \mathbb{E}[\mathbb{1}(X_1 > 0, X_2 > 0)] = I_{11}$$

This leads to the integral formulation :

$$\mathbb{E}[\_/(X_1)\ \_/(X_2)] = \int I_{11}(\sigma_{12})\, d\sigma_{12} + \text{Constant}$$

D.3 Calculating the Constant (Independent Case $\rho = 0$)

The constant is the value of the expectation when $\rho = 0$, i.e., when X_1 and X_2 are independent.

$$C = \mathbb{E}[\_/(X_1)] \cdot \mathbb{E}[\_/(X_2)]$$

The expectation of a single ReLU is :

$$\mathbb{E}[\_/(X)] = \sigma \phi_1\left(\frac{\mu}{\sigma}\right) + \mu \Phi_1\left(\frac{\mu}{\sigma}\right)$$

The constant is therefore the product of two such terms :

$$C = \left[\sigma_1 \phi_1\left(\frac{\mu_1}{\sigma_1}\right) + \mu_1 \Phi_1\left(\frac{\mu_1}{\sigma_1}\right)\right] \times \left[\sigma_2 \phi_1\left(\frac{\mu_2}{\sigma_2}\right) + \mu_2 \Phi_1\left(\frac{\mu_2}{\sigma_2}\right)\right]$$

This is the implementation of the constant in code, this is fairly long for non zeros beta, that's why we focus on the hypothesis with beta = 0. This in order to make experiments and proofs that MMNNs are superior to FCNNs for 2 hidden layer networks.

D.4 Exact moments and correlation for Kibble variables

Let $(X_i, Y_i)_{i=1}^r$ be i.i.d. Gaussian pairs with zero mean, unit variances and correlation ρ, and define

$$U = \sum_{i=1}^{r} X_i^2, \qquad V = \sum_{i=1}^{r} Y_i^2.$$

Then (U, V) follows Kibble's bivariate chi-square law with r degrees of freedom and correlation ρ. By Isserlis' (Wick's) theorem (or by expanding $\mathbb{E}[(X_i - \rho Y_i)^n (Y_i)^m]$ recursively) :

$$\mathbb{E}[X_i^2] = 1, \quad \mathrm{Var}(X_i^2) = 2, \quad \mathbb{E}[X_i^2 Y_i^2] = 1 + 2\rho^2, \quad (X_i^2, Y_i^2) = 2\rho^2.$$

Using independence across i :

$$\mathbb{E}[U] = r, \quad \mathrm{Var}(U) = 2r, \qquad \mathbb{E}[V] = r, \quad \mathrm{Var}(V) = 2r,$$

$$(U, V) = \sum_{i=1}^{r} (X_i^2, Y_i^2) = 2r\,\rho^2,$$

$$(U, V) = \frac{(U, V)}{\sqrt{\mathrm{Var}(U)\,\mathrm{Var}(V)}} = \frac{2r\rho^2}{\sqrt{(2r)(2r)}} = \rho^2,$$

$$\mathbb{E}[UV] = \mathbb{E}[U]\mathbb{E}[V] + (U, V) = r^2 + 2r\,\rho^2.$$

D.5 Product of Square Roots of Kibble Variables

Let $S = \sqrt{U}\sqrt{V} = \|X\|\|Y\|$. Due to correlation between U and V, we cannot directly multiply expectations. Instead, using Cauchy-Schwarz inequality :

$$\mathbb{E}[S^2] = \mathbb{E}[UV] = \mathbb{E}[U]\mathbb{E}[V] + (U, V) = r^2 + 2r\rho^2$$

Therefore :

$$\mathbb{E}[S] \leq \sqrt{\mathbb{E}[S^2]} = r\sqrt{1 + \frac{2\rho^2}{r}}$$

The variance can be estimated similarly by expanding $(S - \mathbb{E}[S])^2$ and using correlation terms :

$$\mathrm{Var}(S) \approx \frac{r}{2}(1 + \rho^2) + O(1)$$

This suggests S/r approaches a distribution with mean $O(\sqrt{1 + \frac{2\rho^2}{r}})$ and variance of order $O(1/r)$.

D.6 Independence Properties and Asymptotic Behavior

The orientation variable r is independent of magnitudes (U, V). For the normalized product S/r, we have :

$$\frac{S}{r} = \frac{\|X\|\|Y\|}{r} \xrightarrow{r\to\infty} 1$$

This convergence occurs with fluctuations of order $O(1/\sqrt{r})$, showing that in high dimensions, the normalized product concentrates around 1 regardless of the correlation ρ.

E

EXPÉRIENCES NUMÉRIQUES

(paragraphe)

APPENDIX A : RAW SCALING DATA FOR N

Slopes for N scaling:

Configuration	Slope	R^2	Points
D_IN=20, L=2, M=10	0.852	0.993	13
D_IN=20, L=12, M=10	0.943	0.999	11
D_IN=50, L=4, M=10	0.898	0.997	10
D_IN=20, L=8, M=20	0.927	0.998	10
D_IN=20, L=2, M=30	0.798	0.995	9
D_IN=50, L=10, M=10	0.928	0.998	9
D_IN=100, L=8, M=10	0.897	0.999	8
D_IN=20, L=26, M=10	0.886	0.999	8
D_IN=50, L=18, M=10	0.901	0.999	8
D_IN=50, L=12, M=20	0.902	1.000	8
D_IN=50, L=6, M=20	0.882	0.999	8
D_IN=20, L=22, M=20	1.032	0.951	8
D_IN=50, L=4, M=30	0.857	0.999	7
D_IN=100, L=18, M=10	0.864	0.999	7
D_IN=200, L=10, M=10	0.889	1.000	7
D_IN=20, L=6, M=40	0.869	0.998	7
D_IN=20, L=18, M=30	0.853	0.976	7
D_IN=50, L=16, M=20	0.889	0.998	7
D_IN=50, L=20, M=20	0.889	0.999	7
D_IN=20, L=42, M=10	0.883	0.999	7
D_IN=100, L=12, M=20	0.906	1.000	7
D_IN=20, L=16, M=30	0.872	0.990	7
D_IN=200, L=4, M=10	0.869	0.999	7
D_IN=100, L=32, M=10	0.878	0.999	6
D_IN=20, L=46, M=10	0.871	0.999	6
D_IN=20, L=48, M=10	0.861	1.000	6
D_IN=20, L=38, M=20	0.912	0.963	6
D_IN=200, L=2, M=30	0.759	0.998	6
D_IN=20, L=2, M=60	0.777	0.996	6
D_IN=100, L=6, M=30	0.877	1.000	6
D_IN=1000, L=2, M=10	0.762	0.998	6
D_IN=500, L=8, M=10	0.881	0.999	6
D_IN=20, L=8, M=50	0.884	0.998	6
D_IN=200, L=14, M=10	0.875	0.999	6
D_IN=20, L=16, M=40	0.837	0.996	6
D_IN=100, L=20, M=20	0.882	0.999	6
D_IN=50, L=38, M=10	0.867	1.000	6
D_IN=50, L=20, M=30	0.891	1.000	6
D_IN=50, L=14, M=30	0.915	1.000	6
D_IN=50, L=30, M=20	0.875	0.999	6
D_IN=100, L=36, M=10	0.875	0.999	5
D_IN=500, L=2, M=30	0.744	0.997	5
D_IN=1000, L=12, M=10	0.876	1.000	5
D_IN=50, L=40, M=20	0.857	0.998	5
D_IN=100, L=2, M=50	0.747	0.997	5

D_IN=200, L=6, M=30 | 0.867 | 1.000 | 5
D_IN=20, L=12, M=60 | 0.921 | 0.994 | 5
D_IN=100, L=34, M=10 | 0.866 | 1.000 | 5
D_IN=100, L=40, M=10 | 0.863 | 0.999 | 5
D_IN=1000, L=10, M=10 | 0.873 | 0.999 | 5
D_IN=100, L=18, M=30 | 0.835 | 1.000 | 5
D_IN=100, L=14, M=30 | 0.874 | 0.999 | 5
D_IN=20, L=42, M=30 | 0.844 | 1.000 | 5
D_IN=200, L=32, M=10 | 0.866 | 0.999 | 5
D_IN=20, L=30, M=40 | 0.859 | 0.999 | 5
D_IN=20, L=38, M=30 | 0.836 | 0.991 | 5
D_IN=20, L=2, M=70 | 0.737 | 0.997 | 5
D_IN=100, L=10, M=40 | 0.875 | 1.000 | 5
D_IN=50, L=10, M=50 | 0.849 | 0.999 | 5
D_IN=100, L=32, M=20 | 0.835 | 0.999 | 5
D_IN=20, L=58, M=10 | 0.858 | 0.999 | 5
D_IN=50, L=22, M=40 | 0.855 | 0.992 | 5
D_IN=20, L=24, M=50 | 0.838 | 0.990 | 4
D_IN=500, L=24, M=10 | 0.871 | 0.998 | 4
D_IN=200, L=2, M=50 | 0.726 | 0.999 | 4
D_IN=100, L=8, M=50 | 0.850 | 0.999 | 4
D_IN=500, L=6, M=30 | 0.854 | 0.999 | 4
D_IN=50, L=48, M=20 | 0.841 | 1.000 | 4
D_IN=200, L=18, M=30 | 0.859 | 1.000 | 4
D_IN=500, L=14, M=20 | 0.869 | 1.000 | 4
D_IN=50, L=22, M=50 | 0.854 | 0.999 | 4
D_IN=20, L=10, M=70 | 0.907 | 0.998 | 4
D_IN=50, L=42, M=30 | 0.821 | 1.000 | 4
D_IN=50, L=70, M=10 | 0.842 | 0.999 | 4
D_IN=20, L=90, M=10 | 0.839 | 0.999 | 4
D_IN=2000, L=12, M=10 | 0.863 | 1.000 | 4
D_IN=50, L=52, M=20 | 0.863 | 0.999 | 4
D_IN=2000, L=4, M=10 | 0.832 | 0.999 | 4
D_IN=50, L=2, M=200 | 0.787 | 0.995 | 4
D_IN=100, L=50, M=10 | 0.847 | 0.999 | 4
D_IN=20, L=56, M=20 | 0.832 | 1.000 | 4
D_IN=100, L=22, M=40 | 0.851 | 0.991 | 4
D_IN=50, L=32, M=40 | 0.877 | 0.994 | 4
D_IN=2000, L=6, M=10 | 0.853 | 0.999 | 4
D_IN=100, L=14, M=40 | 0.856 | 0.999 | 4
D_IN=100, L=26, M=30 | 0.854 | 1.000 | 4
D_IN=500, L=16, M=20 | 0.865 | 1.000 | 4
D_IN=20, L=54, M=20 | 1.406 | 0.927 | 4
D_IN=500, L=26, M=10 | 0.857 | 1.000 | 4
D_IN=100, L=36, M=20 | 0.850 | 0.999 | 4
D_IN=100, L=40, M=20 | 0.855 | 0.999 | 4
D_IN=50, L=8, M=60 | 0.879 | 0.999 | 4
D_IN=1000, L=20, M=10 | 0.860 | 1.000 | 4
D_IN=20, L=44, M=30 | 0.834 | 0.996 | 4
D_IN=50, L=8, M=100 | 0.904 | 0.999 | 4

APPENDIX B : RAW SCALING DATA FOR L

Slopes for L scaling:

Configuration	Slope	R^2	Points
N=8, D_IN=20, M=10	1.027	0.957	53
N=16, D_IN=20, M=10	1.026	0.991	41
N=8, D_IN=100, M=20	1.021	0.996	37
N=8, D_IN=50, M=30	1.035	0.995	37
N=8, D_IN=100, M=30	1.031	0.995	32
N=10, D_IN=50, M=30	1.036	0.992	32
N=25, D_IN=50, M=10	1.075	0.969	31
N=10, D_IN=100, M=30	1.047	0.993	27
N=10, D_IN=500, M=10	1.047	0.993	27
N=8, D_IN=200, M=30	1.061	0.986	27
N=40, D_IN=20, M=10	1.074	0.920	26
N=25, D_IN=100, M=10	1.071	0.989	26
N=10, D_IN=20, M=60	1.080	0.986	22
N=8, D_IN=1000, M=20	1.061	0.993	22
N=32, D_IN=100, M=10	1.100	0.988	21
N=16, D_IN=200, M=20	1.082	0.989	21
N=32, D_IN=20, M=30	1.164	0.970	21
N=10, D_IN=500, M=30	1.091	0.991	17
N=10, D_IN=2000, M=10	1.093	0.992	17
N=8, D_IN=20, M=80	1.099	0.992	17
N=32, D_IN=200, M=10	1.132	0.986	16
N=16, D_IN=50, M=50	1.123	0.989	16
N=16, D_IN=1000, M=10	1.113	0.989	16
N=16, D_IN=20, M=60	1.142	0.990	16
N=8, D_IN=500, M=50	1.129	0.992	12
N=10, D_IN=20, M=80	1.152	0.992	12
N=8, D_IN=200, M=60	1.128	0.993	12
N=8, D_IN=100, M=70	1.132	0.993	12
N=50, D_IN=100, M=10	1.205	0.983	11
N=25, D_IN=100, M=40	1.189	0.988	11
N=25, D_IN=20, M=60	1.178	0.985	11
N=16, D_IN=2000, M=10	1.172	0.990	11
N=25, D_IN=500, M=20	1.190	0.988	11
N=8, D_IN=50, M=200	1.158	0.993	9
N=8, D_IN=20, M=500	1.182	0.993	9
N=32, D_IN=20, M=200	1.248	0.983	9
N=16, D_IN=20, M=100	1.240	0.991	9
N=16, D_IN=20, M=200	1.207	0.991	9
N=8, D_IN=100, M=500	1.167	0.993	9
N=10, D_IN=5000, M=20	1.222	0.993	7
N=8, D_IN=1000, M=50	1.205	0.994	7
N=10, D_IN=2000, M=30	1.222	0.993	7
N=10, D_IN=500, M=50	1.224	0.993	7
N=16, D_IN=50, M=70	1.283	0.990	6
N=16, D_IN=500, M=40	1.288	0.992	6

```
N=40, D_IN=200, M=20 | 1.339 | 0.989 | 6
N=32, D_IN=50, M=50 | 1.318 | 0.987 | 6
N=16, D_IN=200, M=50 | 1.286 | 0.992 | 6
N=100, D_IN=20, M=10 | 1.407 | 0.991 | 6
N=32, D_IN=200, M=30 | 1.329 | 0.990 | 6
N=32, D_IN=100, M=40 | 1.326 | 0.989 | 6
N=64, D_IN=20, M=30 | 1.383 | 0.986 | 6
N=64, D_IN=50, M=100 | 1.475 | 0.992 | 4
N=128, D_IN=20, M=20 | 1.526 | 0.994 | 4
```

@miscjacot2020neuraltangentkernelconvergence, title=Neural Tangent Kernel : Convergence and Generalization in Neural Networks, author=Arthur Jacot and Franck Gabriel and Clément Hongler, year=2020, eprint=1806.07572, archivePrefix=arXiv, primaryClass=cs.LG, url=https ://arxiv.org/abs/1806.07572,